Lecture 1:

BSc. SOFTWARE ENGINEERING

COMPONENT BASED SOFTWARE ENGINEERING

SSE 2302

Component-Based Software Engineering (CBSE)

INTRODUCTION

Learning Outcomes

Upon successful completion of this course, the student should be able to:

1. Describe the role of Component-Based Software Engineering (CBSE) within the software life cycle.
2. Apply key elements and common methods for CBSE.
3. Design component-based software systems using well-structured design methods.
4. Describe, Compare, contrast and evaluate structured, Object Oriented, data Oriented and formal approaches to component modeling.
5. Conduct a review of CBSE requirements and using best practices to determine the quality of a CBS.
6. Demonstrate the capacity to use a range of software tools in support of CBS.
7. Analyze a software component problem and be able to design and implement an effective program structures to solve it, including appropriate modularity, separation of abstraction and implementation concerns, use of standard design patterns to solve recurring design problems, and use of standard libraries and COTS.
8. Discuss component composition and integration

INTRODUCTION AND DEFINITION

- **Component-Based Software Engineering (CBSE)** is a process that focuses on the design and development of computer-based systems with the use of reusable software components.
- It not only identifies candidate components but also qualifies each component's interface, adapts components to remove architectural mismatches, assembles components into a selected architectural style, and updates components as requirements for the system change.
- The process model for component-based software engineering occurs concurrently with *component-based development.*

Cont.:

- Component-based software engineering (CBSE) is an approach to software development emerged in the 1990's that relies on the reuse of entities called 'software components'.
- It emerged from the failure of object-oriented development to support effective reuse.
- Single object classes are too detailed and specific. Components are more abstract than object classes and can be considered to be stand-alone service providers.
- They can exist as stand-alone entities.

Example

- Suppose a university wants to develop a **Student Management System**. Instead of developing everything from the beginning, developers could reuse components for:
- User authentication
- Student registration
- Payment processing
- Report generation
- Email/SMS notifications
- Database access
- Document generation
- These components are integrated to form the complete system.

CBSE essentials

- **Independent components** specified by their interfaces.
- **Component standards** to facilitate component integration.
- **Middleware** that provides support for component interoperability.
- **A development process** that is geared to reuse.

software engineering design principles:

Apart from the benefits of reuse, CBSE is based on sound **software engineering design principles**:

- Components are independent so do not interfere with each other;
- Component implementations are hidden;
- Communication is through well-defined interfaces;
- One components can be replaced by another if its interface is maintained;
- Component infrastructures offer a range of standard services.

Characteristics of Software Components

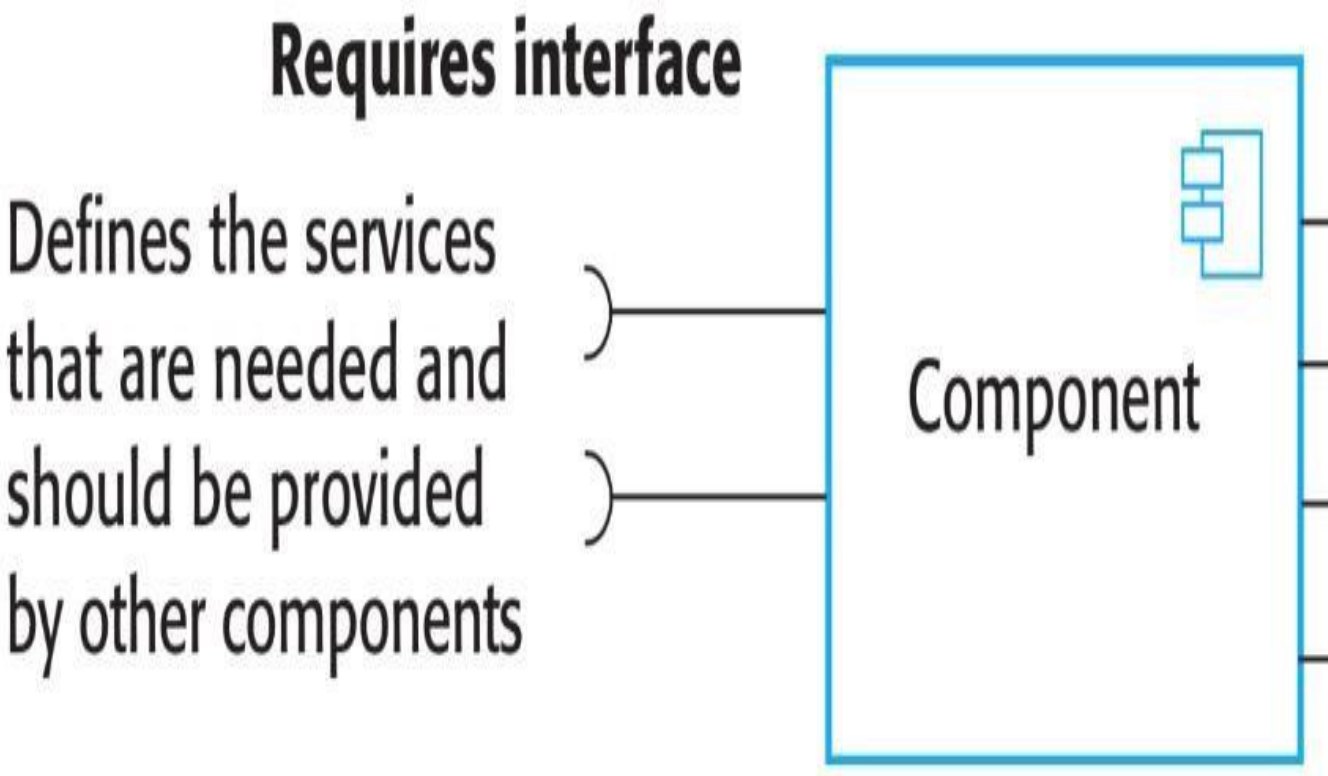
A good software component normally possesses several important characteristics.

1. **Well-defined functionality:** A component should have a clear purpose. For example, a **Payment Component** should focus primarily on payment-related services rather than performing unrelated tasks.
2. **Encapsulation:** The internal implementation of a component is hidden from other components. Other components interact with it through its interface rather than directly accessing its internal code.
3. **Modularity:** A component should represent a relatively independent part of a larger system. A modular system can be divided into manageable parts.
4. **Reusability:** A component should be capable of being used in multiple systems or applications, preferably with little or no modification. For example, an authentication component could be reused in: Student management systems, Hospital systems, Banking systems, E-commerce systems [9]

Cont.:

5. **Replaceability:** A component should ideally be replaceable with another component that provides equivalent services. For example, one payment provider could potentially be replaced by another without redesigning the entire application.
6. **Composability:** Components should be capable of being combined with other components to create larger systems.
7. **Standardized interfaces:** A component should expose clearly defined interfaces that specify how other components communicate with it.
8. **Limited dependencies:** A well-designed component should minimize unnecessary dependence on the internal details of other components.
9. **Deployability:** A component should be capable of being deployed as an identifiable software unit within its target environment.
10. **Documentation:** Reusable components should be accompanied by sufficient documentation describing:

Purpose, Interfaces, Inputs, Outputs, Dependencies, Configuration, Limitations and Usage requirements



Component interfaces:

- **"Provides" interface**
- Defines the services that are provided by the component to other components. This interface, essentially, is the component API. It defines the methods that can be called by a user of the component.
- **"Requires" interface**
- Defines the services that specifies what services must be made available for the component to execute as specified. This does not compromise the independence or deployability of a component because the 'requires' interface does not define how these services should be provided.

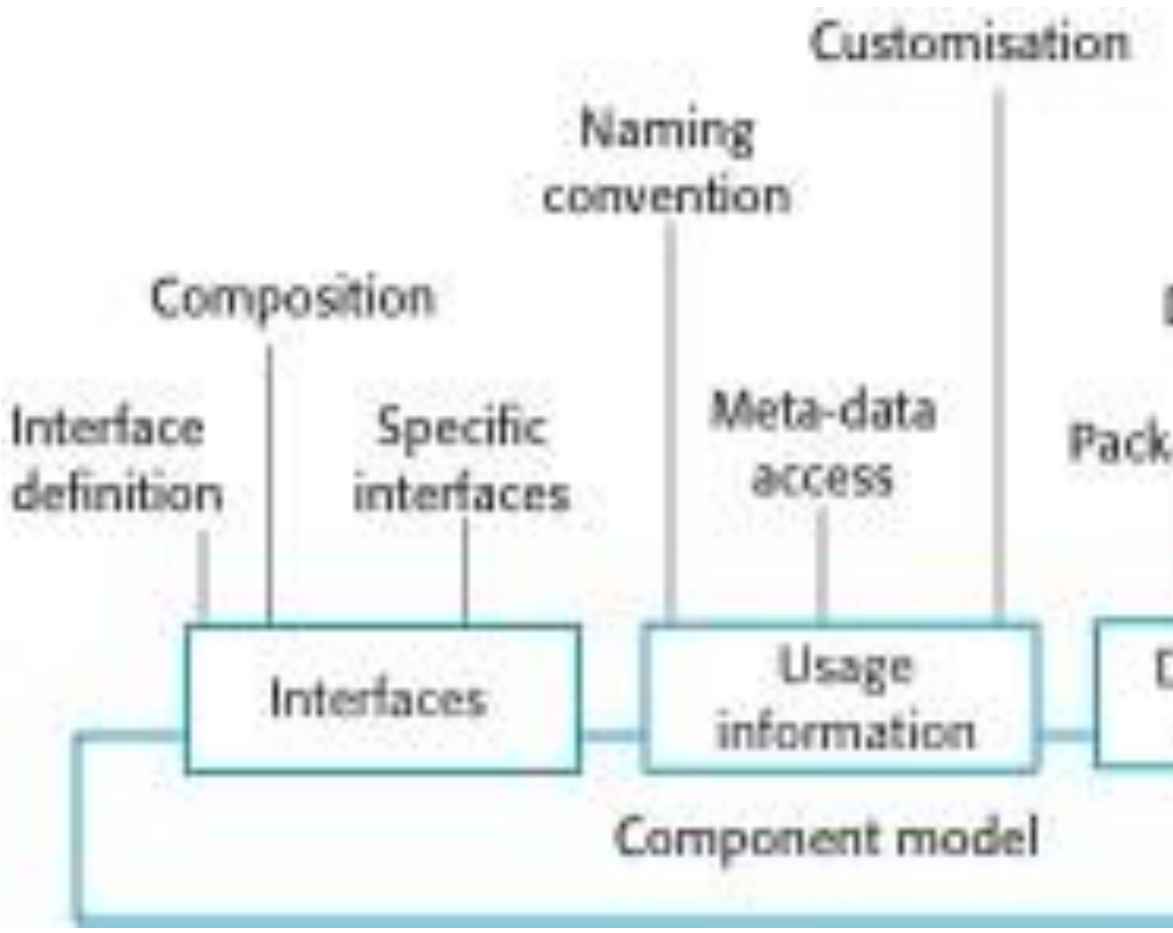
Component interfaces:

Important CBSE concepts

1: Software component

- A **software component** is a self-contained, reusable unit of software that provides specific functionality through well-defined interfaces.
- Examples include:
- Login component
- Payment component
- Shopping-cart component
- Reporting component
- Authentication component
- Database-access component

Basic elements of a component model



Component interface

An **interface** specifies how other software can interact with a component.

- It defines:
  - Services provided
  - Inputs required
  - Outputs produced
  - Methods/functions available
  - Communication protocols
- The implementation of the component can remain hidden.

Cont:

3 Component composition

- **Component composition** is the process of combining two or more components to create a larger software system.
- For example:
- **Login Component + Student Component + Payment Component + Reporting Component → Student Management System**

4 Component repository

- A **component repository** is a storage facility containing reusable software components and their associated documentation.
- A repository may contain:
- Source code, Executable components, Libraries, APIs, Documentation, Version information, Dependencies, Licensing information

Cont:

5 Component adaptation

- Sometimes an existing component does not exactly meet the requirements of a new system.
- **Component adaptation** involves modifying, wrapping, configuring, or otherwise adjusting a component so that it can work within the target system.

6 Component certification

- Before using a component, developers should determine whether it meets required standards.
- Certification may consider:
- Functionality, Security, Performance, Reliability, Compatibility, Maintainability, Quality

Software Components and ComponentBased Systems

Software components

A software component should ideally have the following characteristics:

a) **Reusability:** It can be used in more than one software system.

b) **Modularity:** It represents a relatively independent part of a system.

c) **Encapsulation:** Internal implementation details are hidden from users of the component.

d) **Well-defined interfaces:** The component communicates with other components through clearly defined interfaces.

e) **Replaceability:** A component can potentially be replaced by another component providing equivalent services.

f) **Composability:** The component can be combined with other components.

g) **Independence:** A component should have limited dependence on the internal implementation of other components.

Example: E-commerce system

An online shopping system may consist of: The complete system is produced by integrating these components.

Component	Function
Authentication	Login and user verification
Product Catalogue	Displays products
Shopping Cart	Manages selected products
Payment	Processes payments
Inventory	Tracks stock
Notification	Sends messages
Reporting	Generates reports

Evolution of Software Reuse

- Software reuse did not begin with CBSE. It has developed gradually over several decades.

1 Code reuse: Early developers reused pieces of source code by copying functions or modules from previous programs.

- **Problems**
- Difficult maintenance, Duplicate code, Strong dependencies andDifficult version management

2 Function and procedure libraries

- Developers began creating libraries containing commonly used functions.
- Examples include:

- Mathematical libraries, File-handling libraries and String-processing libraries This reduced repeated programming.

Cont:

3 Object-oriented reuse: Object-oriented programming introduced concepts such as:

 Classes, Objects,
Inheritance, Encapsulation,
Polymorphism These
concepts improved software
reuse.

4 Framework reuse

- Developers began reusing entire application frameworks rather than individual functions.
- Examples include frameworks for:
- Web applications, Mobile applications, Enterprise applications and User interfaces

5 Component-based reuse

- CBSE moved reuse toward **independently developed components** with standardized interfaces.
- Components could be developed by different teams or organizations and integrated into applications.

Cont:

- **6 Service-oriented and modern reuse:** Modern software development extends reuse through:
- Web services, APIs, Microservices, Cloud services, Package managers, Opensource libraries and Software-as-a-Service (SaaS) Thus, software reuse has evolved from:
- **Code → Functions → Classes → Frameworks → Components → Services/APIs → Microservices**

Objectives of CBSE

 The major objectives of Component-Based Software Engineering include:

1. **Promote software reuse:** Reduce the need to develop functionality that already exists.
2. **Reduce development time:** Using existing components can significantly reduce programming effort.
3. **Reduce development cost:** Less development work can result in lower software development costs.
4. **Improve software quality:** Well-tested and mature components can reduce defects.
5. **Increase productivity:** Developers can concentrate on system-specific functionality instead of repeatedly developing common features.
6. **Improve maintainability:** Modular systems are generally easier to modify and maintain.

7 **Support flexibility:** Components can potentially be replaced, upgraded, or rearranged without redesigning the entire system.

8 **Encourage standardization:** CBSE promotes the use of: Standard interfaces, APIs, Communication protocols, Design standards

Benefits of Component Reuse

1 **Reduced development time:** Existing components eliminate the need to implement functionality from scratch.

Example: A developer can use an existing authentication library instead of developing login functionality from the beginning.

2 **Reduced cost:** Reuse reduces: Programming effort, Testing effort, Maintenance effort, Development resources

3 **Improved reliability:** A component that has already been extensively tested may be more reliable than newly written code. However, developers must still verify that it is suitable for the intended environment.

4 **Improved software quality:** Reusable components may have undergone: Testing, Code review, Performance evaluation, Security assessment, Real-world usage

Cont.:

5 **Faster development:** Developers can assemble systems faster by combining existing components.

6 **Easier maintenance:** Changes can sometimes be made to individual components without modifying the entire system.

7 **Scalability:** Components can be independently improved or replaced to support changing system requirements.

8 **Better productivity:** Developers spend more time solving business-specific problems and less time implementing common functionality.

9 **Consistency:** Reusing the same components across different systems can promote consistent functionality and behaviour.

Challenges and Limitations of CBSE

 Although CBSE provides significant advantages, it also presents challenges.

1 **Component identification:** Finding a component that exactly matches the required functionality can be difficult. A component repository may contain hundreds or thousands of components.

2 **Component compatibility:** Components may have incompatible: Interfaces, Data formats, Programming languages, Operating systems, Technologies, Communication protocols

3 **Integration difficulties:** Integrating components developed by different vendors or teams can be complicated.

4 **Limited customization:** A reusable component may not provide exactly the functionality required. Developers may have limited ability to modify it.

5 **Security risks:** Third-party components may contain: Security vulnerabilities, Malicious code, Outdated dependencies and Poor security practices. Therefore, components must be evaluated before deployment.

Cont:

6 **Dependency management:** One component may depend on several other components. This can create a **dependency chain** that makes maintenance difficult.

7 **Version problems:** An updated component may introduce changes that cause existing applications to fail. This is commonly referred to as **version incompatibility** or dependency conflict.

8 **Performance issues:** A reusable component may perform well in one environment but poorly in another.

9 **Licensing issues:** Third-party and open-source components may have different licenses. Organizations must ensure that components are used according to their licensing conditions.

10 **Lack of suitable components:** Sometimes no existing component satisfies the requirements. In such situations, developers may still need to develop a component from scratch.

Role of CBSE in the Software Development Life Cycle

- CBSE can influence almost every stage of the **Software Development Life Cycle (SDLC).**

1 Requirements analysis: Developers identify:

- What the system should do, Required functionality, Required interfaces and Existing components that may satisfy requirements
- **Example:** A requirement for "online payment" may lead developers to search for an existing payment component or API.

2 System analysis : Developers analyze available components and determine:

- Whether they satisfy requirements, Compatibility, Cost, Quality, Security, Licensing and Performance

3 System design: The system architecture is designed around selected components.

- Developers determine: Component responsibilities, Interfaces, Communication mechanisms, Dependencies and Data flow

Cont:

4 **Component selection:** Suitable reusable components are identified and evaluated.

Selection criteria may include: Functionality, Quality, Compatibility, Security, Performance, Cost, Vendor support, Documentation and Licensing

5 **Component adaptation:** Selected components may require: Configuration, Wrapping, Interface conversion, Parameter adjustment and Minor modification

6 **Implementation:** Instead of implementing the entire system from scratch, developers implement only the components that are unavailable or require customization.

7 **Integration:** Components are connected through their interfaces.

Cont:

8 **Testing:** The system is tested at several levels:

Component testing: Tests individual components.

Integration testing: Tests interactions between components.

System testing: Tests the complete system.

Acceptance testing: Determines whether the system satisfies user requirements.

9 **Deployment:** The completed component-based system is deployed in the target environment.

10 **Maintenance:** During maintenance, individual components can potentially be: Updated, Replaced, Removed, Reconfigured and Reused in other systems

CBSE versus Traditional Software Development

Aspect	Traditional Development
Development approach	Build most functionality from scratch
Software reuse	Limited or moderate
Development time	Usually longer
Cost	Can be higher
Quality	Depends heavily on development team
Flexibility	Can be highly customized
Integration	Less emphasis on third-party integration

Cont: CBSE versus Traditional Software Development

Aspect	Traditional Development
Testing	Focuses mainly on newly developed software
Dependenci es	Usually fewer external dependencies
Security	Team controls most source code
Maintenance	Changes may involve large sections of code
Architecture	Often designed around newly developed modules
Main focus	Software construction

Key Principles of CBSE

A successful CBSE approach should follow several principles:

1. **Design for reuse:** Components should be designed so that they can be used in different systems.
2. **Reuse whenever appropriate:** Developers should first investigate whether suitable existing components are available.
3. **Program to interfaces:** Systems should depend on well-defined interfaces rather than the internal implementation of components.
4. **Encapsulate implementation:** The internal details of a component should be hidden from other components.
5. **Minimize dependencies:** Components should have as few unnecessary dependencies as possible.
6. **Evaluate components before reuse:** Components should be evaluated for quality, compatibility, security and licensing.
7. **Support replaceability:** A component should ideally be replaceable by another component providing equivalent services.

Advantages and Disadvantages Advantages

- Faster development
- Reduced development cost
- Increased productivity
- Improved reuse
- Potentially improved reliability
- Easier maintenance
- Standardization
- Reduced duplication
- Easier system evolution

Disadvantages

- Difficult component selection
- Compatibility problems
- Integration complexity
- Security risks
- Licensing concerns
- Dependency management
- Version conflicts
- Limited customization
- Performance uncertainty
- Suitable components may not exist

Recommended Reading

- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.
- Szyperski, C., Gruntz, D., & Murer, S. (2002). *Component Software: Beyond ObjectOriented Programming* (2nd ed.). Addison-Wesley.
- Heineman, G. T., & Councill, W. T. (Eds.). (2001). *Component-Based Software Engineering:*
  Putting the Pieces Together. Addison-Wesley.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- Crnkovic, I., & Larsson, M. (Eds.). (2002). *Building Reliable Component-Based Software*
  Systems. Artech Hous

Lecture 2:

COMPONENT BASED SOFTWARE ENGINEERING

Basic concepts

INTRODUCTION

- A central idea in **Component-Based Software Engineering (CBSE)** is that large software systems can be constructed by combining smaller, reusable software components.
- For components to work together successfully, developers must clearly understand:
  - What a component does
  - What services it provides
  - What services it requires
  - How other components communicate with it
  - What conditions must be satisfied before using it
  - What results should be produced
  - What quality attributes it must satisfy

This leads to three closely related concepts:

- **Components + Interfaces + Specifications = Well-defined building blocks for software systems** A component provides functionality, an interface defines how that functionality is accessed, and a specification describes what the component is expected to do. 2

Software Components

Definition of a Software Component

- A **software component** is an independently deployable and reusable unit of software that provides a well-defined set of services through specified interfaces.
- In simpler terms:
- A software component is a self-contained piece of software designed to perform a particular function and interact with other software through clearly defined interfaces.
- Examples include:
- Authentication component
- Payment-processing component
- Database component

Encapsulation and Modularity

Encapsulation

- **Encapsulation** is the practice of combining data and operations into a unit while restricting direct access to the internal implementation. In CBSE, encapsulation means that users of a component do not need to know how the component performs its operations.
- Example
  - Suppose we have a **Payment Component**.

    A client may call:

    processPayment(amount, account)
  - The client does not need to know whether the component internally uses:
    - Database queries
    - Encryption
    - Network communication
    - Transaction validation
    - Error handling
    - The implementation remains hidden.

Benefits of encapsulation

- Reduces complexity
- Protects internal data
- Reduces coupling
- Makes maintenance easier
- Supports component reuse
- Allows implementation changes without affecting clients

Modularity

- **Modularity** is the process of dividing a large software system into smaller, manageable and relatively independent units called modules or components.

Example

- A banking application could be divided into:
- Banking System
  - *Customer Management*
  - *Account Management*
  - *Transactions*
  - *Loans*
  - *Payments*
  - *Notifications*

- *Reporting*

- Each module performs a specific responsibility.

Good modular design

- A good modular system should have:
- **High cohesion** –
- Elements within a component are strongly related.
- A component should focus on one closely related set of responsibilities.
- **Low coupling** –
- Components have minimal unnecessary dependencies.
- A component should not depend heavily on the internal implementation of other components.

Reusability and Replaceability

Reusability: Reusability is the ability of a software component to be used in multiple applications or contexts without requiring substantial redevelopment.

- Example
  - An **authentication component** may provide: login(), logout(), changePassword(), resetPassword()
  - The same component could be used in:
  - University systems
  - Hospital systems
  - Government systems
  - E-commerce applications

Factors that improve reusability

- A component is more reusable when it has:
- Clear functionality
- Well-defined interfaces
- Minimal dependencies
- Good documentation
- Configurability
- Standard communication mechanisms
- Good portability
- Proper error handling

Replaceability

- **Replaceability** refers to the ability to replace one component with another component that provides equivalent functionality without requiring major changes to the rest of the system.

- Importance of replaceability
  - Prevents vendor lock-in
  - Supports system evolution
  - Simplifies upgrades
  - Allows technology changes
  - Improves maintainability
  - Provides flexibility

Component Granularity

- **Component granularity** refers to the size and scope of functionality contained within a software component.
- Components can be:
- Fine-grained
- Medium-grained
- Coarse-grained

Fine-grained components

- A fine-grained component provides a small amount of functionality.
- **Example**
- A component that performs: CalculateTax()

Advantages

1. Highly reusable
2. Easy to understand
3. Easy to test
4. Can be combined flexibly

Disadvantages

1. Large number of components may be required
2. More communication between components
3. Integration may become complicated

Coarse-grained components

A coarse-grained component provides a relatively large business function.

- **Example**
- A complete **Student Registration Component** might provide:
- Student search
- Course selection
- Registration
- Fee calculation
- Registration confirmation

Choosing appropriate granularity

- There is no universally correct component size.
- Developers should consider:
- Reusability
- Performance
- Complexity
- Communication overhead
- Maintainability
- Business requirements
- Deployment requirements

General principle

- Components should be large enough to provide meaningful functionality but small enough to remain manageable and reusable.

Black-Box and White-Box Components

- **Black-box components:** A **black-box component** is a component whose internal implementation is hidden from the developer using it.
- The developer knows:
- What services it provides
- How to invoke the services
- Required inputs
- Expected outputs
- But does not need to know how it works internally.

 Example

- A developer uses an external payment API:

- makePayment(amount, customer)
- The developer does not need access to the payment provider's internal source code.

Cot:

Advantages

- Strong encapsulation
- Easy reuse
- Reduced complexity
- Implementation can change independently
- Promotes loose coupling **Disadvantages**
- Limited customization
- Difficult to modify internal behaviour
- Internal defects may be difficult to diagnose
- Dependency on component provider

White-Box Components

- A **white-box component** is a component whose internal implementation is visible and potentially modifiable by the developer.
- The developer may have access to:
- Source code
- Internal classes
- Algorithms
- Data structures
- Configuration
- **Example:** An organization downloads an open-source reporting library, examines its source code and modifies it to meet specific requirements.

Cont:

- **Advantages**
- Greater customization
- Internal behaviour can be modified
- Easier to investigate some problems
- Can be optimized for a specific application
- **Disadvantages**
- Greater maintenance responsibility
- Higher risk of modifying the component incorrectly
- More knowledge of internal implementation is required
- Modifications may make future upgrades difficult

Component Interfaces

Definition

- A **component interface** is a defined set of operations or services through which a component communicates with other components or clients.
- The interface describes **what the component can do**, without necessarily describing **how it does it**.

 Example

- A Student component may expose:
- registerStudent()
- updateStudent()
- findStudent()
- deleteStudent()
- Other components use these services through the interface.

2 Importance of interfaces

- Interfaces:
- Promote encapsulation
- Reduce coupling
- Support interoperability
- Enable component replacement
- Define communication rules
- Make components easier to reuse
- Separate implementation from usage

Provided and Required Interfaces

 A component can have two important types of interfaces:

1 Provided interface

 A **provided interface** describes services that a component makes available to other components.

2 Required interface

 A **required interface** describes services that a component needs from other components in order to perform its own functionality.

Example

- Think of a restaurant.
- **Provided services:** meals and drinks the restaurant offers.
- **Required services:** electricity, water, food supplies and payment services that the restaurant needs.
- A software component similarly **provides services** while **requiring services** from its environment.

Component Contracts

- **Definition:** A **component contract** is a formal or informal agreement that specifies the obligations, expectations and behaviour associated with a software component and its interactions with other components.
- A contract answers questions such as:
- What does the component provide?
- What inputs are acceptable?
- What output should be produced?
- What conditions must hold before an operation?
- What conditions will hold after an operation?
- What errors can occur?
- What quality requirements apply?

Design by Contract

- Component contracts are closely associated with **Design by Contract (DbC)**.
- A contract normally consists of:
- **1. Preconditions :**Conditions that must be true **before** an operation executes.
- **2. Postconditions:** Conditions that must be true **after** successful execution.
- **3. Invariants:** Conditions that must remain true throughout the lifetime or valid operation of a component.

Preconditions

- **Definition :**A **precondition** is a condition that must be satisfied before a component operation can be executed correctly.
- Example
  - Consider: withdraw(amount), Possible preconditions:, Account must exist., Account must be active., Amount must be greater than zero. , Sufficient balance must be available.
  - Therefore: Preconditions:
  - 1. amount > 0
  - 2. account exists
  - 3. balance >= amount
  - If these conditions are not satisfied, the operation may fail or reject the request.

Postconditions

- **Definition:** A **postcondition** is a condition that should be true after an operation has completed successfully.
- For the withdrawal operation:
- withdraw(5000)

- Possible postconditions:
- New balance = Old balance - 5000
- Transaction is recorded
- Withdrawal receipt is generated

- Example
  - If the account initially has KSh 20,000:
  - Initial balance = KSh 20,000
  - Withdrawal = KSh 5,000
  - Final balance = KSh 15,000
  - The postcondition specifies what should be true after successful execution.

Functional Specifications

- **Definition:** A **functional specification** describes **what a software component is expected to do**. It identifies: Services provided, Inputs Outputs Processing behaviour, Business rules and Error conditions

Example: Student Registration Component

- Functional requirements may specify that the component must:
- Register a new student.
- Assign a unique student number.
- Allow students to select courses.
- Check course prerequisites.
- Record registration.
- Generate registration confirmation.
- These describe **what the component does**.

Non-Functional Specifications

- **Definition:** A **non-functional specification** describes the quality attributes, constraints and operational characteristics of a component rather than its specific business functions.
- Common non-functional requirements include:
- Performance, Security, Reliability, Availability, Scalability, Usability
- Maintainability
- Portability
- Interoperability
- **22.2 Examples**
- For a payment component:

Example

Functional specification

- The component shall process a payment and return a transaction confirmation.

Non-functional specifications

The com

p
o
n
e
n
t
s
h
a
l
l
:

- Process 95% of valid requests within 2 seconds.
- Encrypt sensitive information during transmission.
- Maintain an availability of 99.9%.
- Support 1,000 concurrent requests.
- Record all successful transactions for auditing.
- The first statement describes **what it does**; the others describe **how well or under what constraints it must operate**.

Component Specification

- A complete component specification should provide enough information for another developer to determine whether the component can be used.
- A component specification may include:

1. Component name: Example: Student Registration Component
2. Purpose: What problem does the component solve?
3. Provided interfaces: Services offered.
4. Required interfaces: Services required from other components.
5. Functional specification: What the component does.

Cont.: Component Specification

6. Non-functional specification: Performance, security, reliability and other constraints.
7. Preconditions: Conditions that must be satisfied before operations.
8. Postconditions: Expected results after operations.

9. Dependencies: Other software, libraries, databases or services required.
10. Error conditions: Possible failures and how they are reported.
11. Configuration requirements: Settings that must be provided.
12. Version and compatibility information: Supported platforms, versions and environments

Example of a Complete Component Specification

- **Component: Student Registration Component**
- **Purpose:** Provides services for registering students for academic programmes and courses.
- **Provided interface** ***registerStudent(), updateStudent(), findStudent(), registerCourse(), cancelCourse()***
- **Required interface**
- ***StudentDatabase, AuthenticationService, CourseManagementService, PaymentService***

Functional specifications

The component shall:

- Create a student registration record.
- Validate student information.
- Check course availability.
- Verify prerequisites.
- Record selected courses.
- Generate registration confirmation.

Non-functional specifications

- The component shall:
- Protect student information.
- Provide acceptable response times.
- Support concurrent users.
- Maintain transaction integrity.
- Be available during registration periods.

Importance of Interfaces in CBSE

- Interfaces are particularly important because they allow components to interact without requiring knowledge of their internal implementation.

- **Without interfaces**
- Component A ——————▶ Internal code of Component B
- This creates strong coupling.
- **With interfaces**
- Component A ——————▶ Interface B ——————▶ Component B
- This promotes:

Loose coupling, Encapsulation, Replaceability, Reusability, Maintainability

Interface Compatibility

- For two components to work together, their interfaces must be compatible.
- Compatibility may involve:

Data compatibility

- Both components agree on:
- Data types
- Formats
- Units
- Encoding

Operation compatibility

- The required operation must exist in the provided interface.

Protocol compatibility

- Components must agree on communication mechanisms.

Behavioural compatibility

The component must behave according to the expectations of its client.

Interface Mismatch

- An **interface mismatch** occurs when a required interface does not match the interface provided by another component.
- **Example**
- Component A requires: getCustomerName(customerID) Component B provides:
- findCustomer(customerNumber)
- Although the functionality may be similar, the interfaces do not directly match

Adapters and Wrappers

- An **adapter** is a component or mechanism that converts one interface into another compatible interface.
- **Example**
- Suppose an application expects:

- sendSMS(phone, message)
- but an existing SMS service provides:
- transmit(number, text)
- An adapter can translate between the two interfaces.
- This allows the existing component to be reused without modifying its internal implementation.

Good Practices in Component Specification

1. When specifying a component, developers should:
2. Clearly define its purpose.
3. Specify all provided interfaces.
4. Specify all required interfaces.
9. Specify functional behaviour.
10. Specify non-functional requirements.
11. Document dependencies.
12. Define possible error conditions.
13. State compatibility requirements.

5. Define inputs and outputs.

14.Keep specifications precise and unambiguous.

6. Document preconditions.

15.

5.Update documentation when the component 7. Document postconditions. changes.

References

1. **Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.**
2. **Szyperski, C., Gruntz, D., & Murer, S. (2002). *Component Software: Beyond ObjectOriented Programming* (2nd ed.). Addison-Wesley Professional.**
3. **Heineman, G. T., & Councill, W. T. (Eds.). (2001). *Component-Based Software Engineering: Putting the Pieces Together*. Addison-Wesley.**

Lecture 3:

COMPONENT BASED

SOFTWARE ENGINEERING

Specification of Components

Meaning of component specification;

- A **component specification** is essentially the **blueprint** of a software component. It defines:
- **Purpose** → What the component is supposed to achieve.
- **Behavior** → How it should act under different conditions.
- **Interfaces** → How it interacts with other components (inputs, outputs, protocols).
- **Constraints** → Any limitations such as performance requirements, resource usage, or compatibility rules.
- Think of it like the **instruction manual** for a Lego block in a larger system: it tells you exactly what the block does, how it connects to others, and what rules must be followed so everything fits together smoothly.

Component specification-Intro

- Component specification refers to the detailed description of the functionalities, features, and requirements of a software component. A software component is an independent module that performs a specific task or set of tasks within a larger software system.
- Component specification defines the interface, behavior, and constraints of a component and provides a blueprint for its development, testing, and integration into the system.
- Component specification is an essential part of the software development process, as it helps to ensure that the components are designed and implemented correctly and meet the system's requirements.
- It provides a clear understanding of the component's purpose, functionality, and dependencies, which helps developers to build software that is modular, scalable, and maintainable.

cont:.:

- The component specification typically includes the following information:
- **Functional requirements:** The specific tasks that the component must perform and the expected outcomes.
- **Non-functional requirements:** The performance, reliability, security, and usability requirements of the component.
- **Interface requirements:** The input and output parameters, data structures, and communication protocols used by the component.

- **Dependencies:** The other components or systems that the component relies on.
- **Constraints:** The limitations or restrictions on the component's design and implementation, such as hardware or software compatibility issues.

Cont.:

- Component specification is usually created by software architects or system analysts, who work closely with stakeholders to identify the system's requirements and design the component's architecture.
- The specification is then reviewed and approved by the development team , who use it as a reference for building and testing the component.
- Any changes or updates to the specification are documented and communicated to all stakeholders to ensure that everyone is on the same page.

Cont.:

- Overall, component specification is a critical aspect of software development that helps to ensure that the components are designed and implemented correctly and meet the system's requirements.
- It provides a clear and detailed description of the component's purpose, functionality, and constraints, which helps developers to build software that is modular, scalable, and maintainable.

Cont.:

- Component-based software development (CBSD) builds applications using independent, reusable software building blocks instead of writing everything from scratch.
- Component requirements describe what these pieces must do and how they must behave to fit together.
- Component requirements fall into a few key areas that guide how you pick, build, and connect software pieces.

Component Requirements

- **Component requirements** are the functional and non-functional specifications that a software component must satisfy so that it can be developed, reused, integrated, and maintained within a software system.

Functional Requirements

- **Specific Services:** Define the exact tasks or business logic a component handles, such as processing a payment or logging a user in.
- **Provided Interfaces:** Specify the methods, inputs, and outputs a component offers for other parts of the system to use.

- **Required Interfaces:** List any external services or dependencies the component needs from other components to do its job

Non-Functional Requirements

- **Independence:** Ensure the component operates on its own with minimal reliance on specific outside code.
- **Scalability:** Require individual components to handle varying workloads or scale up independently.
- **Reliability and Quality:** Set standards for how stable the component must be, since system quality depends on reliable parts

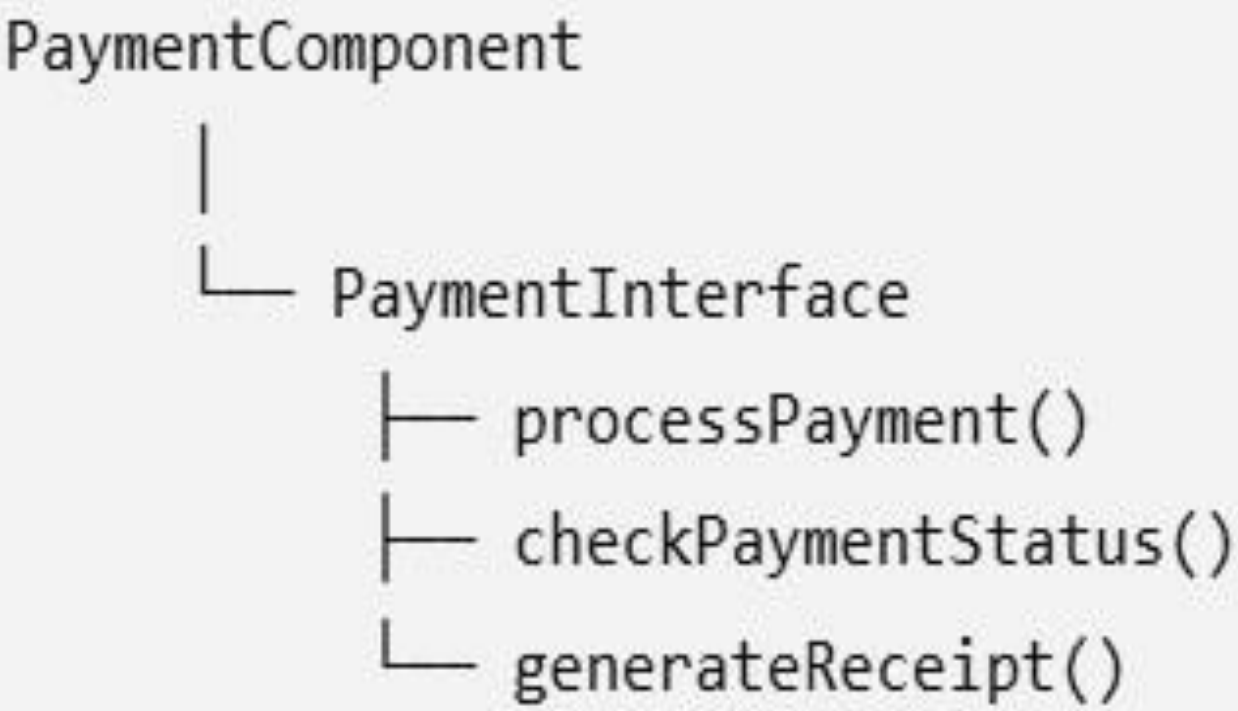
Integration and Design Requirements

- **Standardization:** Require the component to follow a standard model for metadata, communication, and deployment.
- **Composable Contracts:** Demand that all interactions happen strictly through public interfaces so internal code can stay hidden.
- **Adaptability:** Ensure components can be adjusted or fitted with glue code to prevent architectural mismatches

Provided Interfaces

- a **provided interface** is the set of services, operations, or functionalities that a software component **offers to other components or applications**.
- It defines **what the component can provide**, without necessarily revealing how those services are implemented.

Example



Consider a PaymentComponent

nentthatprovidesOthercomponentscancallt

h
e
s
e
o
p
e
r
a
t
i
o
n
s
t
h
r
o
u
g
h
t
h
e
P
a
y
m
e
n
t
I
n
t
e
r
f
a
c

e

w

i

t

h

o

u

t

k

n

o

w

i

n

g

t

h

e

i

n

t

e

r

n

a

l

i

m

p

l

e

m

e

n

t

a

t

i

o

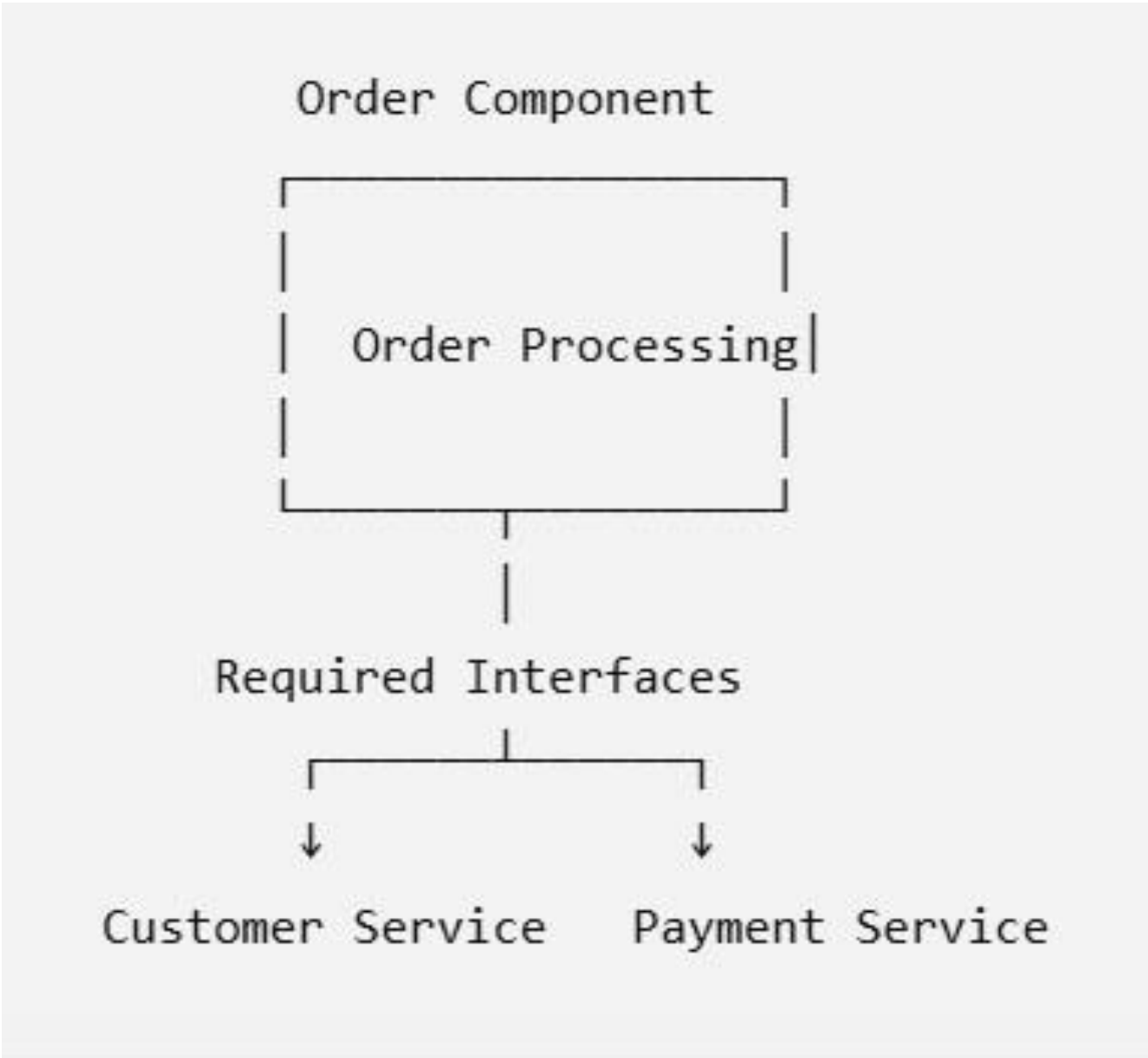
n
o
f
t
h
e
p
a
y
m
e
n
t
c
o
m
p
o
n
e
n
t
.

Characteristics of a Provided Interface

- **Defines available services** – specifies what functionality the component makes available.
- **Specifies operations/methods** – identifies the functions that other components can invoke.
- **Hides implementation details** – users of the interface do not need to know how the service is implemented.
- **Supports component integration** – allows other components to interact with the component in a standard way.
- **Promotes reusability** – components can be reused as long as they provide the required interface.
- **Defines input and output** – specifies parameters accepted and values or responses returned.

Required Interfaces

A **required interface** is a set of services or operations that a software component



needs from other components in order to perform its functions

Example

- Suppose we have an **Order Component** that needs customer information and payment services:

Cont.:

- The Order Component may require:
- getCustomerDetails()
- validateCustomer()
- processPayment()
- checkPaymentStatus()

- The Order Component **does not provide these services itself**. It depends on other components that provide them.

Characteristics of Required Interfaces

- **Identify dependencies** – show services that a component needs from other components.
- **Define expected services** – specify the operations the component expects to access.
- **Support component integration** – connect a component to services provided by other components.
- **Promote loose coupling** – the component depends on an interface rather than the internal implementation of another component.
- **Enable component substitution** – another component can provide the required services as long as it implements the expected interface.

Key Roles of Interfaces in CBSE

- **Enabling Parallel Development:** Clearly defined interfaces allow distinct software engineering teams to develop separate components concurrently. As long as the interface contract remains stable, internal changes will not disrupt parallel workflows.
- **Interchangeability and Upgrades:** Since a system depends on the interface and not the internal mechanics, any component can be cleanly replaced or updated with a newer version—provided it complies with the established interface rules.
- **Facilitating System Composition:** During integration, software engineers construct systems by connecting the "required" interface of one component directly to the "provided" interface of another.
- **Mitigating Incompatibility:** In cases where two third-party components must interact but feature mismatching signatures, developers introduce **adapters**. The adapter acts as a bridge, translating calls to resolve the interface conflict

Lecture 4:

COMPONENT BASED

SOFTWARE ENGINEERING

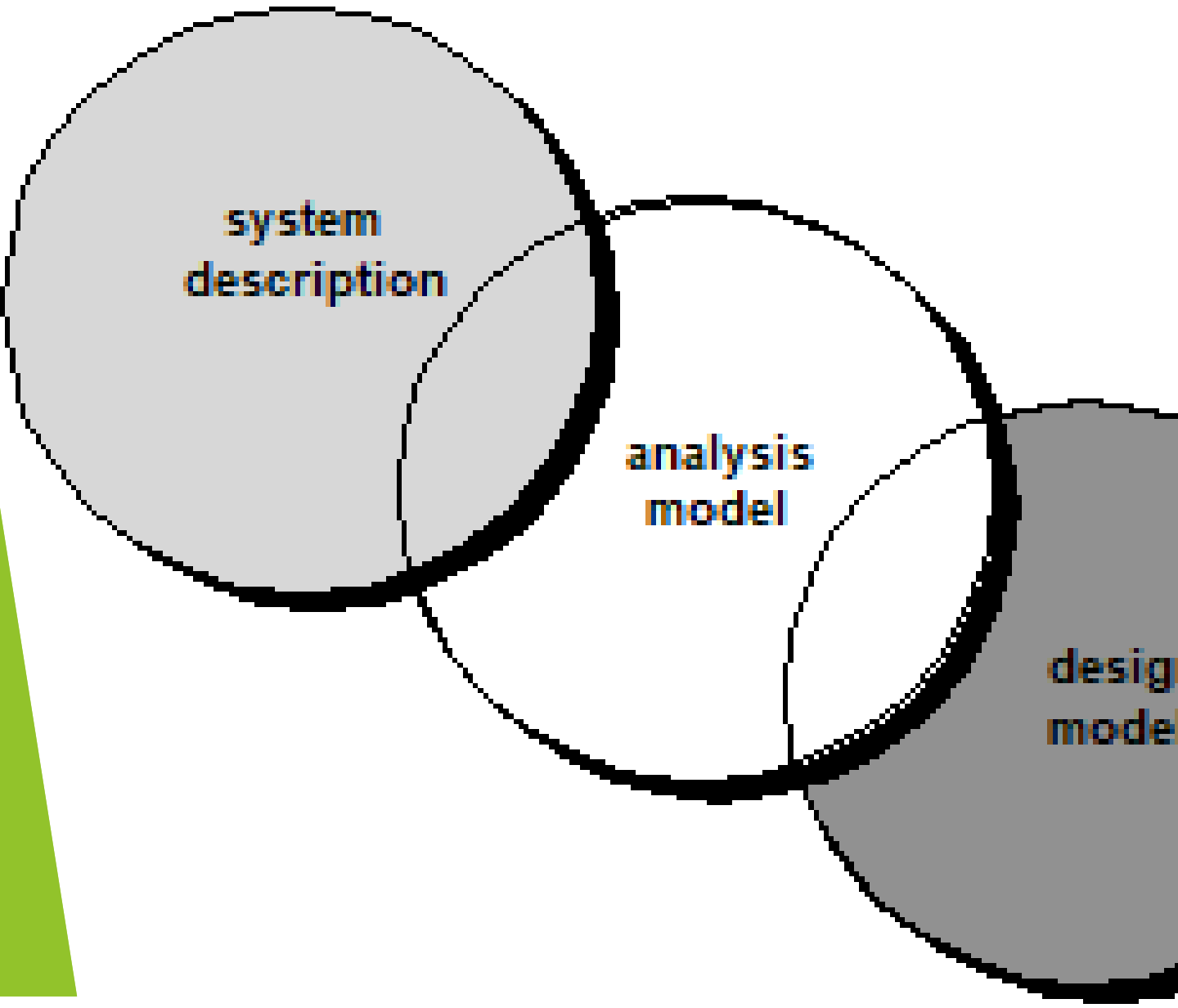
Component Modelling

Requirements Analysis

- *Requirement Analysis* results in the specification of software's operational characteristics; indicates software's interface with other system element; and establishes constraints that software must need.
- Throughout analysis modeling the software engineer's primary focus is on *what* not *how*.
- RA allows the software engineer (called an *analyst* or *modeler* in this role) to:
- Elaborate on basic requirements established during earlier requirement engineering tasks
- Build models that depict user scenarios, functional activities, problem classes and their relationships, system and class behavior, and the flow of data as it is transformed.

Overall Objectives and Philosophy

- The analysis model must achieve three primary objectives:
- To describe what the customer requires
- To establish a basis for the creation of a software design, and
- To define a set of requirements that can be validated once the software is built.
- The analysis model bridges the gap between a system-level description that describes overall functionality as it is achieved by applying software, hardware, data, human, and other system elements and a software design that describes the software application architecture.


system
description
analysis
model
desig
model

Analysis Rules of Thumb

- The model should focus on requirements that are visible within the problem or business domain. The level of abstraction should be relatively high.
- Each element of the analysis model should add to an overall understanding of software requirements and provide insight into the information domain, function and behavior of the system.
- Delay consideration of infrastructure and other non-functional models until design.
- For example, a database may be required, but the classes necessary to implement it, the functions required to access it, and the behavior that will be exhibited as it is used should be considered only after problem domain analysis has been completed.

Cont.:**Analysis Rules of Thumb**

- Minimize coupling throughout the system.
- The level of interconnectedness between classes and functions should be reduced to a minimum.
- Be certain that the analysis model provides value to all stakeholders.
- Each constituency has its own use for the model.
- Keep the model as simple as it can be.
- Ex: Don't add additional diagrams when they provide no new information.
- Only modeling elements that have values should be implemented.

Domain Analysis

- Software domain analysis is the identification, analysis, and specification of common requirements from a specific application domain, typically for reuse on multiple projects within that application domain . . . [Object-oriented domain analysis is] the identification, analysis, and specification of common, reusable capabilities within a specific application domain, in terms of common objects, classes, subassemblies, and frameworks . . . ***Donald Firesmith***
- Define the domain to be investigated.
- Collect a representative sample of applications in the domain.
- Analyze each application in the sample.
- Develop an analysis model for the objects

Analysis Modeling Concepts and Approaches

- One view of analysis modeling, called ***structural analysis***, considers data and the processes that transform the data as separate entities.
- Data objects are modeled in a way that defines attributes and relationships.
- Processes that manipulate data objects are in a manner that shows how they transform data as data objects flow through the system.
- A second approach to analysis modeling called ***object-oriented analysis*** focuses on the definition of classes and the manner in which they collaborate with one another to affect customer requirements. UML and the Unified Process are predominantly Object Oriented.

Data Modeling Concepts

1. **Data Objects:** is a representation of almost any composite information that must be processed by software. By composite, we mean something that has a number of different properties and attributes.
2. **Data Attributes:** *define* the properties of a data object and take one of three different characteristics. They can be used to:
   - Name an instance of the data object.
   - Describe the instance, or
   - Make reference to another instance in another table.
3. ***Relationships:*** Indicates “connectedness”; a "fact" that must be "remembered" by the system and cannot or is not computed or derived mechanically
   - several instances of a relationship can exist
   - objects can be related in many different ways
   - We can define a set of object/relationship pairs that define the relevant relationships. For example:
   - A person *owns* a car. A person is *insured to drive* a car.

Object-Oriented Analysis

- The intent of Object-Oriented Analysis (OOA) is to define all classes (and the relationships and behavior associated with them that are relevant to the problem to be solved.
- To accomplish this, a number of tasks must occur:

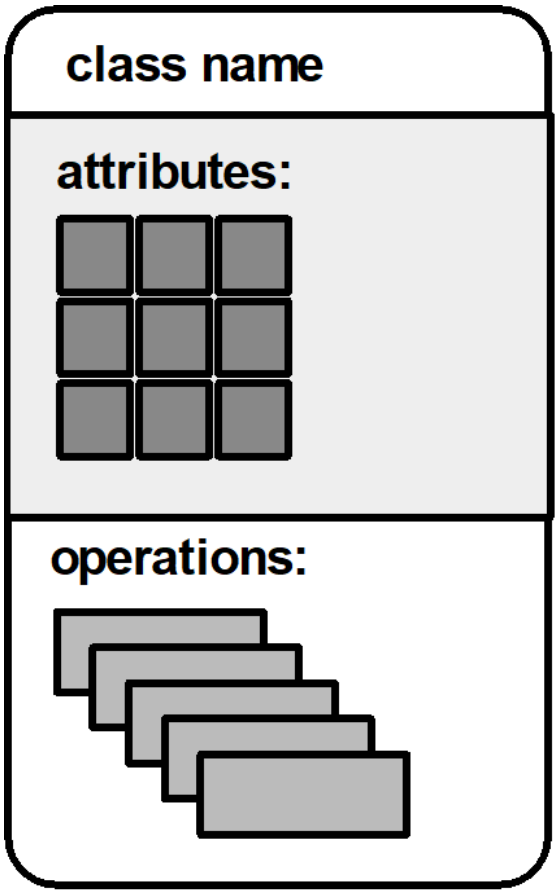
1. Basic user requirements must be communicated between the customer and the software engineer.
2. Classes must be defined.
3. A class hierarchy is defined

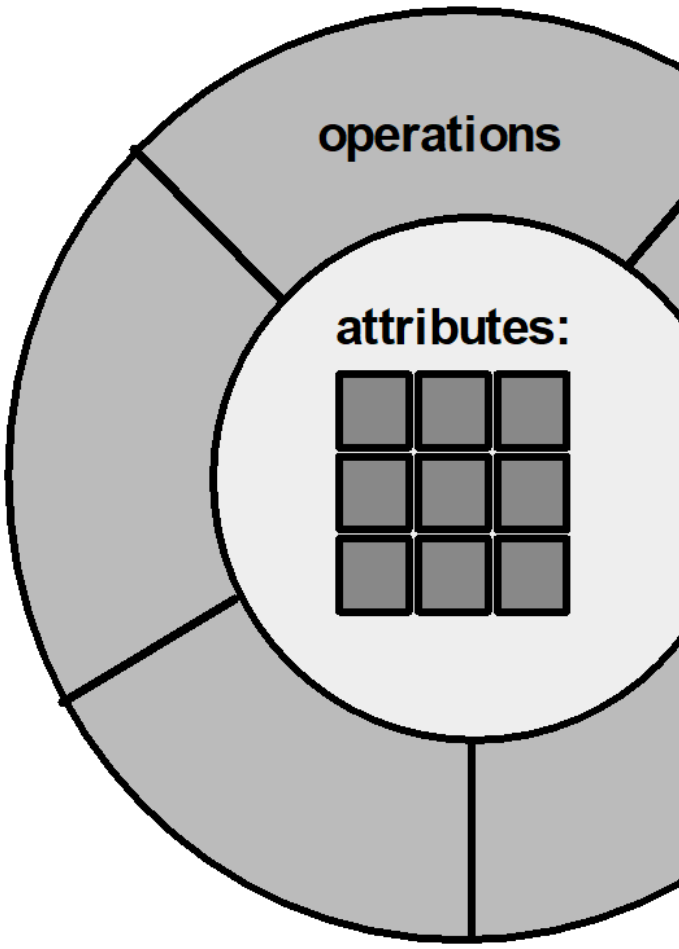
4. Object-to-object relationships should be represented.
5. Object behavior must be modeled.
6. 1 – 5 are repeated iteratively until the model is complete.

Cont.:

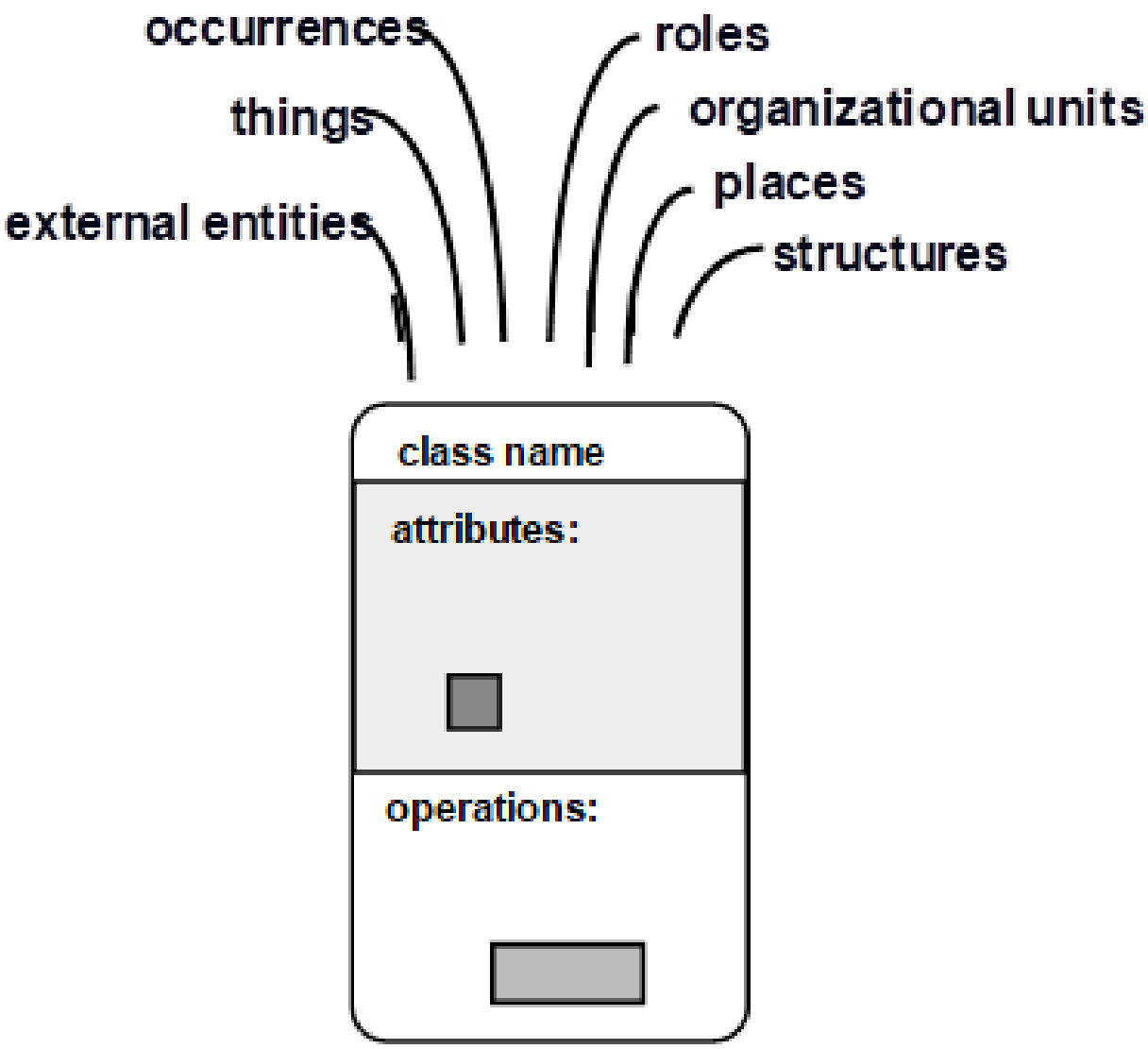
- OOA builds a class-oriented model that relies on an understanding of OO concepts.
- Classes and objects
- Attributes and operations
- Encapsulation and instantiation
- Inheritance
- Object-Oriented thinking begins with the definition of a class, often defined as:
- template
- generalized description
- “blueprint” ... describing a collection of similar items
- a metaclass (also called a superclass) establishes a hierarchy of classes once a class of items is defined, a specific instance of the class can be identified.

Building a class


class name
attributes:
operations:


operations
attributes:

Building a class



Scenario-Based Modeling

- "[Use-cases] are simply an aid to defining what exists outside the system (actors) and what should be performed by the system (use-cases)." Ivar Jacobson
- The concept is relatively easy to understand- describe a specific usage scenario in straightforward language from the point of view of a defined actor.

(1) Writing Use-Cases

(1) What should we write about? : Inception and elicitation provide us the information we need to begin writing use cases.

(2) How much should we write about it?

(3) How detailed should we make our description?

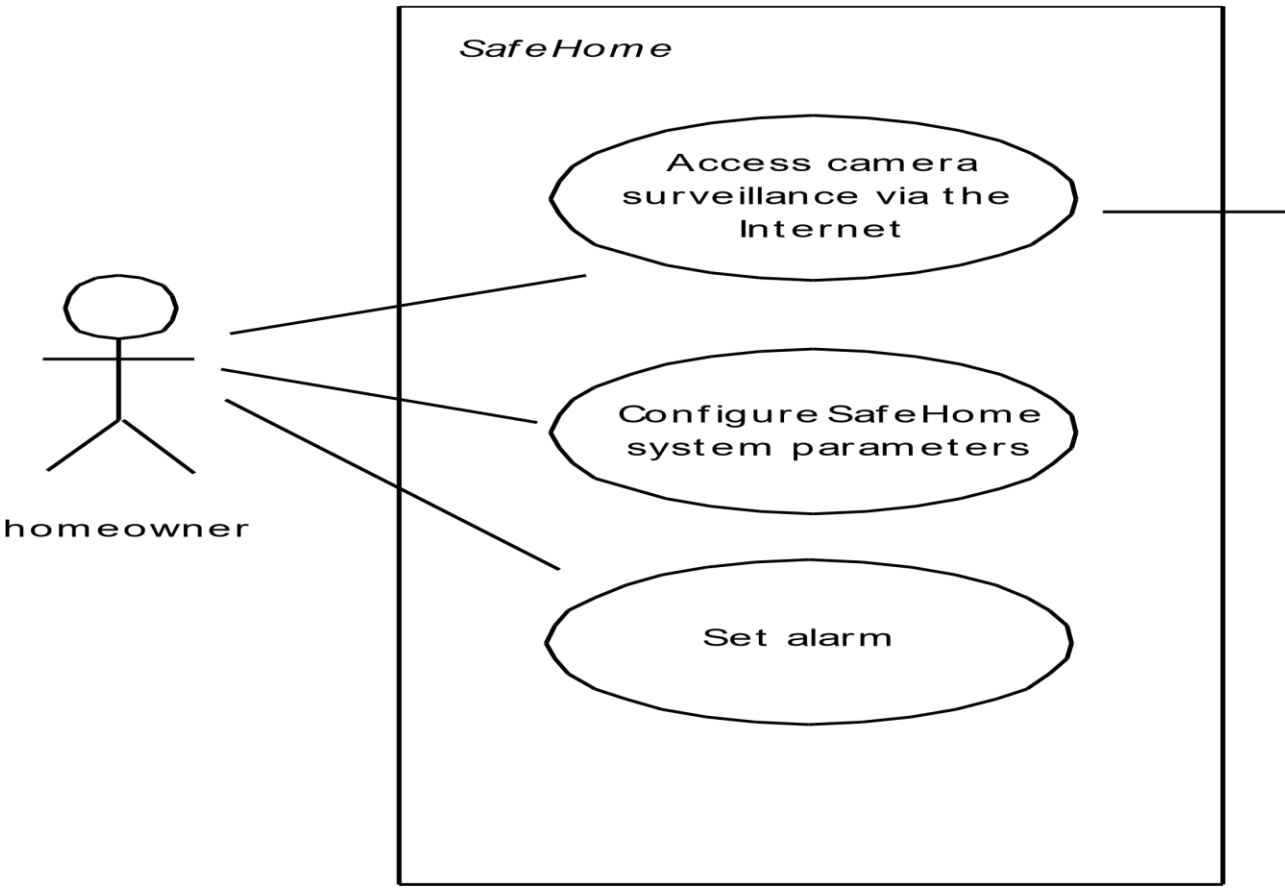
(4) How should we organize the description?

- Use Cases:
- A scenario that describes a "thread of usage" for a system.
- ***Actors*** represent roles people or devices play as the system functions.
- ***Users*** can play a number of different roles for a given scenario.

(2) Developing an Activity Diagram-Nutshel

- What are the main tasks or functions that are performed by the actor?
- What system information will the actor acquire, produce or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

Example



(3) **Swimlane Diagrams**

- The UML *swimlane diagram* is a useful variation of the activity diagram and allows the modeler to represent the flow of activities described by the user-case and at the same time indicate which actor or analysis class has responsibility for the action described by an activity rectangle.
- Responsibilities are represented as parallel segments that divide the diagram vertically, like the lanes in a swimming pool.

(4) Flow-Oriented Modeling

- Represents how data objects are transformed as they move through the system.
- A *data flow diagram* (DFD) is the diagrammatic form that is used to complement UML diagrams.
- Considered by many to be an 'old school' approach, flow-oriented modeling continues to provide a view of the system that is unique.
- The DFD takes an input-process-output insight into system requirements and flow.
- Data objects are represented by labeled arrows and transformations are represented by circles (called *bubbles)*.

Creating a Data Flow Model

- The DFD diagram enables the software engineer to develop models of the information domain and functional domain at the same time.

- As the DFD is refined into greater levels of detail, the analyst performs an implicit functional decomposition of the system.
- **Guidelines**:
- The level 0 data DFD should depict the software/system as a single bubble.
- Primary I/O should be carefully noted.
- Refinement should begin by isolating candidate processes, data objects, and data stores.
- All arrows and bubbles should be labeled with meaningful names.
- Information flow continuity must be maintained from level to level.
- One bubble at one time should be refined.
- Information continuity must be maintained at each level as DFD level is refined. This mean that input and output at one level must be the same as input and output at a refined level.

(5) Class-Based Modeling

- **Identifying Analysis Classes**
- Identify analysis classes by examining the problem statement
- Use a "grammatical parse" to isolate potential classes
- Identify the attributes of each class
- Identify operations that manipulate the attributes
- *Analysis Classes* manifest themselves in one of the following ways:

Cont.:

- *External entities* (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
- *Things* (e.g., reports, displays, letters, signals) that are part of the information domain for the problem.
- *Occurrences or events* (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
- *Roles* (e.g., manager, engineer, salesperson) played by people who interact with the system.
- *Organizational units* (e.g., division, group, and team) that are relevant to an application.

- *Places* (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
- *Structures* (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or related classes of objects.

(6) Creating a Behavioral Model

u The behavioral model indicates how software will respond to external events or stimuli. To create the model, the analyst must perform the following steps:

1. Evaluate all use-cases to fully understand the sequence of interaction within the system.
2. Identify events that drive the interaction sequence and understand how these events relate to specific objects.
3. Create a sequence for each use-case.
4. Build a state diagram for the system.
5. Review the behavioral model to verify accuracy and consistency

State Representations

- In the context of behavioral modeling, two different characterizations of states must be considered:
- the state of each class as the system performs its function and
- the state of the system as observed from the outside as the system performs its function
- The state of a class takes on both passive and active characteristics.
- A *passive state* is simply the current status of all of an object's attributes.
- The *active state* of an object indicates the current status of the object as it undergoes a continuing transformation or processing.

The States of a System

- State—a set of observable circumstances that characterizes the behavior of a system at a given time
- State transition—the movement from one state to another
- Event—an occurrence that causes the system to exhibit some predictable form of behavior
- Action—process that occurs as a consequence of making a transition

UML2 Structure diagrams

- Structure diagrams show the static structure of the system and its parts on different abstraction and implementation levels and how they are related to each other.
- The elements in a structure diagram represent the meaningful concepts of a system, and may include abstract, real world and implementation concepts, there are seven types of structure diagram as well as behaviour representation diagrams
- Behavior representation diagrams show the dynamic behavior of the objects in a system, which can be described as a series of changes to the system over time, there are seven types of behavior diagrams

Momponents modelling

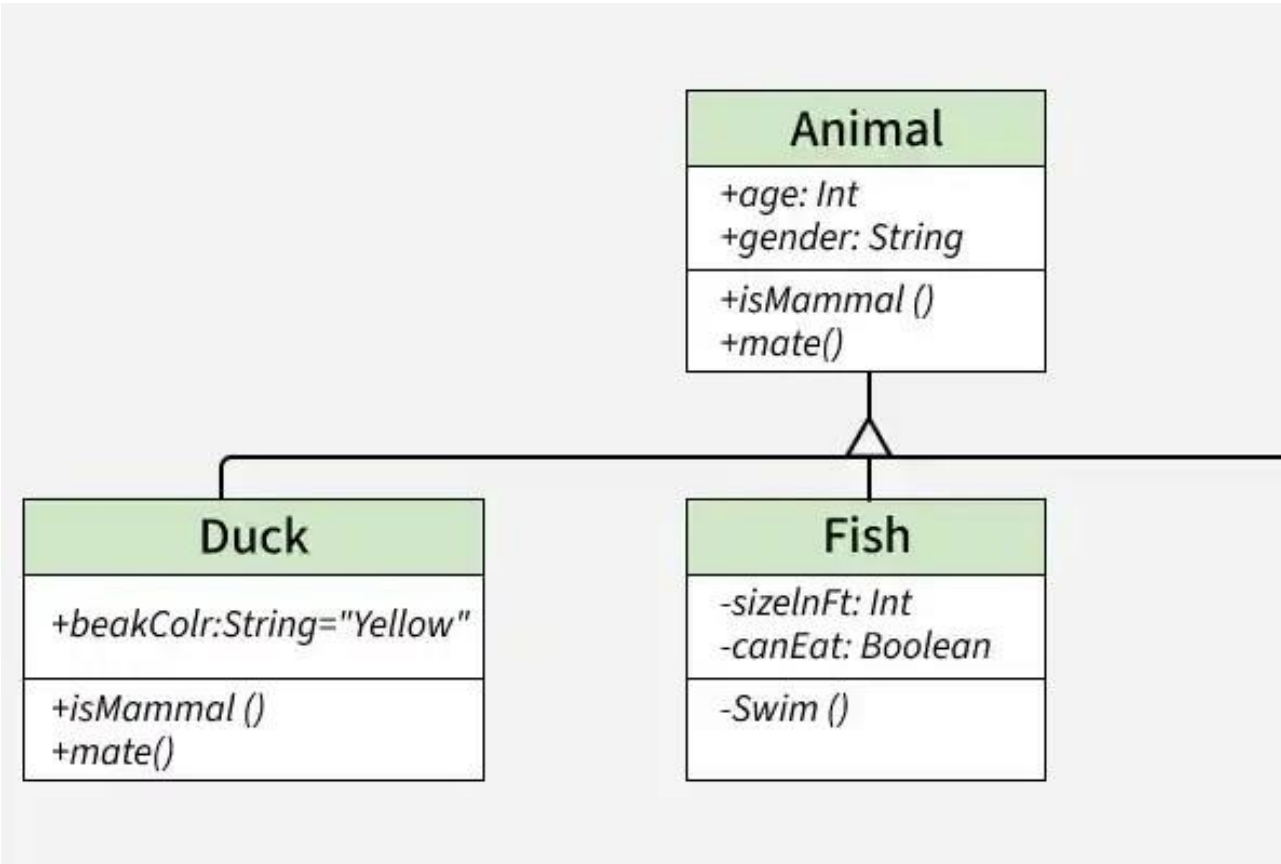
Examples

Implementation representation	**Behavior representation diagrams**
Class Diagram	u Use Case Diagram
Component Diagram	u Activity Diagram
Deployment Diagram	u State Machine Diagram
Object Diagram	u Sequence Diagram
Package Diagram	u Communication Diagram
Composite Structure Diagram	u Interaction Overview Diagram
Profile Diagram	u Timing Diagram

1. Class Diagram

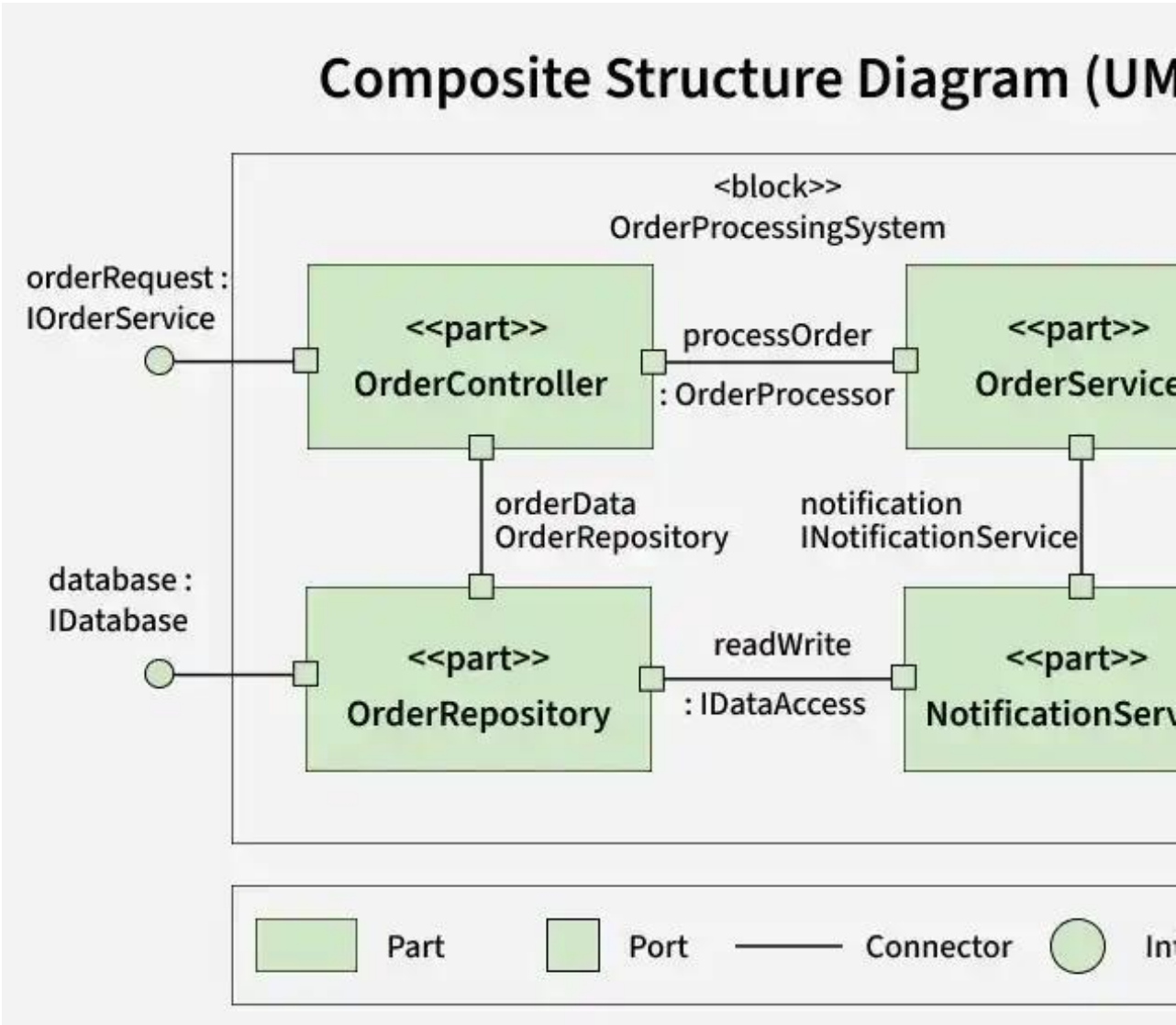
- A class diagram is the most widely used UML diagram and a fundamental part of object-oriented software design. It represents the static structure of a system by showing its classes, attributes, methods, and relationships between classes.
- Class diagrams help visualize the structure of a system and its classes.
- They show relationships between different classes, such as association, inheritance, and dependency.

Example



2. Composite Structure Diagram

- A composite structure diagram represents the internal structure of a class or other structured classifier and shows how its internal parts interact with each other. It provides a detailed view of the parts, ports, and connectors that make up the classifier.
- It shows the internal parts and their relationships, including ports and connectors.
- It provides a detailed view of a classifier's internal structure, unlike a class diagram, which focuses on the overall class.

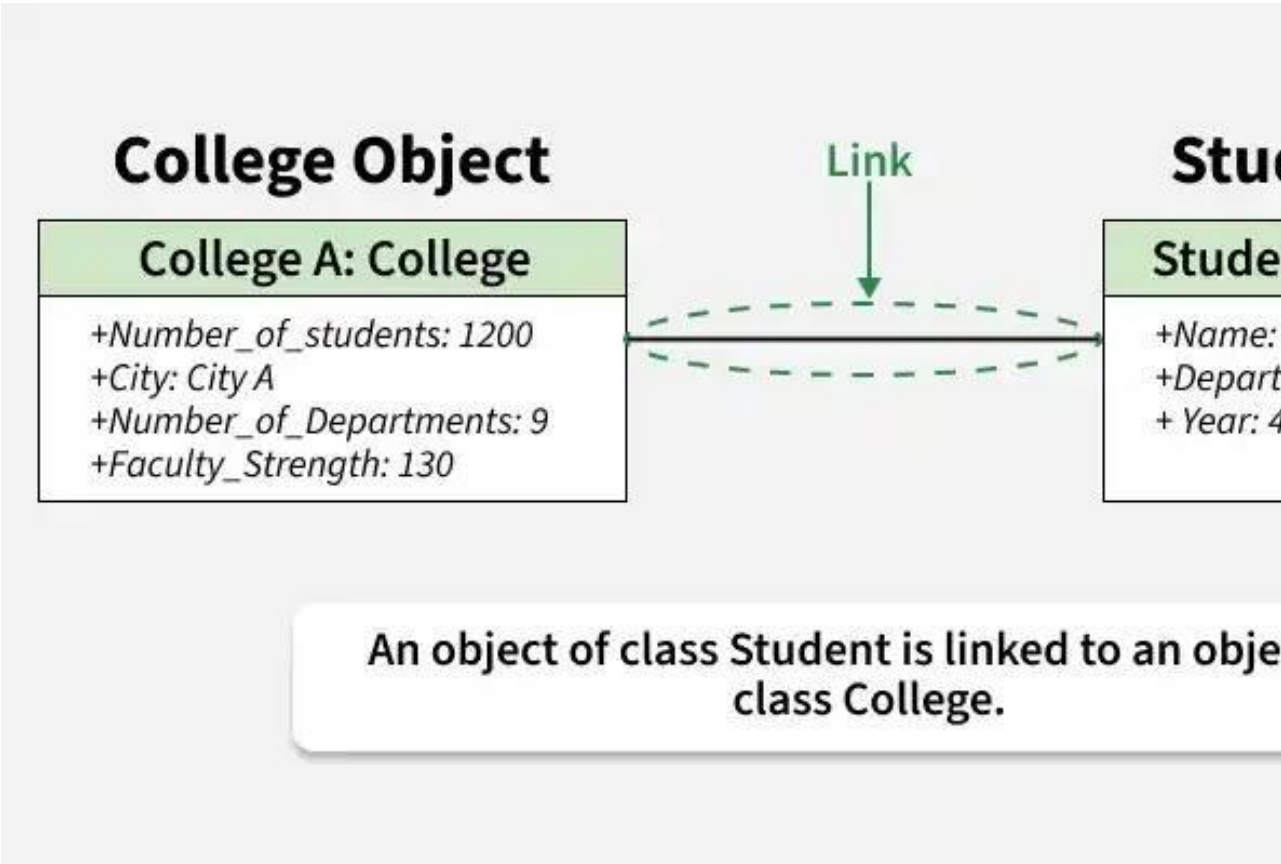


Example: *Consider an Order Processing System where OrderController sends orders to OrderService, which uses OrderRepository to access the database and NotificationService to send notifications. Thus, the diagram shows the internal parts of the system and how they interact with each other.*

3. Object Diagram

- An object diagram represents a snapshot of a system at a particular point in time by showing instances of classes and the relationships between them. It helps visualize the actual state of objects after classes have been instantiated.
- Object diagrams show specific instances of classes and their relationships.
- They provide a snapshot of the system at a particular point in time, unlike class diagrams, which show the general class structure.

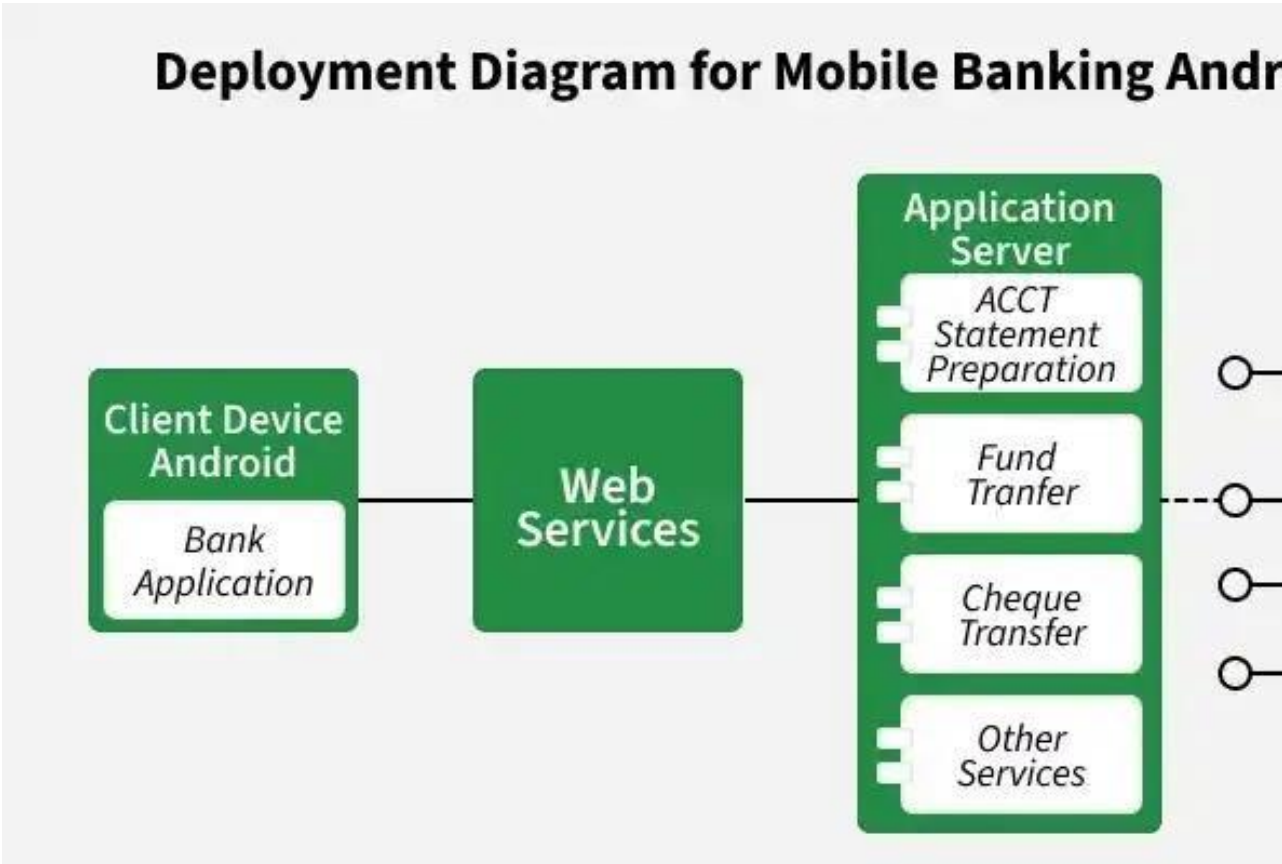
Example



4. Deployment Diagram

- A deployment diagram represents the physical arrangement of hardware and software in a system. It shows how software artifacts are deployed on and distributed across physical or virtual nodes.
- Deployment diagrams show the distribution of software artifacts across hardware or execution environments.
- They are useful for systems deployed across multiple machines with different configurations.

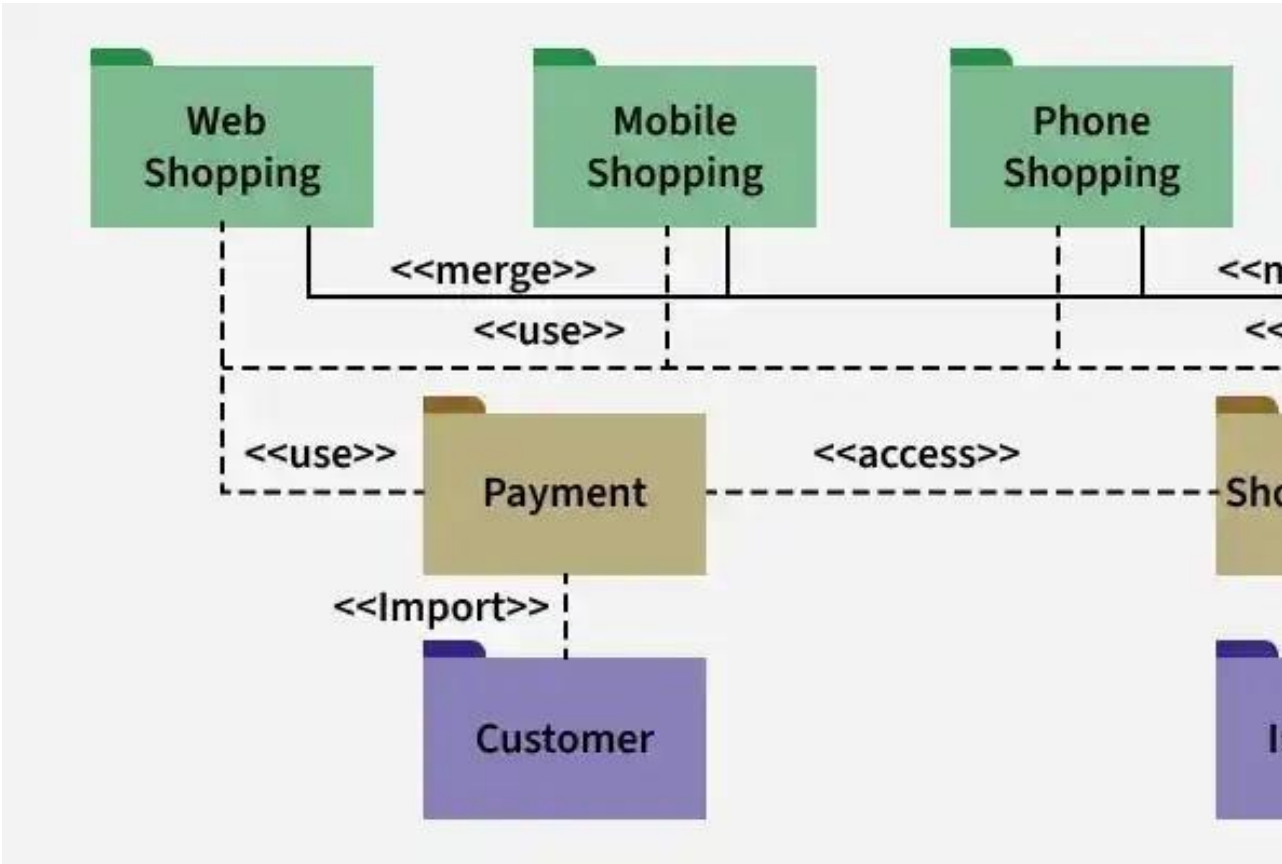
Example



5. Package Diagram

- We use Package Diagrams to depict how packages and their elements have been organized. A package diagram simply shows us the dependencies between different packages and internal composition of packages.
- Packages help us to organize UML diagrams into meaningful groups and make the diagram easy to understand.
- They are primarily used to organize class and use case diagrams.

Example



Components Diagram

- A component diagram is a collection of vertices and arcs and commonly contain:
- components
- Interfaces and dependency
- Aggregation
- Generalization
- Association
- Realization relationships.
- Notes and constraints.

CONT.: Component Diagram

- Component: A deployable unit of code within the system.
- Component Diagram: A diagram that shows relations between various Components and Interfaces.
- A component diagram breaks down the actual system under development into various high levels of functionality.
- Each component is responsible for one clear aim within the entire system and only interacts with other essential elements on a need-to-know basis.

Momponents modelling

Cont.: Component Diagram

- A component diagram represents the organization and dependencies of the physical or replaceable components of a software system. It helps visualize how components are structured and how they communicate through interfaces.
- Component diagrams show the **structural relationships and dependencies** between system components.
- They are useful for **complex and distributed systems**, including microservices, to represent service boundaries, APIs, and communication.

MODELLING COMPONENTS IN UML

- Unified Modeling Language, is a standardized modeling language consisting of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems.
- The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

USING UML

- In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components or software systems.
- It's used to represent the architectures of the software components and the dependencies between them.
- Such as those software components including run-time components, executable components also the source code components.

IMPORTANCE OF UML

- The UML is a very important part of developing object-oriented software and the software development process.
- The UML uses mostly graphical notations to express the design of software projects.
- Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software. In this article, we will give you detailed ideas about what is UML, the history of UML and a description of each UML diagram type, along with UML examples.

Basic Concepts of Component Diagram

(i) A component : represents a modular part of a system that encapsulates its contents and whose manifestation is replaceable within its environment.

u In UML 2, a component is drawn as a rectangle with optional compartments stacked

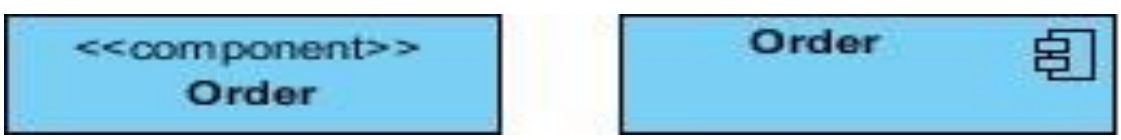


vertically. It can be represented

uA rectangle with the component's name

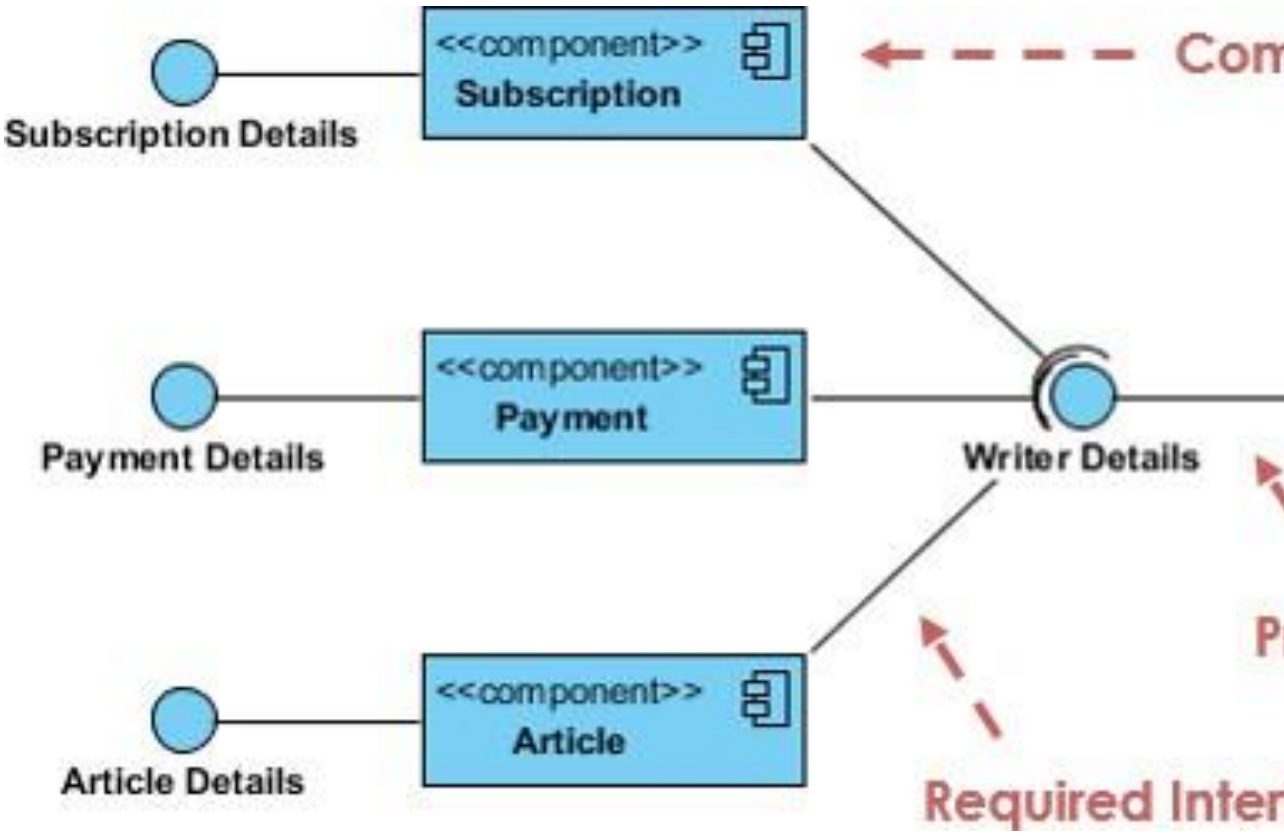
uA rectangle with the component icon

uA rectangle with the stereotype text and/or icon



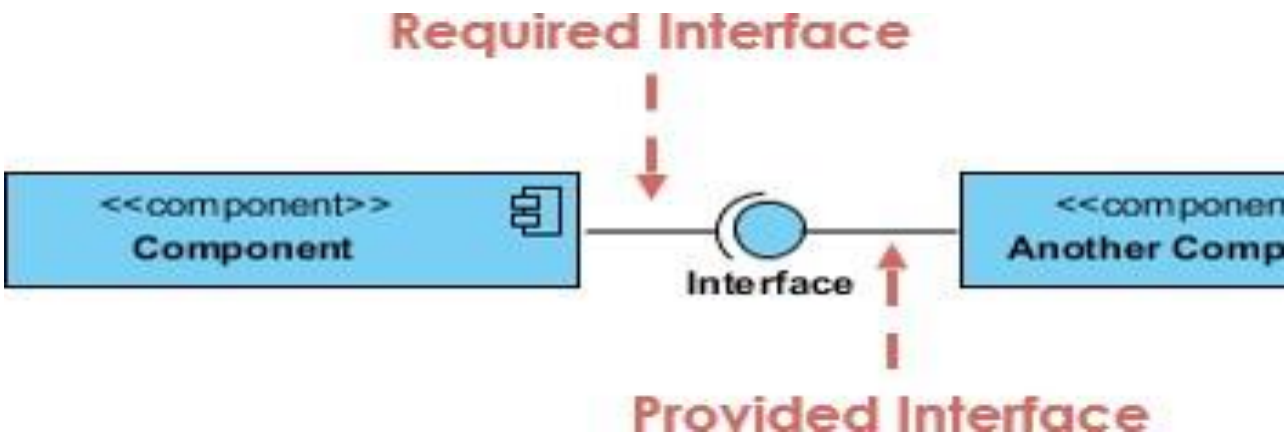
Syntax

EXAMPLE

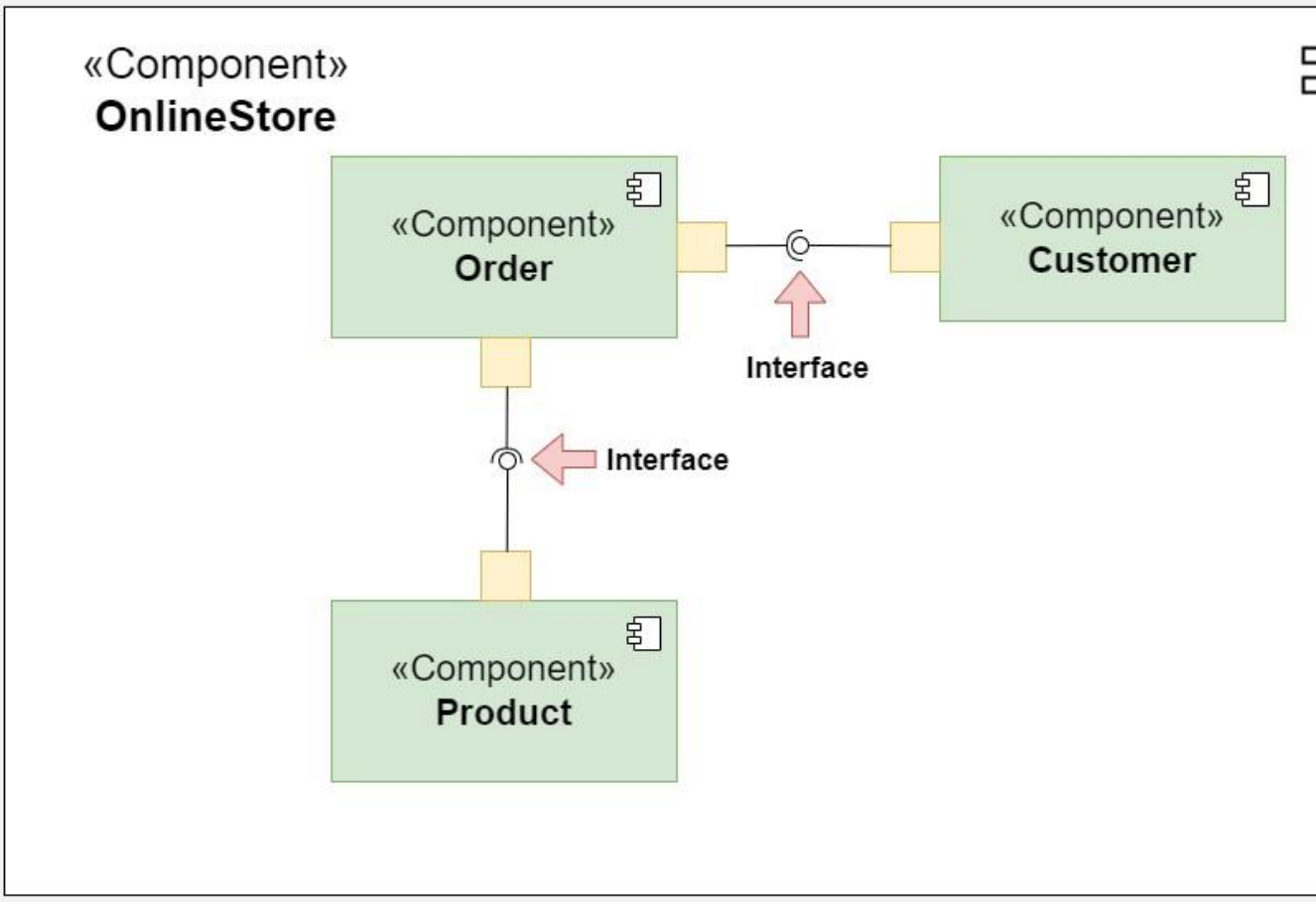


(ii) Interface

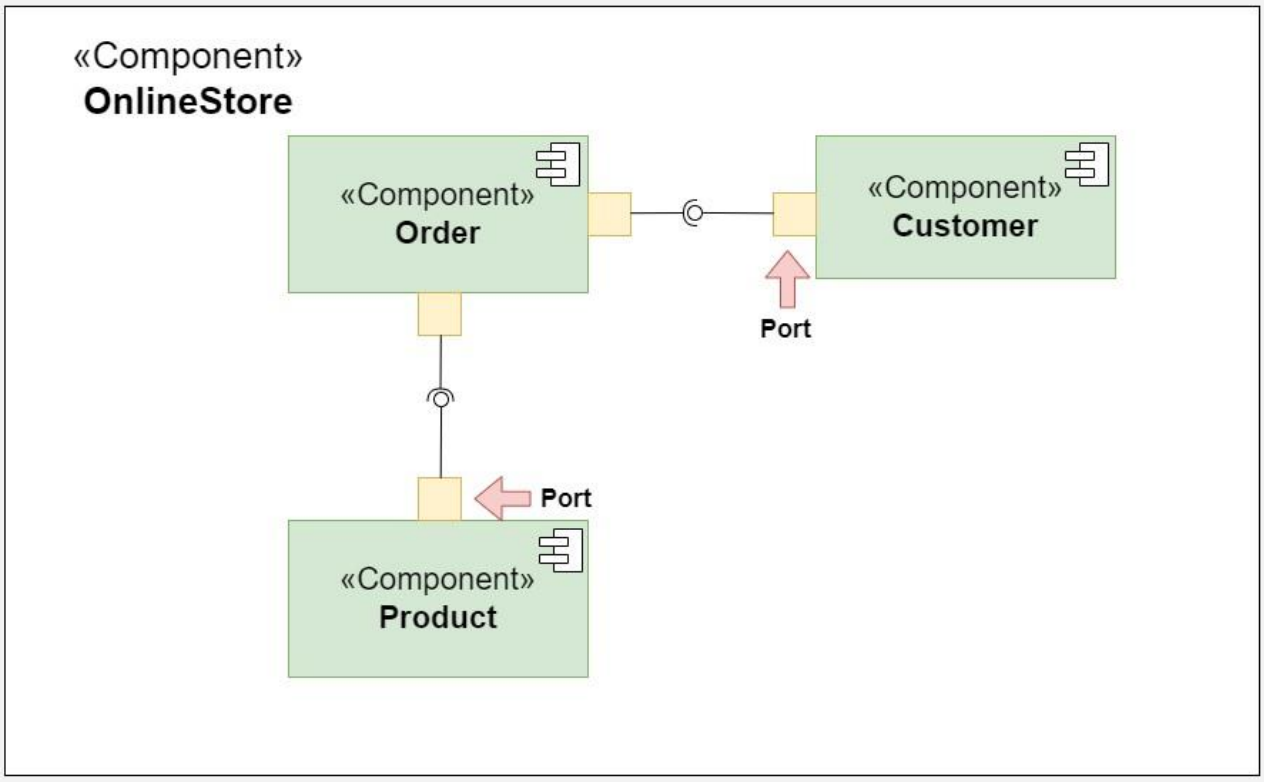
- **Provided interface** symbols with a complete circle at their end represent an interface that the component provides - this "lollipop" symbol is shorthand for a realization relationship of an interface classifier.
- **Required Interface** symbols with only a half circle at their end (a.k.a. sockets) represent an interface that the component requires (in both cases, the interface's name is placed near the interface symbol itself).
- **Function:** Define how components communicate with each other, ensuring that components can be developed and maintained independently.

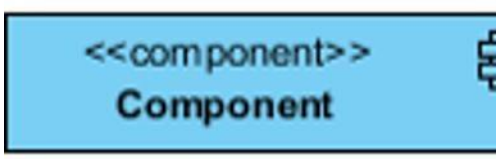


Interfaces in Component-Based Diagram



Ports in Component-Based Diagram





(iii) PORT

- Ports are represented using a square along the edge of the system or a component.
- A port is often used to help expose required and provided interfaces of a component.
- **Role:** Represent specific interaction points on the boundary of a component where interfaces are provided or required.
- **Function:** Allow for more precise specification of interaction points, facilitating detailed design and implementation.

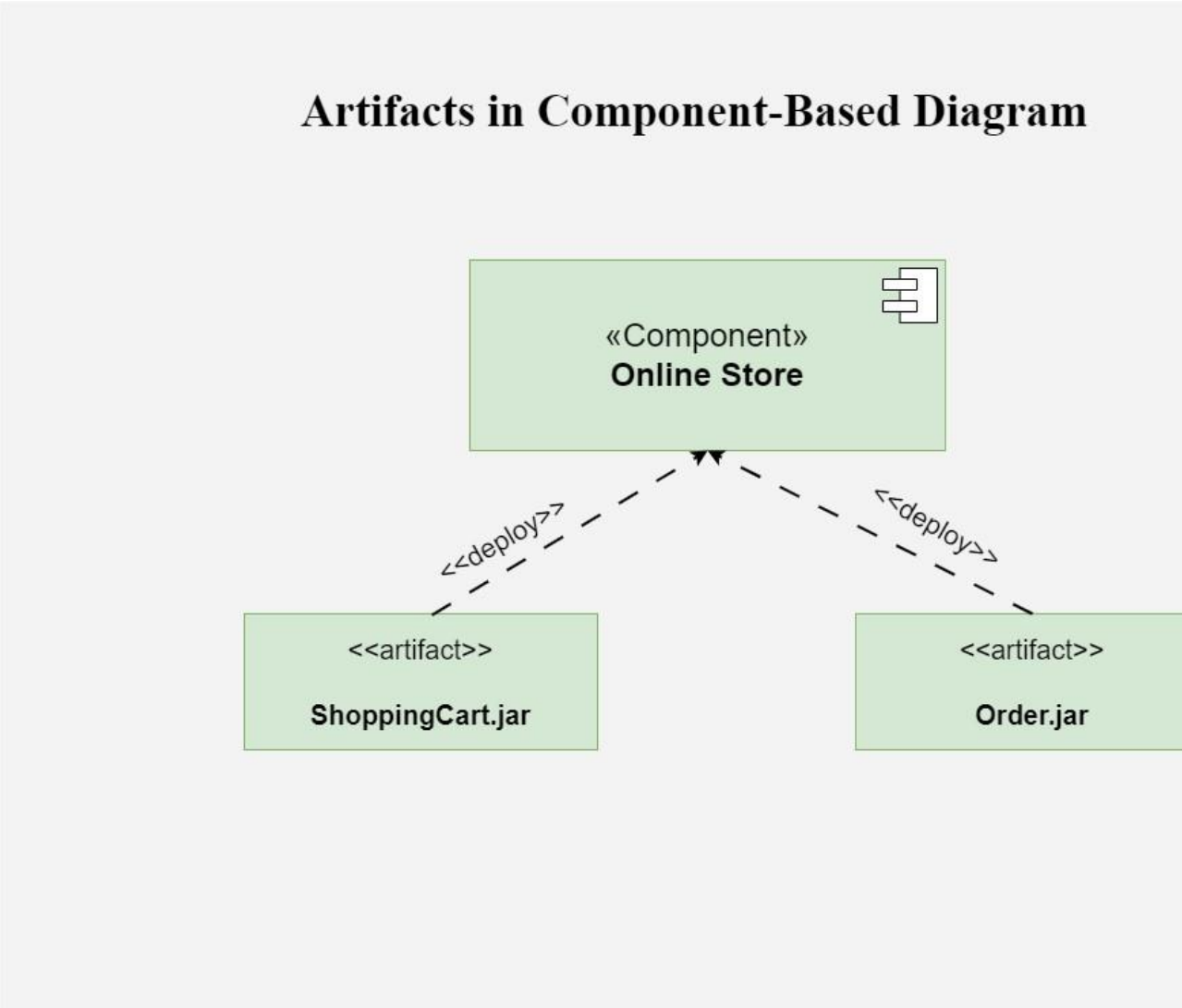
(iv). Artifacts

u Represent physical files or data that are deployed on nodes.

u**Symbol:** Rectangles with the artifact stereotype («artifact»).

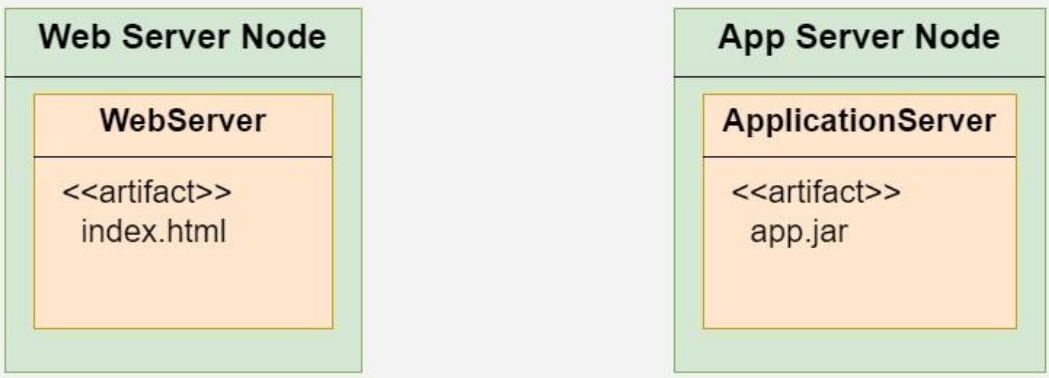
u**Function:** Show how software artifacts, like executables or data files, relate to the components.

Example- **Artifacts**



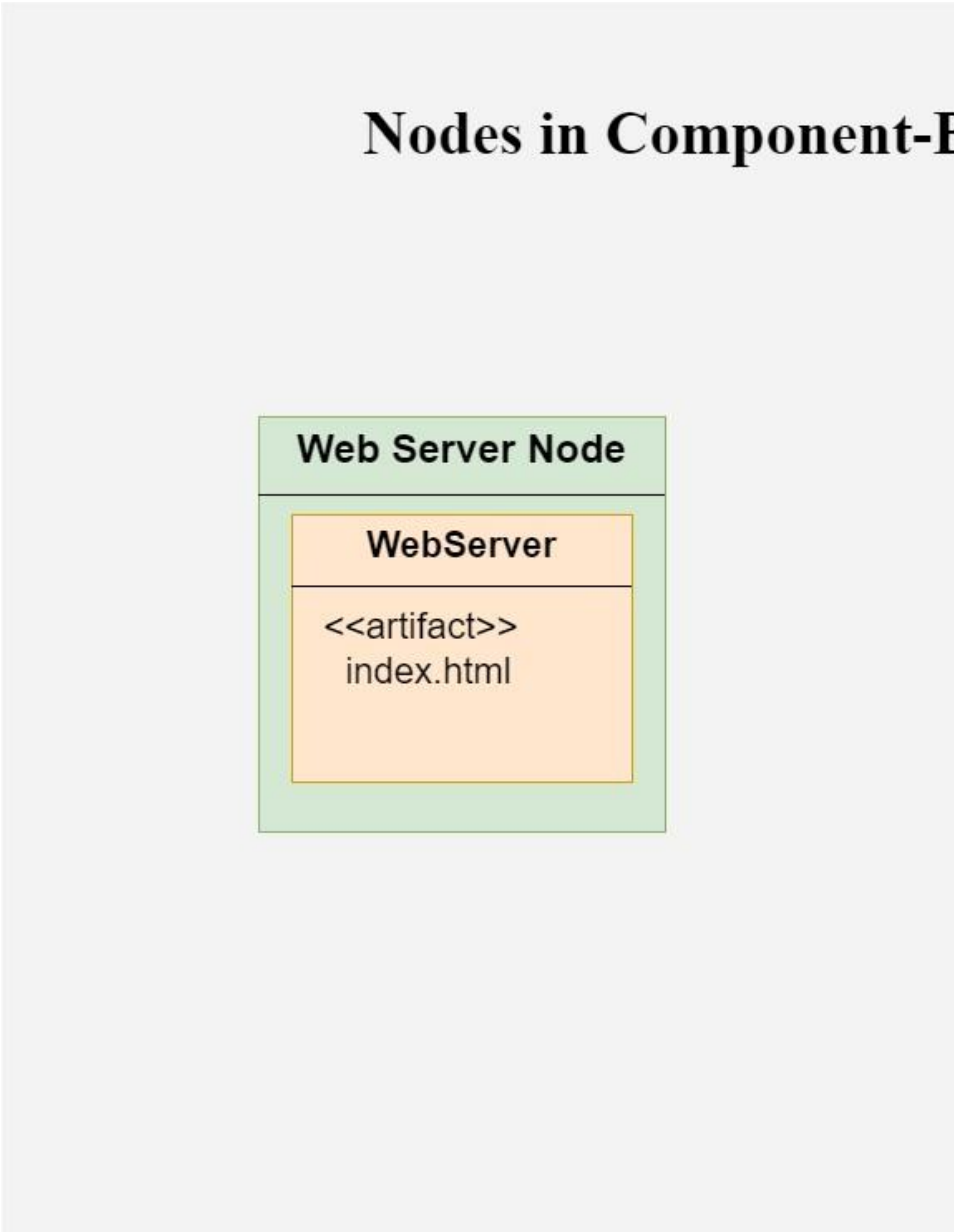
(v) Nodes

Nodes in Component-Based Diagram



- Represent physical or virtual execution environments where components are deployed.
- **Symbol:** 3D boxes.
- **Function:** Provide context for deployment, showing where components reside and execute within the system's infrastructure.

Node


Nodes in Component-E
Web Server Node
WebServer
<<artifact>>
index.html

(vi) RELATIONSHIPS

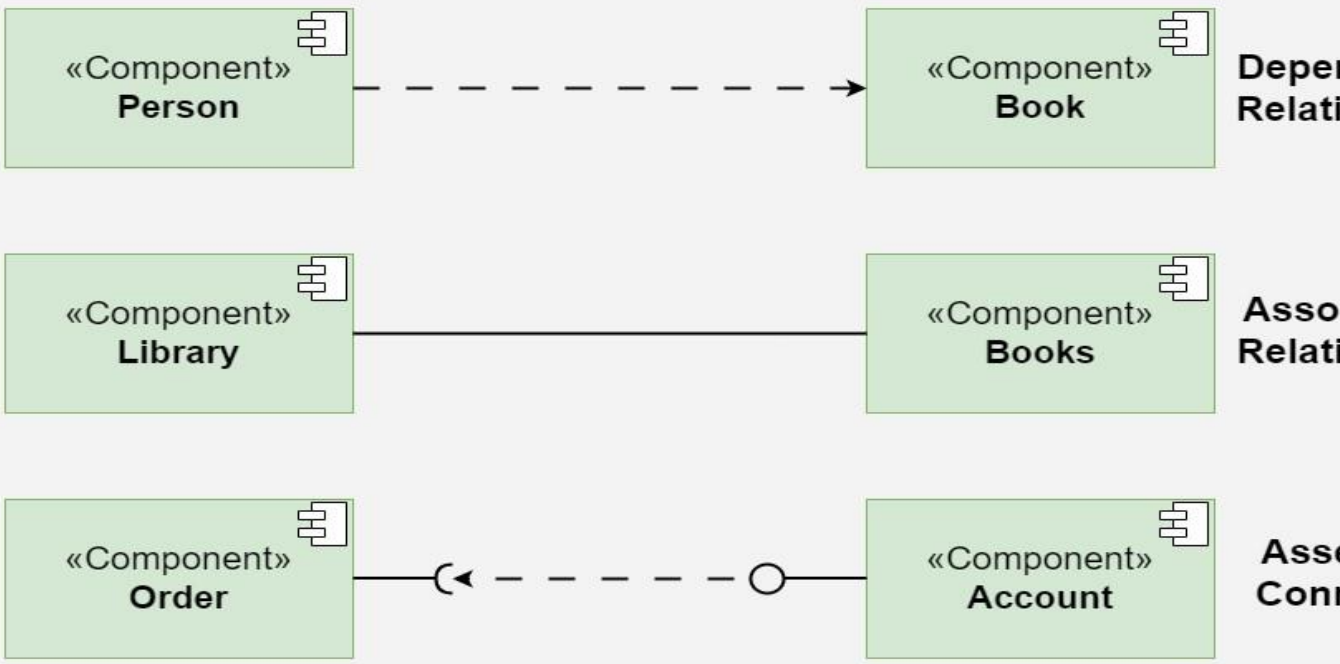
uGraphically, a component diagram is a collection of vertices and arcs and commonly contain components, interfaces and dependency, aggregation, constraint, generalization, association, and realization relationships.

uIt may also contain notes and constraints

Cont.: Relationships

- Relationships represent the connections and dependencies between components and interfaces, showing how different parts of the system interact.
- **Dependency (Dashed Arrow):** Indicates that one component relies on another component or interface to perform its functionality.
- **Association & Assembly Connector:** A solid line shows a direct relationship between components, while an assembly connector links a required interface of one component to a provided interface of another.
- **Function:** Helps visualize communication paths, dependencies, and interactions between components within the system architecture.

Relationships in Component-Based Diagram



ASSOCIATION

- An association specifies a semantic relationship that can occur between typed instances.
- It has at least two ends represented by properties, each of which is connected to the type of the end.
- More than one end of the association may have the same type.
- NOTATION

Aggregation

u A kind of association that has one of its end marked shared as kind of aggregation, meaning that it has a shared aggregation.



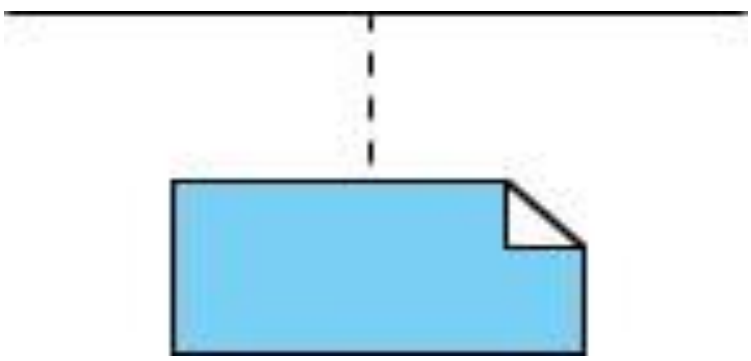
Composition

- Composite aggregation is a strong form of aggregation that requires a part instance be included in at most one composite at a time.
- If a composite is deleted, all of its parts are normally deleted with it.
- NOTATION



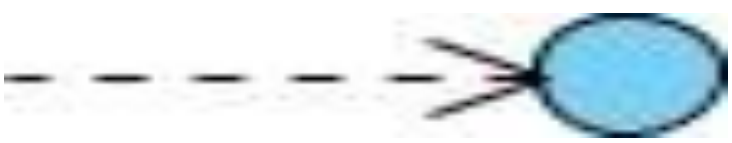
Constraint

• A condition or restriction expressed in natural language text or in a machine readable language for the purpose of declaring some of the semantics of an element.



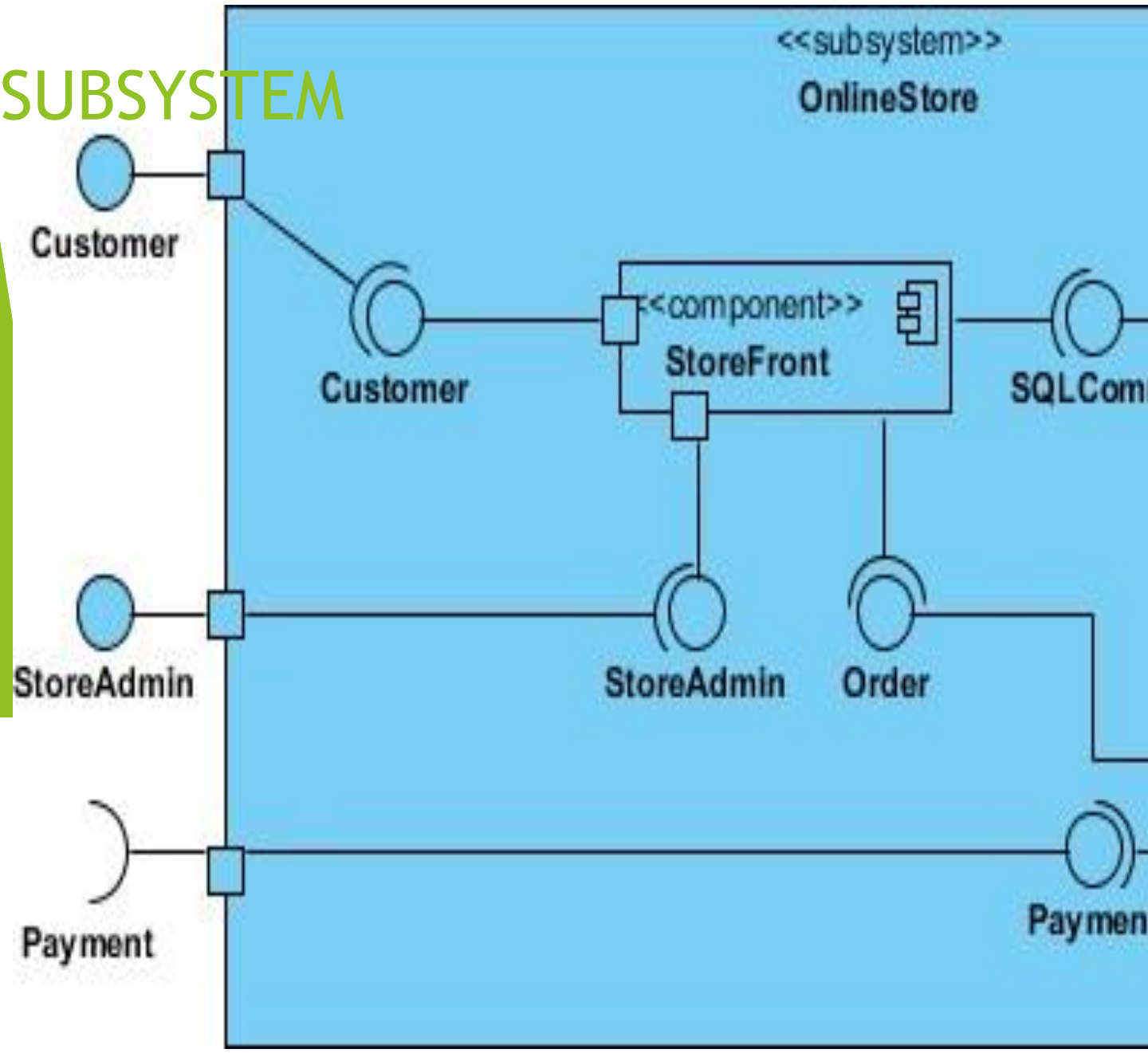
Dependency

- A dependency is a relationship that signifies that a single or a set of model elements requires other model elements for their specification or implementation.
- This means that the complete semantics of the depending elements is either semantically or structurally dependent on the definition of the supplier element(s).

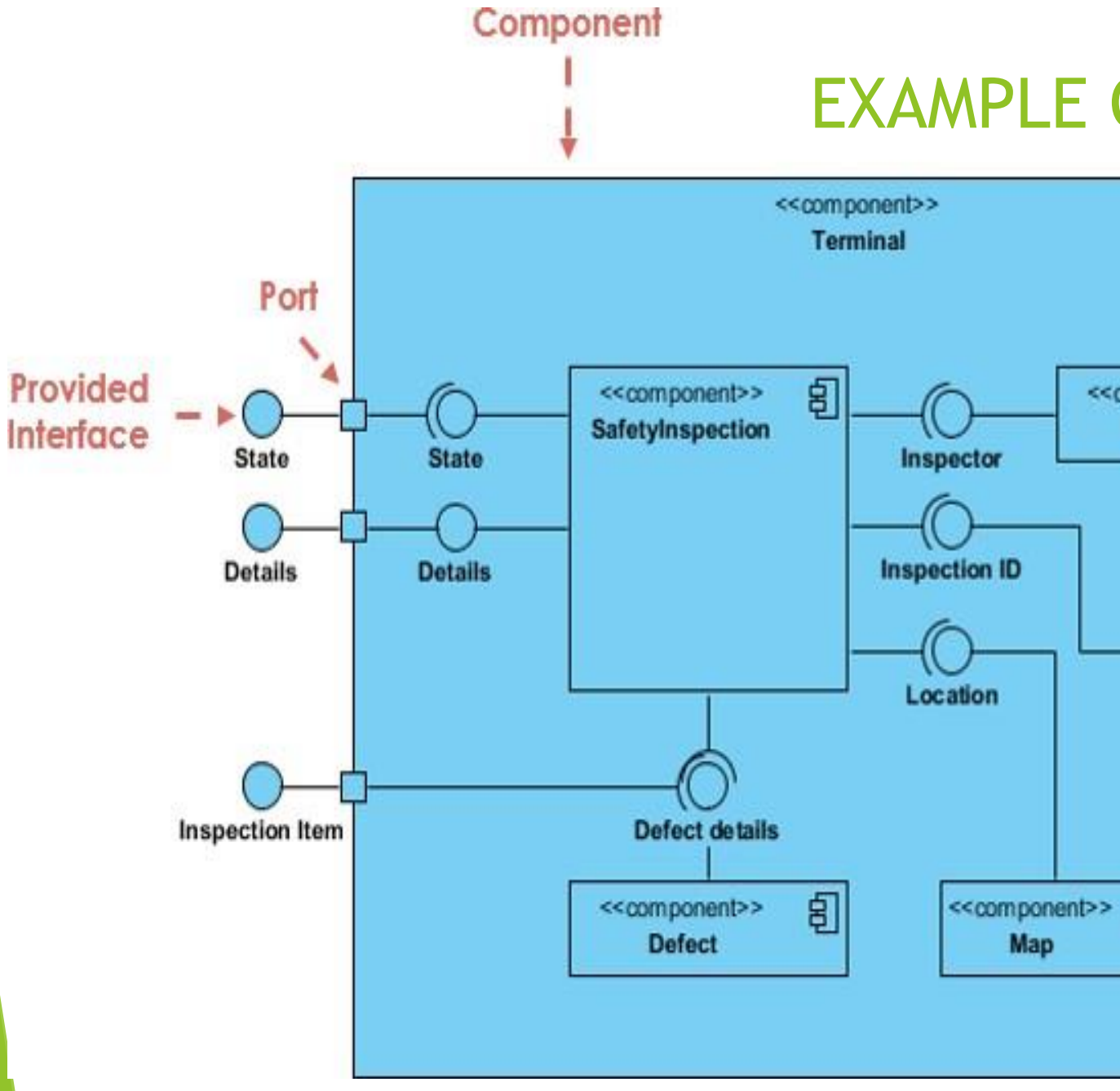


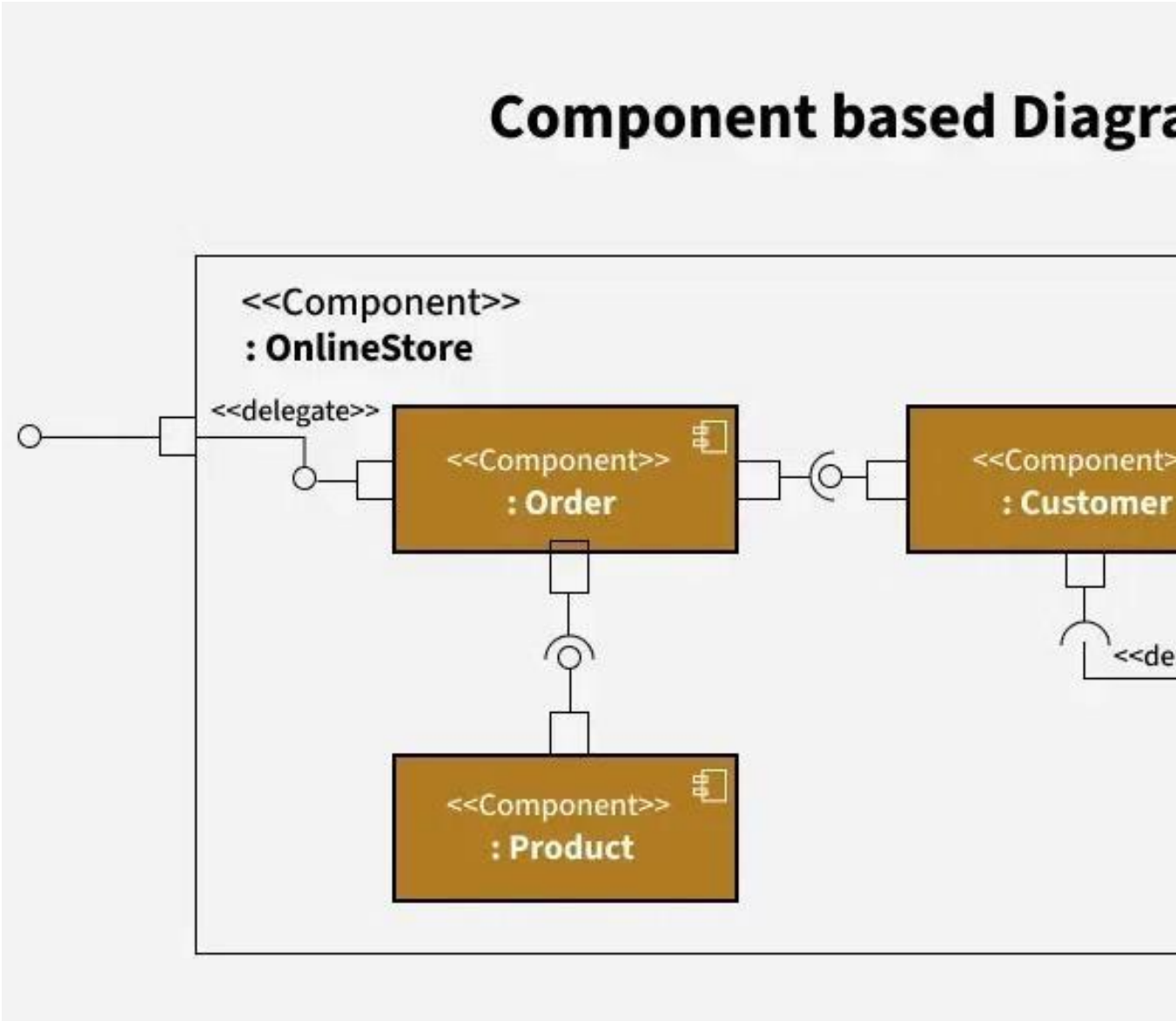
Subsystems

- The subsystem classifier is a specialized version of a component classifier.
- Because of this, the subsystem notation element inherits all the same rules as the component notation element.
- The only difference is that a subsystem notation element has the keyword of subsystem instead of component. EXAMPLE



EXAMPLE


Component
<<component>>
Terminal
Port
Provided Interface
State
State
<<component>>
SafetyInspection
Inspector
Details
Details
Inspection ID
Location
Inspection Item
Defect details
<<component>>
Defect
<<component>>
Map



Example of Component Based Diagram

This component diagram represents an **Online Store** system, breaking it down into various functional components and showing how they interact. Here's a breakdown of each part:

Explanation

1. **OnlineStore Component :** Represents the main system and encapsulates the core functionality of the online store. It contains the Order, Customer, and Product components.

2. **Order Component:** Handles order-related operations within the OnlineStore system.

- Connected to the Product component to access product information for orders.
- Connected to the Customer component to associate orders with customers.
- Uses delegate connectors (<>) to route certain internal operations to other components or systems

3. Customer Component: Manages customer data and customer-related activities.

- Connected to the Order component for processing and managing customer orders.
- Linked to the Account component through a delegate connection, allowing access to account-related information from an external system.

Cont.:

4. Product Component: Manages product information and product-related operations.

u Connected to the Order component so orders can reference and use available products.

5. Account Component: Represents an external component located outside the OnlineStore boundary.

- Connected to the Customer component through a delegate relationship.
- Handles account-related functions that are delegated from the OnlineStore system.

Applications of Component-Based Diagrams

Component-Based Diagrams help visualize system structure, improve communication, and support the development and maintenance of software systems.

- **System Design and Architecture:** Helps architects understand system structure, component interactions, and dependencies.
- **Requirements Analysis:** Assists stakeholders in understanding functional and nonfunctional system requirements.
- **System Documentation:** Serves as a reference for documenting architectural decisions and overall system design.
- **Software Development:** Guides developers by defining component boundaries and interactions.

- **Code Generation and Implementation:** Can be used as a foundation for generating code and accelerating development.
- **System Maintenance and Evolution:** Makes it easier to analyze, modify, and extend the system as requirements change.

Tools and Software available for Component-Based Diagrams

Several tools and software are available for creating Component-Based Diagrams, ranging from general-purpose diagramming tools to specialized UML modeling software. Here are some popular options:

1. **Lucidchart:** Lucidchart is a cloud-based diagramming tool that supports creating various types of diagrams, including Component-Based Diagrams.
2. **Microsoft Visio:** Microsoft Visio is a versatile diagramming tool that supports creating Component-Based Diagrams and other types of UML diagrams.
3. **Visual Paradigm:** Visual Paradigm is a comprehensive UML modeling tool that supports the creation of Component-Based Diagrams, along with other UML diagrams.
4. **Enterprise Architect:** Enterprise Architect is a powerful UML modeling and design tool used for creating Component-Based Diagrams and other software engineering diagrams.
5. **IBM Rational Software Architect:** IBM Rational Software Architect is an integrated development environment (IDE) for modeling, designing, and developing software systems.

Lecture 5:

COMPONENT BASED

SOFTWARE ENGINEERING

Component-Based Software Design

What is Component Design?

- Component design is all about taking complex software systems and making them into small, reusable pieces or simply modules. These parts are responsible for directing certain

functionalities, so programming them is like building a puzzle with small pieces, which eventually create more complex architectures. Component design exhibits a modularized

approach to software development where the units are organized systematically to facilitate control of complexity and increase manageability.

- Elements being the fundamental building blocks of software architecture, they are mostly responsible for enriching the software's functionality or offering their services.
- When several features are executed in independent units called components, with this approach modularity, reusability, and maintainability are ensured.
- This technique promotes flexibility and scalability, enabling the formation of dynamic systems that are composed of more basic units that interlock each other.

Importance of Component-Based Design in Software Development

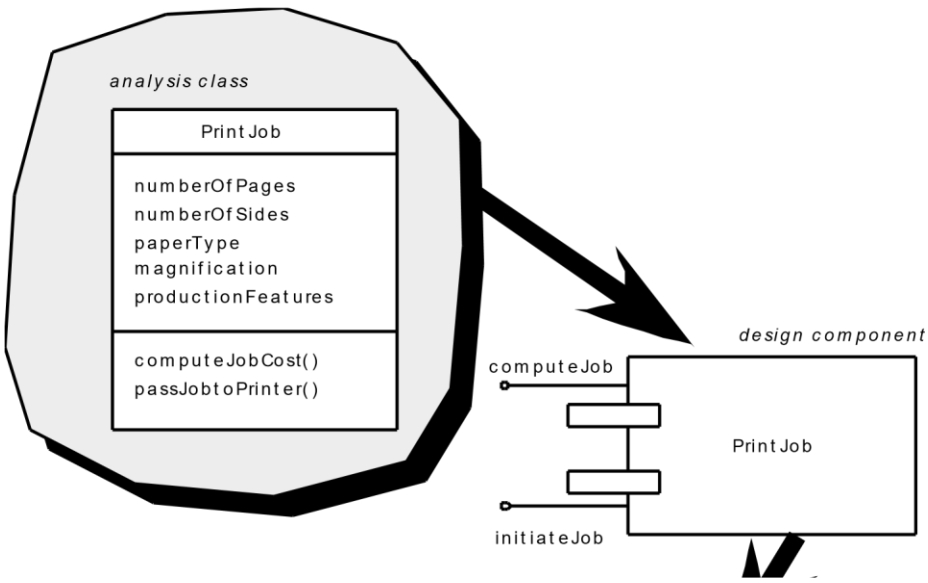
- Component-based design leads to various beneficial outcomes at all the development life stages of the software.
- It improves modularity, giving the possibility for all developers to work with different parts separately without changing the whole system. This modularity enables code reusability since components can be reused across several projects as this will save time and effort.
- Component-based design is so important for maintainability as it results from being broken down into different parts and depicts functionality, with which errors are easily traced, and debugging, and testing are made possible.

Cont.: **Importance of Component-Based Design in Software Development**

- When it comes to scalability, component-based design forms another vital benefit. A situation when a particular software system, as it turns out to be more complex, has to be scalable enough, leads to making this ability the priority.
- Breaking down elements and establishing logical interfaces facilitate the creation of a modular system that permits effortless integration of more capabilities without causing the architecture to buckle. It highlights the fact that adaptable software systems can not only be quickly modified but also evolve according to the changing requirements.

An Object-Oriented View

- Each class within a component has been fully elaborated to include all attributes and operations that are relevant to its implementation.
- As part of the design elaboration, all interfaces (messages) that enable the classes to communicate and collaborate with other design classes must also be defined.
- To accomplish this, the designer begins with the analysis model and elaborates analysis classes (for components that relate to the problem domain) and infrastructure classes (or components that provide support services for the problem domain).



Cont: Conventional view:

- Conventional view: a component is a functional element of a program that incorporates processing logic, the internal data structures that are required to implement the processing logic, and an interface that enables the component to be invoked and data to be passed to it.
- A conventional component, also called a *module*, resides within the s/w architecture and serves one of three important roles as:

Roles

1. A *control component* that coordinates the invocation of all other problem domain components,
2. A *problem domain component* that implements a complete or partial function that is required by the customer,
3. An *infrastructure component* that is responsible for functions that support the processing required in the problem domain.

Component level design has two main goals:

1. Identify the components that are needed to build the system.

u This includes determining the boundaries of each component and how they relate to one another.

2. Define the interfaces between components.

- This makes the components loosely coupled and independent, allowing them to be developed and tested separately before being integrated into the full system.

When designing components, software engineers consider:

- **Functionality**: What is the purpose of the component? What features will it provide?
- **Data**: What data does the component need to operate? What data does it produce? How will it interface with data from other components?
- **Dependencies**: On what other components does this component rely? How will those dependencies be handled?
- **Reusability**: Can this component be reused in other systems? If so, how can its design be made more generic?
- **Scalability**: How will this component handle increases in load or volume? Can its performance be optimized?

Note:

- Component-level design is a fundamental step in building a robust, modular software architecture.
- By determining how to break a complex system into discrete, interconnected parts, software engineers can create solutions that are flexible, extensible, and resilient to changes in requirements.
- The end result is a system that is greater than the sum of its parts.

Designing Class-Based Components

- These principles can be used to guide the designer as each S/W component is developed.
- **The Open-Closed Principle (OCP)**. *"A module [component] should be open for extension but closed for modification.* The designer should specify the component in a way that allows it to be extended without the need to make internal (code or logic-level) modifications to the component.
- **The Liskov Substitution Principle (LSP).** *"Subclasses should be substitutable for their base classes.* A component that uses a base class should continue to function properly if a class derived from the base class is passed to the component instead.
- **Dependency Inversion Principle (DIP).** *"Depend on abstractions. Do not depend on concretions."* Abstractions are the place where a design can be extended without great complications.

Cont.:

- **The Interface Segregation Principle (ISP).** *"Many client-specific interfaces are better than one general purpose interface.* The designer should create a specialized interface to serve each major category of clients.
- **The Release Reuse Equivalency Principle (REP).** *"The granule of reuse is the granule of release."* When classes or components are designed for reuse, there is an implicit contract that is established between the developer of the reusable entity and the people who will use it.
- **The Common Closure Principle (CCP).** *"Classes that change together belong together."* Classes should be packaged cohesively. When some characteristic of an area must change, it is likely that only those classes within the package will require modification.
- **The Common Reuse Principle (CRP).** *"Classes that aren't reused together should not be grouped together."* When one or more classes with a package changes, the release number of the package changes.

Component-Level Design Guidelines

u **Components**

u Naming conventions should be established for components that are specified as part of the architectural model and then refined and elaborated as part of the component-level model

u **Interfaces**

u Interfaces provide important information about communication and collaboration (as well as helping us to achieve the OCP)

u **Dependencies and Inheritance**

u It is a good idea to model dependencies from left to right and inheritance from bottom (derived classes) to top (base classes).

Cohesion

- *Cohesion* implies that a component or class encapsulates only attributes and operations that are closely related to one another and to the class or component itself.
- Levels of cohesion
- Functional: occurs when a module performs one and only one computation and then returns a result.
- Layer: occurs when a higher layer accesses the services of a lower layer, but lower layers do not access higher layers.

Cont.: **Cohesion**

- Communicational: All operations that access the same data are defined within one class.

- Sequential: Components or operations are grouped in a manner that allows the first to provide input to the next and so on.
- Procedural: Components or operations are grouped in a manner that allows one to be invoked immediately after the preceding one was invoked.
- Temporal: Operations that are performed to reflect a specific behavior or state.
- Utility: Components, classes, or operations that exist within the same category but are otherwise unrelated are grouped together.

Coupling

uConventional view:

uThe degree to which a component is connected to other components and to the external world uOO view:

uA qualitative measure of the degree to which classes are connected to one another. Keep coupling as low as possible.

uLevel of coupling

Level of coupling

- **Content**: Occurs when one component "superstitiously modifies data that is internal to another component. "Violates Information hiding"
- **Common**: Occurs when a number of components all make use of a global variable.
- **Control**: Occurs when *operation A()* invokes *operation B()* and passes a control flag to *B*. The control flag then "directs" logical flow within *B*.
- **Stamp**: Occurs when **ClassB** is declared as a type for an argument of an operation of **ClassA**. Because **ClassB** is now a part of the definition of **ClassA**, modifying the system becomes more complex.

Cont.:

u**Data**: Occurs when operations pass long strings of data arguments. "Testing and maintenance becomes more difficult."

u**Routine call**: Occurs when one operation invokes another.

u**Type use**: Occurs when component **A** uses a data type defined in component **B.**

u**Inclusion or import:** Occurs when component **A** imports or includes a package or the content of component **B.**

uExternal: Occurs when a component communicates or collaborates with infrastructure components (O/S function).

Cont.: **Conducting Component-Level Design**

u The steps discussed in this section provide a reasonable task set for designing a component. You should emphasize that

(1) Design classes in the problem domain are usually custom-designed, however, if an organization has encouraged design for reuse, there may be an existing component that fits the bill;

(2) Design classes corresponding to the infrastructure domain can sometimes be often from existing class libraries;

3) A UML collaboration diagram provides an indication of message passing between components.

Steps

- Step 1. Identify all design classes that correspond to the problem domain.
- Step 2. Identify all design classes that correspond to the infrastructure domain.
- Step 3. Elaborate all design classes that are not acquired as reusable components.
- Step 3a. Specify message details when classes or component collaborate.
- Step 3b. Identify appropriate interfaces for each component.
- Step 3c. Elaborate attributes and define data types and data structures required to implement them.
- Step 3d. Describe processing flow within each operation in detail.

Cont.:

- Step 4. Describe persistent data sources (databases and files) and identify the classes required to manage them.
- Step 5. Develop and elaborate behavioral representations for a class or component.
- Step 6. Elaborate deployment diagrams to provide additional implementation detail.
- Step 7. Factor every component-level design representation and always consider alternatives.

Component Lifecycle Management

Component Lifecycle Management is a systematic approach to managing the lifecycle of software components from their creation to decommissioning.

Stage 1: Creation: Components either get initialized statically or dynamically at design time based on the needs of the program and according to the entire program or system architecture.

Stage 2: Initialization: Components are set up, configured and prepared for use, usually this include parameterization, allocation of resources and the side setup.

Cont.: Component Lifecycle Management

- **Stage 3: Utilization:** Components are utilized within the software system, fulfilling their designated functionalities and interacting with other components as necessary.
- **Stage 4: Maintenance:** Components undergo maintenance activities such as updates, bug fixes, and optimizations to ensure continued functionality and performance.
- **Stage 5: Decommissioning:** Components are decommissioned, or disposed, when they are no longer needed, either from obsolescence, redundancy, or architectural changes.

Characteristics of Component-Based Design

Below are the characteristics of component-based design:

- **Modularity:** Some parts of a program retain a certain function or service, essentially serve as reusable software modules, interchangeable and therefore independent components, enhancing flexibility in development, testing, and maintenance.
- **Reusability:** Component architectural features are envisioned to be adaptable to and reusable on different projects giving rise to shorter development time and lower overheads and making the work of developers much more easier and scientific.

Characteristics of Component-Based Design

- **Interoperability:** Components are connected through clearly defined interfaces, ensuring that software systems maintain interaction continuity and carry out various functions within their ecosystems.
- **Encapsulation:** Components have got encapsulated constructs, which precisely reveal internal details at the interface level, through a provision of only a few interfaces essential for interacting with other components, thus enforcing abstraction by hiding underlying implementation details.
- **Scalability:** Component-based architectures simplify scalability by providing a way for systems to grow organically via the addition or modification of components without the impacting overall the architecture.

Types of Components

u **UI Components**

u User Interface components provide an easy and more convenient way to encapsulate logic by combining presentational and visible elements such as buttons, forms, and widgets.

u **Service Components**

u Service components are the base of business logic or application services, in which they serve as the platform for activities such as data processing, authentication, and communication with external systems.

u **Data Components**

u Through data abstraction and provision of interfaces for data access, data components take care of database interaction issues and provide data structures for querying, updating, and saving data.

Cont.:

u **Infrastructure Components**

u The hardware elements regard as fundamental services or resources like logging, caching, security and communication protocols which a software system depends on.

u **Integration Components**

u Integrated components for data communication and data exchange between different systems or modules are the integration components, which enable protocol translation, workflow orchestration, and data exchange.

u **Reusable Components**

u A reusable component, in turn, encapsulates common functionality or algorithms that can be utilized across multiple projects as well as different domains, which promotes code reuse and uniformity.

Component Design Patterns

- Component design patterns present the designer with patterns that can be reused to solve typical problems that design architects face in the construction of componentbased architectures Such patterns carry guidance on the effective design and implementation of components as well as the art of integrating these effectively.

Some popular component design patterns include:

- **Composite Pattern:** The Composite pattern allows objects to be composed into tree structures, treating individual objects and compositions of objects uniformly.
- **Decorator Pattern:** The decorator pattern provides the facility of dynamic enhancement in the object behavior by wrapping decorators that are added to existing functionalities without changing the interface.

Cont.: **Component Design Patterns**

- **Adapter Pattern:** Through Adapter pattern, interface incompatibility issues are solved as a bridge is built that translates requests from one interface to another by using this one.
- **Factory Pattern:** Factory pattern comprehends the process of preparing objects, with the outcome of providing customers the ability to create objects without specifying their concrete classes, hence facilitating flexibility and decoupling.
- **Observer Pattern:** The Observer pattern constitutes a unidirectional link between objects, a one-to-many dependency, where objects notifying and updating other objects automatically, as their state changes.

- **Facade Pattern:** The Facade pattern provides a simplified interface to a complex system, encapsulating its intricate details and presenting a unified interface to clients.
- **Singleton Pattern:** The Singleton pattern, which ensures that a class has only one instance and grants global access to that object, enables one to control centralized access to shared objects and data.

Tools and Technologies for Component Design

- **Component Frameworks:** Components frameworks for instance, Angular, React, and Vue.js include a variety of pre-built components, templates, and utilities for building user interfaces that users can interact.
- **Dependency Injection Containers:** Injection Containers of IoC nature like Spring IoC, Guice and Dagger can take the control of inversion by managing the component dependencies as well as the producing the instances for the dependent classes.
- **Middleware Platforms:** API middleware systems like Apache Camel or MuleSoft or RabbitMQ connect different system and services by means of message routing, transformation, and mediation.
- **Component Repositories:** These repositories like npm, Maven Central, and NuGet pool together the storage, sharing, and discovery of libraries and reusable components.
- **Component Testing Tools:** Tools like Jest, JUnit, and Mockito are used for automated testing of components; this lets their functionality, reliability and adherence to specification to be verified.

Common Mistakes in Component Level Design

(i) Lack of Cohesion

- Cohesion refers to the degree to which the elements of a component belong together.
- A cohesive component has a singular, focused purpose and contains closely related elements.
- Lack of cohesion is one of the most common mistakes in component level design. It often results in components that are difficult to understand, reuse, and maintain.

Cont.:

(ii) Excessive Coupling

- Coupling refers to the degree of interdependence between components.

- Excessively coupled components are too dependent on each other, making them hard to reuse and maintain independently.
- Loose coupling, on the other hand, reduces interdependencies and allows components to operate independently.
- Excessive coupling is a frequent mistake that contradicts the goals of component-based design.

Cont.:

(iv) Violating Encapsulation

- Encapsulation means hiding the internal details and implementation of a component from the outside.
- It enables the internal workings of a component to change without impacting other components.
- Failing to properly encapsulate components exposes their inner workings, creating tight coupling and dependencies on implementation details that may change.
- This mistake compromises the flexibility and maintainability of the overall system.

Cont.:

(v) Lack of Abstraction

u Abstraction involves suppressing irrelevant details about a component in order to reduce complexity and facilitate understanding. Lack of abstraction results in low-level components that are overly complex, difficult to understand, and reuse. Effective abstraction focuses on the essential purpose and functionality of a component, hiding unnecessary details.

(vi) Not Considering Reusability

u If components are not designed with reuse in mind, the system will end up with many redundant components that cannot be reused in other contexts. This error leads to increased development and maintenance efforts. On the other hand, reusable components can be used in multiple systems, reducing their development cost

Challenges of Component Design

Below are the challenges of component design:

- **Granularity:** Proper level of granularity is what designers of components need to choose accurately, as excessively fine components tend to clutter and increase overhead, while coarse grained ones are inappropriately inflexible and have a limited ability to reuse.
- **Interoperability:** Maintaining the seamless operation of components from unmixed vendors or technologies may be a difficult task. For this reason, standardized interfaces, protocols and compatibility tests have to be carried out.

- **Dependency Management:** Keeping dependencies between the components, such as versions, compatibility, and conflicts can be extremely difficult especially in the bigger systems, which have a augmented number of dependencies.
- **Performance Overhead:** The component-based architectures may produce performance overhead for its added levels of abstraction, communication and runtime dependency which require meticulous optimization and profiling thus may result in runtime performance issues.

Lecture 6:

COMPONENT BASED SOFTWARE ENGINEERING

Architecting Component-Based Systems

Introduction

- In general, software architecture serves as a blueprint, outlining how a system is structured and organized. It determines whether your system will be scalable, maintainable, and how well it will perform.
- Currently, more organizations and developers are turning to component-based architecture (CBA) to build complex software systems.
- This approach emerged to address the need for highly scalable, loosely coupled solutions. With CBA, an application is split into smaller, manageable, and reusable parts called components.
- Each component contains specific functionality and interacts with others through well-defined interfaces.

What is component-based architecture?

- Component-based architecture is a framework for building software based on reusable parts. Each component contains well-defined functionality in a binary unit that is stored in a library and then dropped into an application without modifying other components.
- When building a structure out of Legos, you can choose various shapes, sizes, and colors. Some blocks are purpose-built to be doors, windows, and other structural elements. Each block has all the features it needs to connect to others, and adding or subtracting blocks generally has minimal impact on the structure.
- Software components are much more complex than little pieces of plastic, but the concept is similar.

- Each element performs a task in an architecturally defined way. For simplicity, components are stored in a library, assembled by developers, and talk to each other through APIs.

What is Software Connector?

- software connectors are architectural building blocks that mediate, coordinate, and facilitate interaction, data exchange, and control flow between independent components.
- A connector in software architecture refers to an intermediary that facilitates communication and coordination between different software components or systems. It acts as an intermediary, facilitating the exchange of data, requests, or responses between different systems, allowing them to work together seamlessly.

Types of Software Connectors

- **Procedure Call Connectors:** is a fundamental software connector type that models the flow of control and transfers data between software components via routine or method invocations. Allow a component to directly invoke procedures, methods, or functions of another component (similar to standard programming language function calls or RPC - Remote Procedure Calls).
- **Event-Based Connectors:** Event-based connectors are communication mechanisms that allow software components to interact by generating, sending, receiving, and responding to events. An event is a notification that something has happened in a software system. In component-based architecture, one component can generate an event, while another component responds to it without necessarily knowing the internal details of the component that generated it.

Event-based connectors	**Procedure-call connect**
Components communicate through events.	Components communica
Often support asynchronous communication.	Often involve a direct re
Usually promote loose coupling.	Can create stronger depe
Example: notifying a component that a payment succeeded.	Example: calling a meth

Cont.:

- **Data-Flow Connectors:** Data-flow connectors are communication mechanisms that transfer data between software components so that the data can be processed, transformed, or stored. Unlike procedure calls that transfer control, these connectors are primarily concerned with the **flow of data** and often handle synchronization implicitly.
- **Client-Server Connectors:** serve as the architectural "glue" that separates interaction from computation, explicitly modeling the request-response communication between independent software modules. Manage request-reply protocols where a client component sends a service request and blocks or waits for a response from a server component.

Benefits of Connectors

- **Decoupling:** Connectors separate the components, allowing them to be developed, deployed, and scaled independently.
- **Flexibility:** They enable different components to interact using various communication protocols, making the system adaptable to changes.
- **Interoperability:** Connectors facilitate the integration of heterogeneous systems and technologies.
- **Scalability:** By managing communication, connectors can help balance load and distribute requests efficiently.
- **Maintainability:** Changes to the communication logic can be made independently of the components, simplifying system updates and maintenance.
- **Reliability:** Connectors can provide mechanisms for error handling, retries, and failover, enhancing the robustness of the system.

Architectural Style

- an **architectural style** dictates how independent, reusable components are structured, encapsulated, and coordinated.
- Architectural style defines how the components of a software system are organized and how they interact with one another.

Layered Architecture as an Architectural Style

- Layered architecture is one of the classic **architectural styles** It organizes a system into horizontal layers, where: Each layer has a specific responsibility (e.g., presentation, business logic, persistence).
- A layer typically only interacts with the layer directly below it (strict layering) or with any lower layer (relaxed layering).
- Layers expose well-defined interfaces and hide their internal implementation.
- This matches the definition of an architectural style: a named, reusable set of design decisions about structure and interaction.

Client-server component architecture

- **Client-server component architecture** is a distributed software design model where an application's workload is split

between two primary entities: clients (which request services or resources) and servers (which provide them).

- This architecture structures application logic into three core layers— **presentation, processing, and data**—and relies on several critical components to function

Key Technical Components

- The architecture relies on several interconnected software and hardware components to execute a request-response cycle:
- **Client:** The user-facing program or device (e.g., web browsers, mobile apps) that initiates communication by sending a request over the network.
- *Thin Client:* Only handles presentation logic; relies entirely on the server for processing.
- *Fat/Thick Client:* Runs some processing and business logic locally before communicating with the server.
- **Server:** A high-capacity computer or background process that continuously "listens" for incoming requests, executes the requested tasks, and returns a response. A single server environment may distribute roles into specialized nodes like file servers, database servers, and mail servers.

Cont.:Key Technical Components

- **Network & Protocols:** The communication channel (including routers, switches, and firewalls). Components communicate using standardized protocols (like HTTP, HTTPS, or gRPC) to ensure they agree on data formats.
- **Database:** The centralized repository (e.g., SQL or NoSQL databases) where the application's persistent data is organized and secured.
- **Middleware / Load Balancers:** Software components that sit between the client and server to handle routing, authentication, logging, or traffic distribution across multiple server instances

Common Architectural Tiers

Tier Type	Description

2-Tier Architecture	The client handles the interface and business logic, communicating directly with a database server.
3-Tier Architecture	Introduces an application server between the client UI and the databas handle the business logic independently.
N-Tier Architecture	Blends multiple specialized microservices, caching layers, and enterp service buses into a deeply segmented system.

Others styles include

- **Microservices** are a software architectural style in which an application is divided into small, independent services, with each service responsible for a specific business function and communicating with other services through well-defined interfaces.
- In component-based architecture (CBA), microservices can be viewed as independently deployable software units that contain components responsible for particular tasks

Cont.: Microservices

- For example, when a customer places an order:
  - The Order Service records the order.
  - The Payment Service processes the payment.
  - The Delivery Service arranges shipment.
  - The Product Service updates stock availability.

Event-driven architectural style

- Event-driven architectural style is a software architecture in which components communicate and coordinate their activities by generating, transmitting, and responding to events.
- An event is a notification that something has happened in a system, such as a user clicking a button, a payment being completed, or a new order being placed.

- In event-driven architecture (EDA), components do not call each other directly. Instead, they **react to events**. An event is a significant change in state (e.g., "order placed", "temperature exceeded"). The flow of control is **implicit** — the publisher of an event does not know (or care) which components will react.
- Individual components can produce or respond to events to perform different tasks without requiring direct communication between every component.

Example in a mobile application

- Consider a mobile shopping application.
- When a customer successfully places an order:
- The Order Component generates an OrderPlaced event.
- The event broker distributes the event to interested components.
- The Notification Component sends an order confirmation.
- The Inventory Component updates the available stock.
- The Delivery Component begins preparing the shipment.

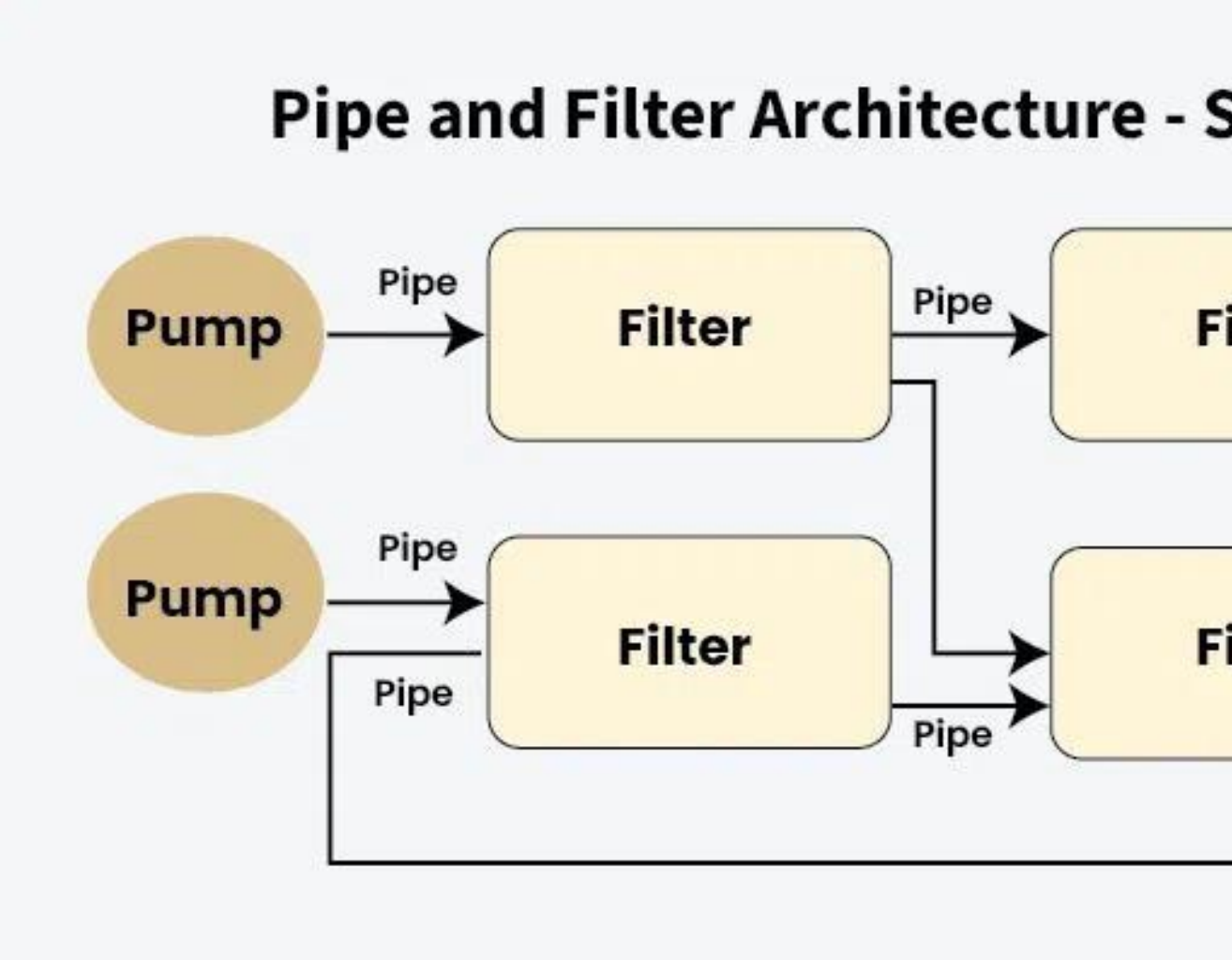
Pipe-and-filter architectural style

- Pipe-and-filter architectural style is a software architectural style in which a system is organized as a sequence of independent processing components called filters, connected by communication mechanisms called pipes.
- Each filter receives data, processes or transforms it, and passes the resulting data to the next filter through a pipe. Each filter is dedicated to a particular processing function, whether it involves transforming, validating, or aggregating data. Data flows through these filters via pipes, which carry the output of one filter to the input of another.

Cont.: Pipe-and-filter architectural style

- This architecture enhances modularity, as each filter operates independently and concentrates on a singular function. It also promotes reusability, allowing filters to be utilized across different systems or applications.
- Moreover, it is both flexible and scalable; filters can be added, removed, or rearranged with little effect on the overall

system, and multiple instances of filters can function concurrently to manage larger data sets.



Example

- Consider a mobile application that calculates students' examination results.
- Input filter: Receives students' marks.
- Validation filter: Checks that the marks fall within the permitted range.
- Calculation filter: Calculates the average mark.
- Grading filter: Assigns a grade based on the average.
- Output filter: Displays the student's results.
- The pipes transfer the data between these filters in the required order.

Component composition

- Component composition is a design pattern where you build complex user interfaces by combining smaller, independent, and reusable components like Lego bricks. Instead of writing a single monolithic component bloated with dozens of props and

conditional checks, you distribute responsibilities across cooperating parent and child component

- Component composition is the process of combining two or more software components to form a complete software system or application.
- In component-based architecture, each component performs a specific function and interacts with other components through defined interfaces.
- The aim is to assemble components so that they work together to satisfy the requirements of a software system.

Methods of component composition

1. Sequential composition: Components are connected in a sequence, where the output of one component becomes the input of another. Example: A component validates data before another component processes it.
2. Hierarchical composition: Components are organized into levels, where a higher-level component coordinates lower-level components. Example: An order management component coordinates payment and delivery components.

Cont.: Methods of component composition

3. Additive composition: Components are combined to add new functionality to an existing system. Example: Adding a notification component to an existing mobile application.
4. Parallel composition: Components perform different tasks concurrently or independently.

Example: After an order is placed, separate components process inventory updates and customer notifications.

Note: These are common ways to describe component composition; terminology and classifications may differ across course materials.

Steps involved in component composition

- Identify the system requirements.
- Identify or select suitable software components.
- Examine component interfaces and dependencies.
- Connect compatible components using appropriate connectors.

- Integrate the components into a complete system.
- Test the integrated system to ensure that the components work correctly together.

Advantages of component composition

- Reusability: Existing components can be reused in different applications.
- Reduced development time: Developers do not need to build every function from scratch.
- Maintainability: Individual components can be modified independently when interfaces remain compatible.
- Flexibility: Components can be replaced or added to extend functionality.
- Improved productivity: Development work can be divided among different teams.

framework

- *A set of cooperating classes that makes up a reusable design for a specific class of software. A framework provides architectural guidance by partitioning the design into abstract classes and defining their responsibilities and collaborations. A developer customizes the framework to a particular application by subclassing and composing instances of framework classes.“*
- In essence, component-frameworks break the system down into variant "components" that represent abstract processing that the system supports.
- These components are generally thought of as "pluggable" in the sense that the system can be configured with a set of components to accomplish a particular task or set of tasks.
- The framework has the effect of decoupling or isolating the components from each other since it the component interactions are handled by the framework in a manner that is opaque to the components

key characteristics of a component-framework system

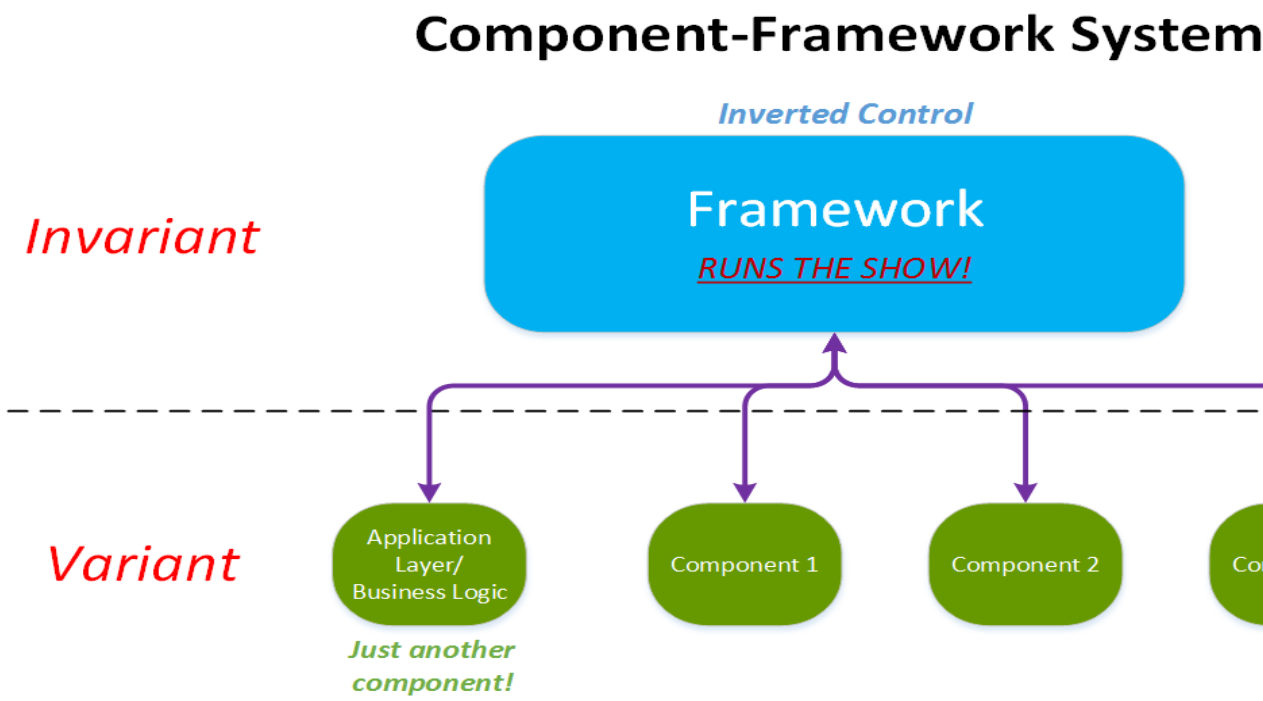
- *its "inversion of control" or "Hollywood Principle": "Don't call us, we'll call you!*" Since the framework portion of the system represents the invariant aspects of the system that mediates

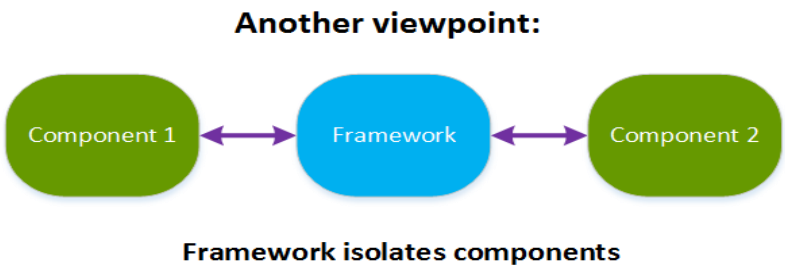
the dynamics of the variant components, *control of the system must be delegated to the framework* because the components are only providing services to the framework and are unaware of how those services interact with each other.

- The framework that is "running the show", directing the calls to the component services, not the other way around.

Inversion of control virtually eliminates flow control in the components and thus reduces code complexity in the components.

- Frameworks can vary greatly in size, ranging from small data structures to enterprise-class distributed applications.



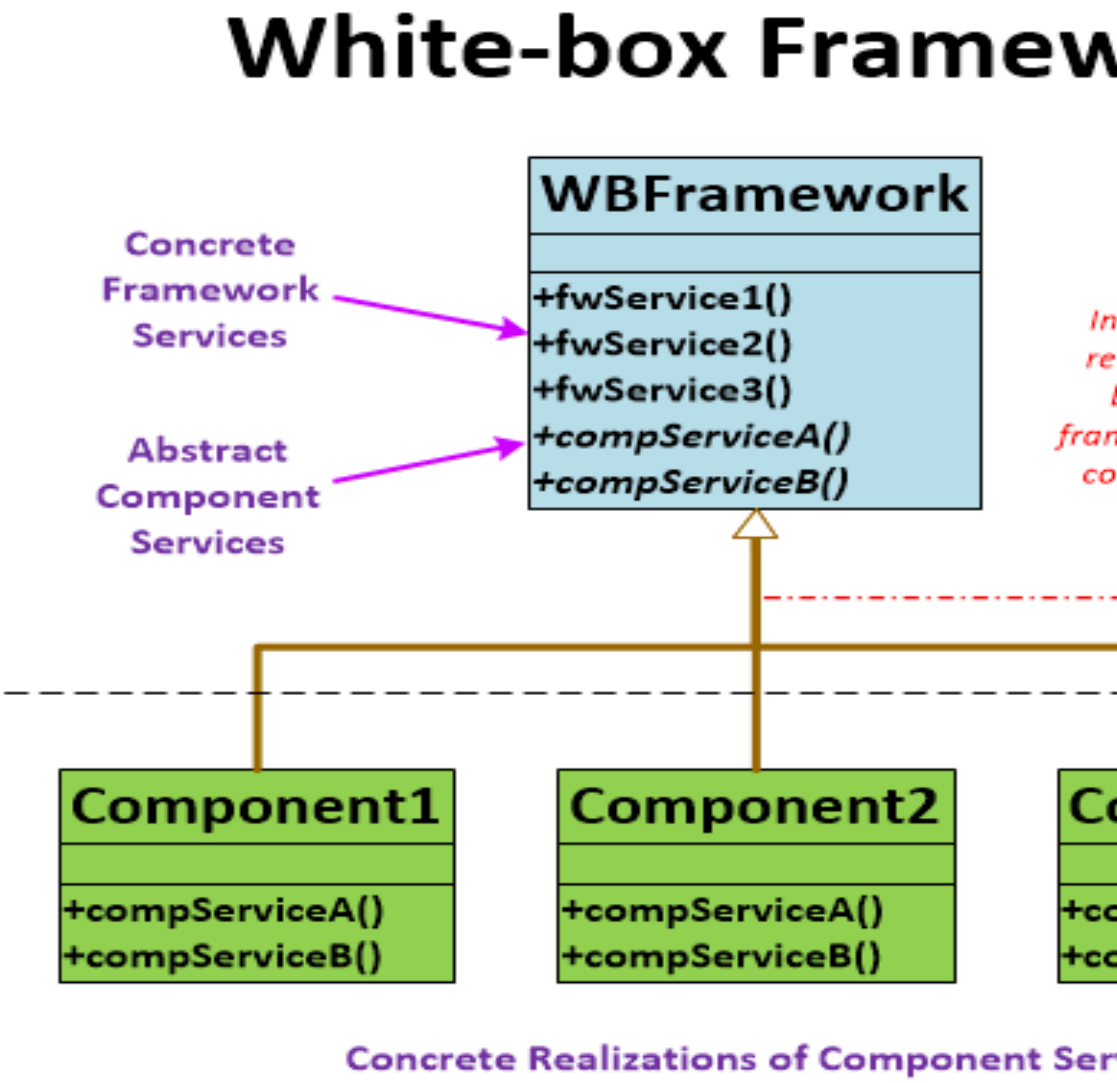


Black-box vs. White-box Frameworks

Component-framework systems come in two basic "flavors": white-box and blackbox. The two types differ in how the framework relates to the components.

White-box framework

- *In a white-box framework, the components have an inheritance relationship with the framework, i.e. are subclasses of the framework.*
- That is, the framework services are concrete methods that each component inherits from their superclass(es).
- The implementation of the framework is thus in the component superclass(es).
- The framework calls to the components by calling abstract methods in the superclass from the concrete framework methods.
- The concrete component classes override these abstract methods with their specific behaviors.
- This calling of abstract methods from concrete methods is called the Template Design Pattern and is characteristic of white-box frameworks.



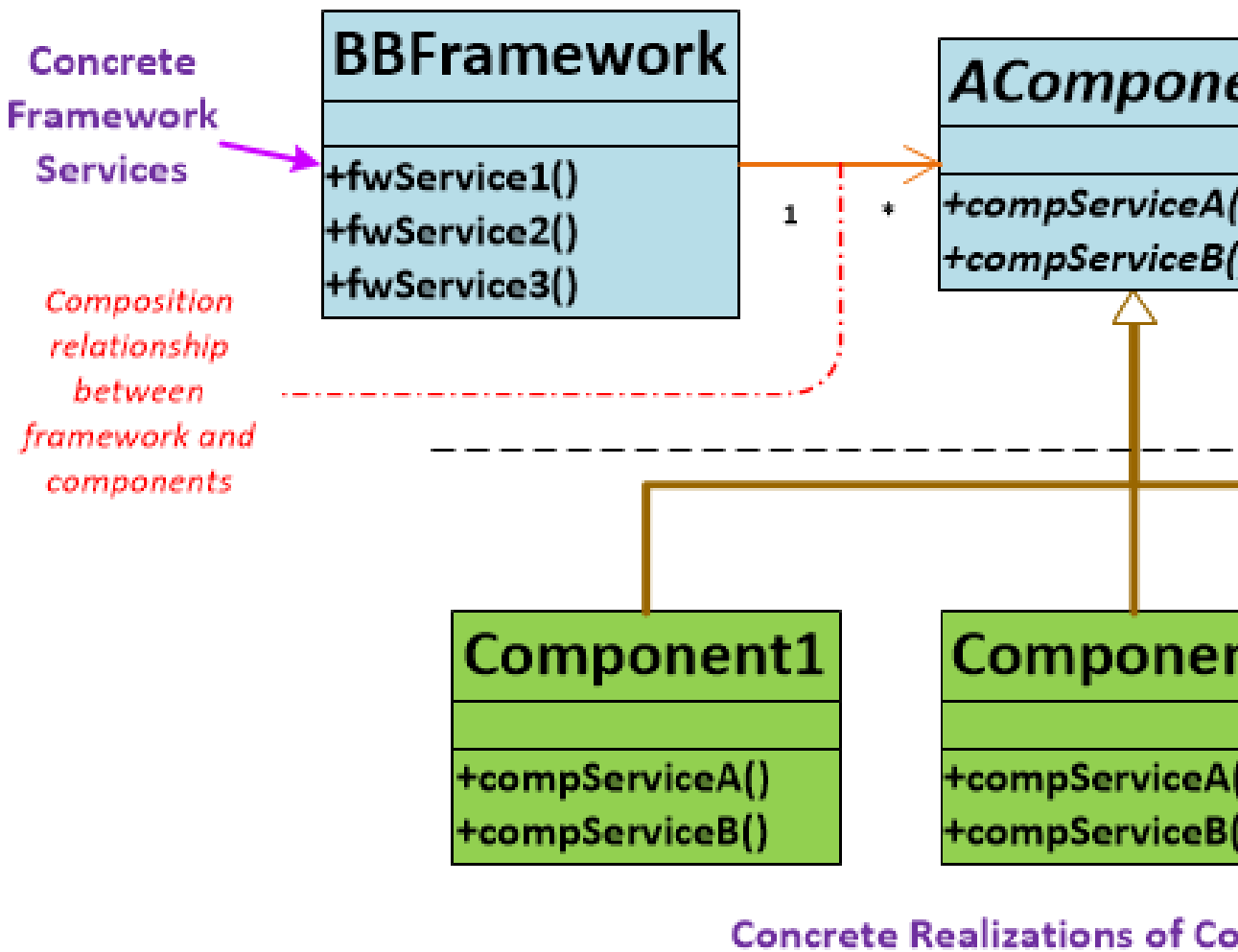
black-box framework

- *In a black-box framework, the components have a compositional relationship with the framework, i.e. are typically fields of the framework*. Here, the framework and components are completely separate objects with no inheritance relationship whatsoever.
- Typically, the compositional relationship is bi-directional so that not only can the framework call on services provided by the component, but the component in turn, can call on services provided by the framework.

Cont.: *black-box framework*

- Framework services manifest themselves as methods of the framework object while component service are methods of the component objects.
- Since the components are variant entities, the framework must reference the components as abstract entities with abstract behaviors (methods).
- This delegation by the framework to an abstract entity to fulfill some operational purpose is the Strategy Design Pattern and is characteristic of black-box frameworks.
- Often, a black-box framework is a specific, concrete entity but for systems that need upgradeability in the framework without disturbing the components, the framework may present itself to the components as an abstract entity.

Black-box Fra



Cont.:

- White-box frameworks tend to create greater coupling between the components because they are all required to have the same inheritance ancestry.
- While a disadvantage in many systems, it is advantageous in systems where tightly regulated control of the component processes is required.
- Such systems include graphical user interfaces (GUIs) and security architectures.
- White-box frameworks also tend to be easier to understand and program because of the fewer objects involved.

Cont.:

- Black-box frameworks promote maximum decoupling between the framework and its components, leading to increased flexibility and extensibility.

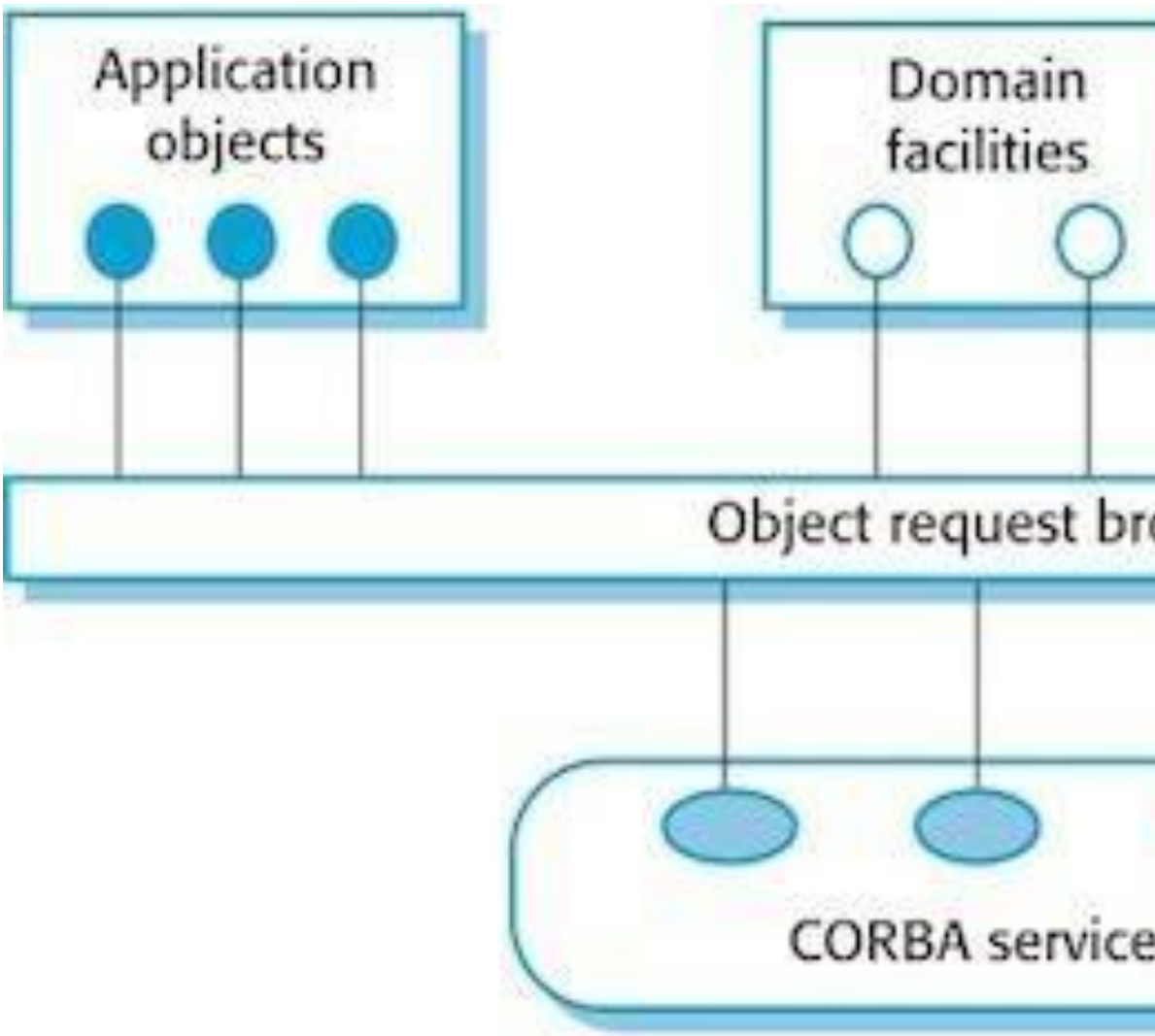
- For this reason, most large and/or highly-configurable systems are black-box frameworks.
- The disadvantage is that black-box systems can involve many abstract entities, making them more difficult to understand and program.

CORBA – Common Object Request Broker Architecture

- CORBA is a standard for an object request broker architecture that was developed by the Object Management Group in the 1990s. It was hoped that this would be widely adopted and that implementations of the standard would be available from competing vendors.
The OMG standards are available from their website.
- The OMG's vision of a distributed application is shown in Figure 1, which I have adapted from Siegel's diagram of the Object Management Architecture (Siegal, 1998). This proposes that a distributed application should be made up of a number of components:

cont.:

- Application objects that are designed and implemented for this application.
- Standard objects that are defined by the OMG for a specific domain. These domain object standards cover finance/insurance, electronic commerce, healthcare, and a number of other areas.
- Fundamental CORBA services that provide basic distributed computing services such as directories, security management, etc.
- Horizontal CORBA facilities such as user interface facilities, system management facilities, etc. The term horizontal facilities suggests that these facilities are common to many application domains and the facilities are therefore used in many different applications.



Cont.:

- The CORBA standards cover all aspects of this vision. There are four major elements to these standards:
- An object model for application objects where a CORBA object is an encapsulation of state with a well-defined, language-neutral interface described in an IDL (Interface Definition Language).
- An object request broker (ORB) that manages requests for object services. The ORB locates the object providing the service, prepares it for the request, sends the service request and returns the results to the requester.
- A set of object services that are general services likely to be required by many distributed applications. Examples of services are directory services, transaction services and persistence services.
- A set of common components built on top of these basic services that may be required by applications. These may be vertical domain-specific components or horizontal, generalpurpose components that are used by many applications.

Cont.:

- The CORBA object model considers an object to be an encapsulation of attributes and services, as is normal for objects.
- However, CORBA objects must have a separate interface definition that defines the public attributes and operations of the object.
- CORBA object interfaces are defined using a standard, languageindependent interface definition language (IDL).
- If an object wishes to use services provided by another object then it accesses these services through the IDL interface. CORBA objects have a unique identifier called an Interoperable Object Reference (IOR).
- This IOR is used when one object requests services from another.

These requests are processed by the object request broker.

Lecture 7:

COMPONENT BASED SOFTWARE ENGINEERING

Component Engineering Process

DEFINITION

- The component engineering process involves a series of activities used to identify, analyze, specify, develop, document, test, store, deploy, and maintain software components for building software systems.

1. Component identification

- Component identification is the process of determining which software components are required to satisfy the system's requirements.
- Activities:
- Analyze system requirements.
- Identify the functions the system must perform.
- Search for existing reusable components.
- Determine which components need to be developed.
- Example: In a mobile banking application, identifying login, account management, and money transfer components.

2. Component analysis

- Component analysis is the process of examining identified or available components to determine whether they are suitable for use in the intended system.
- Activities:
- Evaluate component functionality.
- Check compatibility with the system architecture.
- Assess reliability, security, performance, and reusability.
- Examine dependencies, licensing, and limitations.
- Example: Checking whether an existing authentication component supports the security requirements of a mobile banking application.

3. Component specification

- Component specification is the process of defining the requirements, responsibilities, interfaces, and expected behaviour of a software component.
- A component specification describes what a component should do and how other components can interact with it.
- Activities:
- Define component functions.
- Specify inputs and outputs.
- Define interfaces and dependencies.
- Establish performance and security requirements.
- Example: Specifying that a login component accepts a username and password and returns an authentication result.

4. Component development

- Component development is the process of designing, coding, and implementing a software component according to its specification.
- Activities:
- Select appropriate programming languages and tools.
- Design the component's internal structure.
- Write and compile the code.
- Implement required interfaces.
- Follow coding standards and best practices.

- Example: Programming a transaction component that transfers money between bank accounts.

5. Component documentation

- Component documentation is the process of preparing information that explains a component's purpose, functionality, interfaces, usage, and limitations.
- Typical documentation includes:
- Component description and purpose.
- Installation and configuration instructions.
- Interface and API details.
- Usage examples.
- Dependencies and version information.
- Known limitations and troubleshooting guidance.
- Example: Documenting how developers can call a payment component's API and interpret its responses.

6. Component testing

- Component testing is the process of verifying that an individual software component operates according to its specification.
- Types of testing may include:
- Unit testing: Checks individual functions or units within a component.
- Functional testing: Verifies that the component performs its intended tasks.
- Interface testing: Checks communication through the component's interfaces.
- Security testing: Checks for vulnerabilities.
- Performance testing: Evaluates response time and resource usage.
- Example: Testing whether a payment component correctly processes valid payments and rejects invalid requests.

7. Component repositories

- A component repository is a centralized or distributed storage facility where reusable software components and their related information are stored, organized, and retrieved. They function as centralized or distributed databases where verified software components are stored, indexed, and made accessible to development teams
- A repository may contain:
- Component source code or compiled packages.

- Documentation.
- Version information.
- Dependencies.
- Interface specifications.
- Testing results and metadata.
- Examples: GitHub repositories, Maven Central for Java packages, and npm for JavaScript packages.

Cont.: Importance:

- Promotes component reuse.
- Makes components easier to locate and retrieve.
- Supports version management.
- Reduces duplication of development effort.

8. Component deployment

- Component deployment is the process of installing, configuring, and making a software component available in its intended operating environment.
- Activities:
- Package the component.
- Configure dependencies and settings.
- Install it in the target environment.
- Integrate it with other components.
- Verify its operation after deployment.
- Example: Deploying a payment service to a production server so that a mobile banking application can use it.

9. Component maintenance

- Meaning: Component maintenance involves modifying, updating, repairing, and improving software components after they have been developed or deployed.
- Types of maintenance include:
  - Corrective maintenance: Fixes defects and errors.
  - Adaptive maintenance: Adjusts a component to changes in its operating environment.
  - Perfective maintenance: Improves performance, usability, or functionality.

- Preventive maintenance: Reduces the likelihood of future problems.

▶ Example: Updating an authentication component to address a security vulnerability or support a new authentication method

Lecture 8:

COMPONENT BASED SOFTWARE ENGINEERING

Component Selection, Qualification and Adaptation

Component repositories

▶ A component repository is a **centralized, organized store of reusable components and their related information**, such as specifications, documentation, test results, and usage history. It supports the **identification (search) and retrieval** of components, making reuse practical instead of ad hoc.

▶ Without a repository, developers often don't know what already exists, so they rebuild components that could have been reused. **Why repositories are needed**

▶ Make reusable components **discoverable**

▶ Reduce duplicated development effort

▶ Store **metadata** needed for qualification and selection

▶ Manage **versions** and dependencies

▶ Improve quality by sharing tested, certified components

▶ Support governance, licensing, and security control

component selection criteria

▶ Selection is the process of identifying and choosing the best candidate component from available internal or external sources.

▶ It is the activity of identifying, evaluating, and choosing the reusable components (COTS, open source, or in-house) that best satisfy a system's requirements and architecture.

▶ It is one of the core steps in component-based development, because a poor choice affects integration effort, quality, cost, and long-term maintenance.

▶ The goal is to find a component that matches the system's requirements while minimizing cost and development time.

Cont.:

- **Requirements Matching:** Evaluating functional requirements (what the component does) and non-functional requirements (performance, scalability, security, and license constraints).
- **Market Analysis:** Identifying available open-source, commercial, or legacy in-house alternatives.
- **Viability Assessment:** Checking the vendor's reputation, community support (for open source), documentation quality, and release history to avoid abandoned software.
- **Trade-Off Analysis:** Balancing the cost of buying/adopting vs. building from scratch, alongside long-term maintenance implications.

The Selection Process

- Component selection is intertwined with architectural design. The choice of architecture affects which components can be used, and the availability of components can influence architectural decisions .

A structured approach typically follows these stages:

- **Requirement Analysis**: Capture system requirements and define the component requirements needed to meet them .
- **Make-or-Buy Decision**: Determine whether to develop a custom component or reuse an existing one. This strategic decision is typically made early to minimize rework .
- **Candidate Identification**: Search component libraries or marketplaces using functional attributes (what the component does) .
- **Filtering & Evaluation**: Reduce the candidate set using condition attributes (implementation specifics) and then evaluate the survivors against a detailed set of criteria .
- **Integration & Verification**: Integrate selected components and verify that the composed system behaves as intended

Common challenges

- Incomplete or inaccurate vendor documentation
- Mismatch between component interfaces and system needs (architectural mismatch)
- No access to source code, which makes testing and debugging hard
- Vendor lock-in and unexpected end-of-life
- Conflicting criteria, such as the cheapest option being the least reliable

Evaluation Criteria

- Evaluation criteria are the measurable standards used to judge whether a candidate component is suitable. They are usually grouped into categories, and each is given a weight based on project priorities.
- **1. Functional criteria**
- **Functional completeness:** Does it cover all required features?
- **Functional correctness:** Does it produce accurate results under the stated rules?
- **Appropriateness:** Does it avoid unneeded features that add complexity?

2. Quality criteria (based on ISO/IEC 25010)

ATTRIBUTE	WHAT TO EVALUATE
Reliability	Maturity, failure rate, fault tolerance, recoverabili
Performance efficiency	Response time, throughput, memory and CPU usa
Security	Authentication, encryption, vulnerability history,
Maintainability	Modularity, code quality, ease of change and testi
Portability	Runs across required platforms, OS, and environ
Usability	Ease of learning, API clarity, configuration effort
Compatibility	Interoperability and co-existence with other comp

3. Architectural and integration criteria

- Quality and clarity of **provided and required interfaces**
- Support for standards and protocols (REST, SOAP, JDBC, etc.)
- **Coupling and cohesion** of the component
- Degree of **architectural mismatch** with the system
- Effort required for wrappers, adapters, or glue code
- Customizability and extensibility (configuration, plug-ins)

4. Non-technical (business) criteria

Criterion	Questions to ask

Cost	License fees, training, integration, maintena
Licensing	Proprietary vs. open source, legal restriction
Vendor viability	Reputation, financial stability, longevity
Support	SLAs, response times, community activity,
Lock-in risk	How hard is it to replace later?

5. Documentation and testability criteria

- Completeness and accuracy of documentation
- Availability of examples, tutorials, and test suites
- Certification or third-party evaluation results
- Ability to test the component in isolation

6. Risk criteria

- Black-box behavior with no source access
- Dependence on third-party updates or hidden dependencies
- Security exposure and compliance issues (e.g., GDPR)
- Risk of end-of-life or abandonment

Component qualification

- Component qualification is the process of determining whether a pre-existing component (often Commercial Off-The-Shelf, or COTS) is "fit for use" in a specific system context .
- It goes beyond simple selection: selection identifies candidates, while qualification proves a chosen candidate can be safely and reliably integrated into your architecture.
- The fundamental goal is to ensure the component satisfies system requirements with high quality and reliability .
- This is particularly critical in safety-critical domains like aerospace, automotive, and nuclear power, where an unqualified component could lead to catastrophic failure .
- Standards like DO-178C (avionics) and ISO 26262 (automotive) mandate rigorous qualification for software libraries and components used in critical paths

Qualification activities

1. **Documentation review:** Check specifications, interface descriptions, and known limitations.
2. **Vendor and source assessment:** Reputation, support, update history, and license terms.
3. **Interface compatibility analysis:** Compare the component’s interfaces with the architecture’s needs.
4. **Prototyping and trial integration:** Build a small test integration to expose mismatches early.
5. **Testing and benchmarking:** Functional tests, performance tests, and security checks, often in a sandbox.
6. **Risk assessment:** Identify black-box risks, hidden dependencies, and lock-in.
7. **Decision:** Mark the component as qualified, qualified with adaptation, or rejected.

Aspects examined during qualification

Aspect	What is checked
Application programming interface (API)	Are the provided and required interfaces comple
Development and integration tools	What tools, languages, and platforms are needed
Run-time requirements	Resource usage, memory, performance, and resp
Services required	Dependence on OS services, middleware, databa
Security features	Access control, authentication, encryption, vuln
Embedded design assumptions	Hidden assumptions about data formats, error ha
Exception handling	How errors and failures are reported and recover

Possible outcomes

- **Qualified:** Meets requirements as is.
- **Conditionally qualified:** Usable after adaptation (wrappers, glue code, configuration).
- **Not qualified:** Fails mandatory criteria and is rejected.

Challenges

- Incomplete or inaccurate documentation
- No source code access for inspection
- Architectural mismatch discovered late
- Limited vendor information or test evidence
- Time and cost of thorough testing

Component Adaptation

- Component adaptation is the process of modifying or adjusting a software component so that it can work correctly with other components or meet the requirements of a particular software system.
- It is necessary when a reusable component does not fully match the system's required interfaces, data formats, behavior, or operating environment.
- Example: A mobile application uses a payment component that accepts amounts in US dollars, but the application operates in Kenyan shillings. The component may need to be adapted to handle the required currency format.

Why it is needed

- **Interface mismatch:** different method names, parameters, or data formats
- **Functional gaps:** missing features or extra unwanted behavior
- **Architectural mismatch:** conflicting assumptions about control flow, data sharing, or concurrency
- **Platform or protocol differences:** e.g., SOAP vs. REST, different data types
- **Non-functional gaps:** performance, security, or logging requirements not met

Component Mismatches

A mismatch is a **conflict between what a component offers or assumes and what the system or other components expect**.
Mismatches are the main reason adaptation is needed, and they are the main cause of integration cost and failure in CBA In a perfect world, software components would plug into each other like Lego bricks. In reality, components often suffer from **mismatches** because they were built by different vendors, at different times, or for different systems.
These mismatches generally fall into three categories:…….

Cont.: Component Mismatches

These mismatches generally fall into three categories:

- **Signature Mismatches:** Differences in method names, data types, or the number of parameters.
- **Behavioral Mismatches:** Differences in the expected order of operations or execution logic.
- **Semantic Mismatches:** Differences in what the data actually means (e.g., one component measures temperature in Celsius, the other in Fahrenheit).

Why they occur

- Components are built independently, by different teams or vendors, for different contexts

- Each component embeds **assumptions** about its environment that are rarely documented
- Components are often black boxes, so assumptions are discovered late
- Standards and technologies differ between components

Common Adaptation Techniques

1. Wrappers

1. A wrapper is a piece of code that **encloses a component and presents a new interface to the outside world**, while delegating the real work to the component inside. Clients talk only to the wrapper; the component stays untouched. It is the most common **black-box** adaptation technique, especially for COTS and third-party components.

- Think of it as a protective shell: whatever goes in or out passes through the wrapper, which can translate, filter, or enrich it

Why use a wrapper

- The source code is unavailable or must not be modified
- The component's interface doesn't match what the system expects
- You need to add behavior (security, logging, validation) the component lacks
- You want to **isolate vendor-specific details** so the component can be replaced later
- You need to expose a legacy component in a modern form (e.g., a COBOL routine as a REST service)

What a wrapper can do

- **Translate** interfaces, data, and protocols
- **Add** cross-cutting concerns such as security, logging, or error handling
- **Restrict** access to only the functions you want exposed
- **Combine** several calls into one simpler operation
- **Isolate** the system from changes in the component

Advantages and disadvantages

ADVANTAGES	DISADVANTAGES
No source changes needed	Extra layer adds some p
Preserves vendor upgrades and support	More code to write, test
Isolates vendor lock-in	Cannot fix deep interna
Adds missing features such as security or logging	Hidden assumptions ins
Makes components replaceable	Many wrappers increas complexity

Cont.: Common Adaptation Techniques

2. Superimposition / Aspect-Oriented Adaptation

- This is a more sophisticated, **"gray-box"** technique. It allows behavior to be "superimposed" onto a component. Instead of just wrapping the outside, the adaptation logic can intercept and modify messages at various points *within* the component's execution. This is useful for adding orthogonal concerns like logging, security, or fault tolerance across multiple method
- Superimposition is an adaptation technique where you **add new behavior on top of a component's existing behavior without changing its source code**, by "layering" extra functionality over it. The original component keeps working as before, and the superimposed code is woven in at defined points to modify, extend, or constrain what it does.
- The term comes from the academic literature on component adaptation (notably Bosch's work on "superimposition" as a way to adapt black-box components). It is conceptually close to **aspect-oriented programming (AOP)**.

Cont.: Common Adaptation Techniques

3 Adapter pattern

- The Adapter pattern lets two pieces of code work together even though their interfaces are incompatible. The adapter sits in the middle and **translates calls** from the interface the client expects (the *target*) into calls on the interface the component actually provides (the *adaptee*)
- In CBA, you usually cannot change a COTS or third-party component's source. The client code in your system already expects a particular interface. The adapter lets you:
- Keep the **client code unchanged**
- Keep the **component untouched** (so vendor upgrades and support still work) Isolate all translation logic in **one place**

Cont.: Common Adaptation Techniques

4. Configuration (Parameterization)

- Adjusting a component's behavior through **settings it already exposes**, rather than writing new code. The component was designed with variability in mind, and you simply select the options you need. **What you can adjust**
  - Connection details (database, server, ports)
  - Performance settings (cache size, thread pool, timeouts)
  - Security options (authentication mode, encryption level)
  - Logging level and format
  - Locale, currency, and date formats
  - Which optional features are active

Common form

FORM	EXAMPLES
Property / config files	application.properties, .ini, YAM XML
Deployment descriptors	web.xml, Spring XML, Kuberne

Environment variables	Database URL, API keys, featur
Runtime parameters	Constructor arguments, setter m
Admin consoles / GUIs	Vendor dashboards for turning f
Feature toggles	Enable or disable optional modu

Cont.: Common Adaptation Techniques

5. Extension (Plug-ins and Extension Points)

- Adding **new behavior** to a component through hooks the vendor deliberately provides. You write your own code, but it plugs into the component through a defined interface, so the core remains untouched.
- is a mechanism for **extending system functionality through predefined "extension points"** , rather than modifying the original component's code. It is another pattern for component adaptation, but with more emphasis on **architectural-level extensibility**
- **Extension Point**: A predefined interface or contract declared by a host component/application, specifying "where extensions are allowed" and "what shape extensions must have
- **Extension (Plug-in)**: An independently developed module that implements the extension point contract and "plugs into" the host system to add functionalit
- The key is: the host component declares the extension point but **does not need to know who will extend it**, and the plug-in implements the extension point but **does not need to know the host's internal implementation**

Common extension mechanisms

MECHANISM	HOW IT WORKS
Plug-ins / add-ons	Self-contained modules loaded by the host

Callbacks / hooks	You register functions called at specific events
Interfaces to implement	Component calls your class through an agreed interface
Subclassing (frameworks)	Override designated methods
Scripting support	Embedded scripts alter behavior
Middleware / interceptors	Code runs before or after component calls
Custom APIs / SDKs	Vendor SDK for building add-on logic

Cont.: Common Adaptation Techniques

5. ***Protocol adaptation***

- Adjusting communication procedures so components can exchange information using compatible protocols.
- Example: Using an adapter to allow a component that expects one communication protocol to interact with a service that uses another.

6. Refactoring / Re-engineering

This is a **white-box** technique that requires the component's source code. The component is structurally altered—for example, a monolithic component is "fragmented" into smaller, distributed parts to fit a new deployment architecture.

Challenges of Component Adaptation

- Adaptation may introduce new errors.
- It can increase system complexity.
- Some components may have limited documentation.
- Adaptation may reduce performance.
- Changes to a component may affect future updates or compatibility.
- The component's source code may not be available for direct modification.

Lecture 9:

Component Reuse and Reusability

Introduction

- **Component-Based Architecture (CBA)** and **Component-Based Software Engineering (CBSE)** are software paradigms explicitly designed around the core philosophies of **component reuse** and **reusability**.
- While "reusability" refers to the *design potential* of a component to be applied in different contexts, "reuse" is the *actual practice* of utilizing those components to build systems

What is code reuse and why is it important?

- In today's rapidly evolving software development landscape, code reusability is essential for producing efficient and maintainable applications. Code reusability refers to writing code in such a way that it can be reused across multiple contexts with little or no modification required. This approach saves time on repetitive tasks, shrinks codebase size, and improves maintainability.
- Code reuse is critical, as it encourages clean, modular, and efficient software. Reusing code allows developers to focus on implementing new features and functionality **instead of reinventing the wheel for each project**. Plus, code reuse lowers the likelihood of introducing bugs or inconsistencies, as developers use previously tested components that have been proven successful elsewhere.

Examples of code reuse

- Code reuse plays a vital role in creating efficient and maintainable software, but it's essential to understand the practical applications of this concept.

Good examples of code reuse

- The following are some examples of effective code reuse in practice:
- **Authentication:** Create a reusable authentication module that handles user registration, login, and session management, allowing for easy integration into multiple applications.
- **Data access layer**: Develop a reusable data access layer that abstracts database operations, allowing you to switch between different databases or implement caching without changing the application logic.
- **Logging and monitoring:** Implement a central logging library that standardizes how all your applications (web, mobile, etc.) log events and capture metrics, facilitate consistent log analysis, and monitor across your projects.

Making existing components reusable

- Making existing components reusable is the process of identifying, modifying, and improving existing software components so that they can be used in different applications or systems without developing them from scratch.
- Many organizations already have working software, such as legacy systems, libraries, and in-house modules, that were not originally designed for reuse. Making these components reusable involves identifying valuable parts of the existing software and transforming them into **well-defined, documented, and independent components** that can be integrated into new systems.
- This process is often referred to as **reuse reengineering** or **component mining (harvesting)**. It helps organizations save development time and costs, reduce code duplication, and improve software quality. It can also be less risky than rewriting software from scratch because the existing code may already have been tested and proven in practical use.
- **Example:** A developer extracts and adapts a login module from an existing school management system so that it can also be used in a banking application and an ecommerce website.

Why existing code is hard to reuse

PROBLEM	EXAMPLE
Tight coupling	Modules depend on global variables and each other's in
Hidden assumptions	Hardcoded file paths, single-user or single-thread logic
Poor interfaces	No clear API; behavior reached through shared data
Mixed concerns	Business logic mixed with UI and database code
Missing documentation	Original developers have left
Outdated technology	Old languages, platforms, or protocols
Low cohesion	One module does many unrelated things [6]

Identifying Candidates for Reuse

Before refactoring, you need to find the right parts of your codebase. The key indicators are:

- **Duplicate Code**: Repeated patterns across files signal a clear opportunity for centralization.
- **High Cohesion, Low Coupling**: Code that is self-contained and doesn't rely heavily on the rest of the system makes an excellent component.
- **Independent Lifecycle**: Code that can be released, versioned, or owned by a separate team independently is a strong candidate.
- **Stable Code**: Parts of the codebase that rarely change are safer to extract and reuse without fear of breaking changes

The Transformation Process (Legacy to Component)

- For existing software (especially legacy systems), making it reusable often follows a structured re-engineering path:
- **Reengineering Phase**: Analyze the domain and structure of the existing software to understand its functions and dependencies. You extract "candidate components" that show high reusability potential, often measured by how often they are called in the system.
- **Refactoring Phase**: This is where the actual restructuring happens. You modify the internal structure of the candidate code without changing its external behavior, making it flexible and robust. The goal is to create a clean, component-like structure.
- **Certification/Qualification**: After refactoring, the new component is tested and validated to confirm it meets the quality standards and can be treated as a true reusable asse

Techniques for Making Existing Components Reusable

1. Component adaptation – Adjusting an existing component to meet the requirements of another system.
2. Standardizing interfaces – Providing consistent interfaces so that different applications can interact with the component easily.
3. Reducing dependencies – Minimizing unnecessary reliance on other components so the component can operate in different environments.
4. Parameterization – Allowing users to configure a component through parameters instead of changing its source code.
5. Generalization – Designing a component to handle different situations rather than solving only one specific problem.
6. Encapsulation – Hiding internal implementation details and exposing only the necessary operations through public interfaces.
7. Documentation – Providing clear instructions on the component's purpose, interfaces, configuration, dependencies, and usage.

Key Technical Steps for Reusability

To make a component truly reusable across different contexts, specific technical attributes must be added or enhanced:

- **Define Explicit Interfaces**: A reusable component needs a clearly defined public API. This means identifying the **Provided Interfaces** (what it offers) and **Required Interfaces** (what it depends on).

- **Add a Configuration Interface**: Instead of hard-coding behavior, provide mechanisms (like parameters or config files) that allow users to customize the component's behavior without modifying its underlying code.
- **Ensure Independence**: The component should be designed to minimize dependencies on specific environments or other modules, making it easy to integrate into new systems.
- **Create a Wrapper (if needed)**: If the code cannot be directly modified, a "wrapper" component can be created. This wrapper encapsulates the original code and exposes a standardized interface, effectively making the legacy logic reusable as a black bo

Techniques for improving reusability

- **Encapsulation:** Hide internal data and expose only operations.
- **Dependency injection:** Pass in dependencies rather than hardcoding them.
- **Separate interface from implementation:** Program to interfaces.
- **Parameterization:** Replace constants and assumptions with configuration.
- **Remove global state:** Replace shared variables with explicit parameters.
- **Standardize data formats and protocols:** Use common formats such as JSON or XML.
- **Add error handling and logging:** Make behavior predictable and diagnosable.
- **Improve naming and documentation:** Make purpose and usage clear.

Challenges

- Reverse engineering poorly documented code is slow and errorprone
- Hidden dependencies surface late
- Risk of **breaking existing behavior** without good tests
- Reengineering cost may exceed expectations
- Organizational resistance and lack of ownership
- Reusable versions can become over-generalized, complex, and slow

Main strategies

Strategy	Description
Wrapping (blackbox)	Leave code unchanged and expose it through a new interfa
Refactoring (whitebox)	Restructure the code internally to improve modularity and
Reengineering / rewriting	Re-implement the functionality with a clean design while k behavior
Service-enabling	Expose functionality as a web service or API
Extracting from a monolith	Gradually carve out modules (e.g., strangler pattern)
Packaging as library or container	Bundle code with dependencies for deployment

Development for Reuse (Designing Components for Reuse)

- Development for reuse is the process of **designing and building components from the start so that they can be reused in many systems**, rather than serving only one application.
- It is the producer side of CBSE: one team builds general, well-documented, highquality components and publishes them to a repository, and other teams later develop *with* reuse by consuming them.
- It contrasts with the previous topic. Making existing components reusable fixes code after the fact. Development for reuse builds reusability in from the beginning
- In component-based software engineering (CBSE), developers consider reusability from the beginning by designing components that are modular, independent, configurable, well-documented, and accessible through clearly defined interfaces

Implications

Development for reuse is not just a technical activity; it has significant organizational implications:

- **Higher Upfront Cost**: Reusable components are more expensive to build than their single-use equivalents because of the additional effort in generalization, testing, and documentation. This cost should be borne as an **organizational investment**, not charged to a single project.
- **Certification and Trust**: Consumers of a reusable component must be able to trust it without access to its development process. This necessitates rigorous testing, documentation, and often formal certification of the component's behavior.
- **Maintenance Lifecycle**: Components have a lifecycle. If they are not kept general and mature enough to meet evolving requirements, they can degenerate into assets that are difficult to use and maintain, eventually becoming obstacles to efficient reus

Design principles for reusable components

PRINCIPLE	MEANING
High cohesion	Component does one well-defined job
Low coupling	Few dependencies on other components a
Encapsulation	Hide internals; expose only the interface
Standardized interfaces	Use common protocols, formats, and con
Abstraction and generality	Solve the general problem, not one specif
Parameterization	Use configuration instead of hardcoded v
Independence	Avoid reliance on global state or a particu
Explicit dependencies	Declare everything the component requir
Substitutability	Components with the same interface can

Advantages

- Reduces future software development time and costs.

- Minimizes duplication of code.
- Promotes consistency across applications.
- Improves software maintainability.
- Encourages the development of reliable, reusable components.

Development with Reuse (Building Systems from Existing Components)

- Development with reuse is the **consumer side of Component-Based Software Engineering (CBSE)**. It is the process of building a new software system by identifying, evaluating, qualifying, selecting, adapting, and integrating existing software components instead of developing all the required functionality from scratch.
- The central idea is to **reuse what already exists rather than reinventing it**. Developers search for components that meet system requirements, assess their suitability, adapt them where necessary, and assemble them into the target system. This shifts the focus from writing all the code to selecting, qualifying, adapting, and integrating existing software assets.
- During this process, developers consider available components when defining system requirements and architecture, resolve incompatibilities through adaptation techniques, and test the integrated system. The components must also be maintained as system requirements, technologies, and component versions change.

Types of reuse sources

Source	Advantages
In-house components	Familiar, controllable, source available
Open source	Free, flexible, community support
COTS (commercial)	Vendor support, polished
Cloud services / APIs	No hosting effort, scalable

Advantages

- Shorter development time and faster time to market
- Lower cost, since design, coding, and testing effort is shared
- Higher quality, because components are already tested in other systems
- Less maintenance of custom code
- Developers focus on what is unique to the application

Risks

Components may not fully fit (mismatch)

Integration and adaptation effort can be underestimated

Vendor lock-in and licensing problems

Dependence on third-party updates and support

Security vulnerabilities in third-party code

Loss of control over component evolution

Key Challenges

Development with reuse is not simply "plug and play." Several difficulties arise:

- **Finding the Right Component**: Repositories may be poorly indexed, and components may be undocumented or described in ways that don't match your needs.
- **No Perfect Match**: You will almost never find a component that exactly satisfies all requirements, forcing trade-offs and adaptation.
- **Architectural Mismatch**: The component's assumptions about its environment may conflict with your system's architecture.
- **Integration Complexity**: Components developed independently may have incompatible protocols, data models, or quality-of-service expectations.
- **Trust and Evidence**: You must trust the component without having developed it. This requires qualification evidence, documentation, and often supplier assessment.

Cont.: Engineering *with* Reuse (Application Assembly)

Activities include

- **Requirements Gathering:** The system's high-level functionality and architectural constraints are mapped out.
- **Repository Mining:** Developers search the **component repository** using keyword or semantic searches to find candidates that meet requirements.
- **Evaluation & Selection:** Found components are analyzed for compatibility (checking provided vs. required interfaces).
- **Integration:** Selected components are extracted, adapted, and assembled into the application layout. If a match isn't found in the repository, a new one is built and routed back to the production pipeline.

Component Documentation

- Component documentation is the process of preparing and maintaining information that describes a software component's purpose, interfaces, dependencies, installation, configuration, and usage.
- It enables developers to understand and evaluate a component, determine its suitability for reuse, and integrate and maintain it without examining its internal source code. This is particularly important for **black-box components**, whose internal implementation is hidden or unavailable. In such cases, documentation is a primary source of

information for assessing the component's functionality, compatibility, reliability, and suitability for reuse.

Importance of documentation

- Supports identification and search (people must find it)
- Enables qualification and selection without reading source code
- Reduces mismatches by exposing hidden assumptions
- Lowers integration and adaptation effort
- Supports maintenance, upgrades, and replacement
- Builds trust in components from other teams or vendors

What Component Documentation Includes

The documentation structure typically covers four logical categories:

- **1. Basic Information** — The component's identity and architectural role:
  - Component name, identification, and type
  - Design rationale and objectives (why it exists)
  - Functional specification (inputs, outputs, behavior, exceptions)
  - Interface descriptions (provided and required interfaces)
  - Composition details (internal vs. external, integration modes)
- **2. Detailed Information** — Technical and operational specifics:
  - Technical implementation details
  - Installation and setup guide
  - Constraints and restrictions (export, security, environment, compiler, portability)
  - Modifiability, performance characteristics, safety and security properties

Cont.:

3. **Acceptance Test** — Evidence that the component meets its stated claims
4. **Support and Additional Information** — Administrative and lifecycle data:
   - Procurement, pricing, warranties
   - Known bugs and limitations
   - Version history and change log
   - Licensing and legal information

Component certification

- Component certification is the process by which an independent party provides a written guarantee that a software component conforms to specified requirements and quality attributes . It is a crucial mechanism for building trust in the component marketplace, allowing system integrators to select parts with confidence.
- The primary goal of certification is to improve trust in components, especially when they are developed by third parties . It moves beyond simple functional testing to provide an objective, third-party assessment of a component's properties.
- A key modern perspective is **claim-based certification**. This means certification should not be limited to just tracing a component's origin or testing its basic functions. Instead, it should be able to certify **any claim** a supplier makes about the component. This is critical for extrafunctional properties like real-time performance, memory usage, or security, which system integrators need to verify for predictable assembly

Types of certification

TYPE	DESCRIPTION
First-party	The producer certifies its own component (weakest ass
Second-party	The consuming organization certifies it internally
Third-party	An independent body certifies it (strongest assurance)
Product certification	Evaluates the component itself, by testing and analysis
Process certification	Evaluates the development process (e.g., CMMI, ISO 9
Model-based / formal	Uses proofs or model checking to establish properties

3. Component Certification Process

- Define certification criteria: Establish the functional, technical, quality, and security requirements.

- Submit the component for assessment: Provide the component and relevant documentation.
- Review documentation: Examine specifications, interfaces, dependencies, and usage instructions.
- Conduct testing: Perform appropriate functional, compatibility, performance, reliability, and security tests.
- Evaluate results: Compare the evidence obtained with the defined certification criteria.
- Make a certification decision: Approve the component if it meets the requirements or reject it if it does not.
- Issue certification and maintain records: Record the outcome, scope, applicable version, and any limitations. Recertification may be needed after significant changes.

Advantages of Component Certification

- Increases confidence in component quality.
- Reduces the risk of using unsuitable components.
- Supports safer software reuse.
- Helps developers compare components against established standards.
- Can reduce duplicated testing when certification evidence is accepted for the intended context.
- Improves accountability through documented assessment results.

Challenges of Component Certification

- Certification can be costly and time-consuming.
- Testing may not reveal every possible defect or security vulnerability.
- Certification may apply only to a particular component version or operating environment.
- Components may change after certification.
- Different organizations may use different certification criteria.
- Hidden source code may limit the depth of assessment

Characteristics of a good reusable component

- **Useful:** meets a real, recurring need
- **Well-defined:** clear purpose and boundary
- **Documented:** specification, examples, assumptions

- **Tested and reliable:** with published test results
- **Portable:** works across required platforms
- **Understandable:** easy to learn and integrate
- **Adaptable:** configurable and extensible
- **Maintainable:** versioned, supported, and owned
- **Secure and compliant:** vetted for vulnerabilities and licensing

Lecture 10:

COMPONENT BASED SOFTWARE ENGINEERING

Component Composition, Integration and COTS

Component composition

- Component composition is a design pattern where you build complex user interfaces by combining smaller, independent, and reusable components like Lego bricks. Instead of writing a single monolithic component bloated with dozens of props and conditional checks, you distribute responsibilities across cooperating parent and child component
- Component composition is the process of combining two or more software components to form a complete software system or application.
- In component-based architecture, each component performs a specific function and interacts with other components through defined interfaces.
- The aim is to assemble components so that they work together to satisfy the requirements of a software system.

Methods of component composition

1. Sequential composition: Components are connected in a sequence, where the output of one component becomes the input of another. Example: A component validates data before another component processes it.
2. Hierarchical composition: Components are organized into levels, where a higher-level component coordinates lower-level components. Example: An order management component coordinates payment and delivery components.
3. Additive composition: Components are combined to add new functionality to an existing system. Example: Adding a notification component to an existing mobile application.

4. Parallel composition: Components perform different tasks concurrently or independently.

Example: After an order is placed, separate components process inventory updates and customer notifications.

Note: These are common ways to describe component composition; terminology and classifications may differ across course materials.

Steps involved in component composition

- Identify the system requirements.
- Identify or select suitable software components.
- Examine component interfaces and dependencies.
- Connect compatible components using appropriate connectors.
- Integrate the components into a complete system.
- Test the integrated system to ensure that the components work correctly together.

Advantages of component composition

- Reusability: Existing components can be reused in different applications.
- Reduced development time: Developers do not need to build every function from scratch.
- Maintainability: Individual components can be modified independently when interfaces remain compatible.
- Flexibility: Components can be replaced or added to extend functionality.
- Improved productivity: Development work can be divided among different teams.

Component Integration

- **component integration** is the process of assembling independent, reusable software units into a single cohesive system.
- Because these components are often built by different teams or thirdparty vendors, integration focuses heavily on ensuring they can successfully communicate, share data, and interoperate without conflicting

Objectives of Component Integration

- To combine individual software components into a complete system.

- To enable communication between different components.
- To ensure that components work together as expected.
- To identify and resolve compatibility problems.
- To promote software reuse and reduce development time.
- To ensure that the integrated system meets user requirements.

Cont.:

Integration covers several distinct concerns:

- **Interfaces** — Matching APIs, data formats, protocols, and contracts between components. Mismatches here (type differences, calling conventions, unit mismatches) are the most common source of integration bugs.
- **Wiring/glue code** — The adapters, connectors, and orchestration logic that let components talk to each other without modifying the components themselves.
- **Lifecycle & state** — Managing initialization order, shared state, startup/shutdown sequencing, and dependency resolution.
- **Non-functional concerns** — Threading/concurrency models, error propagation, transaction boundaries, logging, security, and performance across component boundaries.

Methods of Component Integration

METHOD	EXPLANATION
Top-down integration	Begins with higher-level componer
Bottom-up integration	Begins with lower-level componen
Big-bang integration	Combines most or all components a
Incremental integration	Integrates and tests components gra

The 3 Core Stages of Integration

Before components can be seamlessly woven together, developers follow a sequence of distinct actions to prepare them:

- **Component Qualification:** Assessing whether a pre-built component meets the system's functional and non-functional requirements.
- **Component Adaptation:** Modifying or wrapping the component if it does not fully align with the target architecture or interface requirements.
- **Component Composition:** The actual assembly phase where components are systematically linked together to form the integrated application

Common Integration Challenges

While integration speeds up development, it also introduces several critical bottlenecks:

- **Architectural Mismatch:** Occurs when a reusable component makes implicit, hidden assumptions about its environment (e.g., its control

model or memory management) that conflict with the overall system design.

- **Interface Incompatibility:** Differences in communication protocols, parameter counts, or data formats that require extensive glue code to fix.
- **Versioning Chaos:** Upgrades or security patches in a single thirdparty component can create a ripple effect, breaking backward compatibility with dependent parts of the system

Interface Matching

Interface matching is the process of ensuring that the interfaces of different software components are compatible so that they can communicate and exchange information correctly.

- Key activities:
- Checking whether the output of one component matches the input requirements of another.
- Ensuring that components use compatible method names, parameters, and return types.
- Verifying that communication protocols are supported by the connected components.
- Adapting incompatible interfaces using adapters or wrappers.

Example: A payment component expects an amount in Kenyan shillings, while the ordering component supplies the amount in a different format. Interface matching ensures that the components exchange the required information correctly.

Dependency management

- Dependency management in component-based engineering (CBE/CBA) is the discipline of ensuring that separately developed components can be assembled and deployed together without conflicts, missing pieces, or version clashes. It sits at the heart of integration because components are rarely self-contained — they depend on other components, libraries, runtime services, and configuration data that must all be present and compatible at deployment time.
- dependency management means identifying, declaring, controlling, and resolving what each component needs in order to work, so components can be assembled, replaced, and upgraded without breaking the system.
- Components are built independently, so at integration time you discover what each one *actually* needs: other components, libraries, runtime services, configuration, or specific versions. Unmanaged dependencies lead to

failed assembly, unpredictable behavior, and components that can't be reused or replaced.

Types of Dependencies

- **Functional dependencies:** Component A calls services (required interfaces) that component B provides.
- **Structural/build dependencies:** Libraries, frameworks, or packages needed to compile or package the component.
- **Runtime/environment dependencies:** Operating system, container, middleware, database, or configuration the component expects.
- **Data dependencies:** Shared schemas, databases, or file formats.
- **Temporal/protocol dependencies:** Required call order, startup order, or timing between components.
- **Transitive dependencies:** Dependencies of your dependencies, which you inherit without choosing.

Main Problems

PROBLEM	DESCRIPTION
Version conflicts	A needs library v1, B needs v2 (“dependency hell”)
Circular dependencies	A depends on B and B depends on A, so neither can be deployed
Hidden dependencies	Undocumented reliance on global state, environment variables,
Tight coupling	Component depends on a concrete implementation instead of an
Dependency bloat	Pulling in large libraries for small features, widening the attack s
Missing dependencies	Required component or service absent in the environment

Common Challenges in Dependency Management

- Missing components or libraries.
- Incompatible component versions.
- Circular dependencies, where components depend on each other in a cycle.
- Changes to one component affecting other components.

- External services becoming unavailable.
- Security risks arising from outdated or vulnerable dependencies.

Data compatibility

- **Data compatibility** refers to ensuring that data exchanged between components is understood consistently on both sides—despite differences in data models, formats, or timing semantics.
- Since components are treated as black boxes communicating through required and provided interfaces, each interface is equipped with a data model defining the types occurring in interface operations.
- When these data models differ, integration fails unless adaptation is applied.

Cont.:

- Data incompatibilities arise at several levels:
- **Syntactic mismatches**: Different field names, structures, or data representations (e.g., XML schemas that define the same data differently).
- **Streaming behavior**: One component passes data one record at a time (streamed), while another expects the entire dataset at once (nonstream).
- **Timing/resolution differences**: Components update data at different intervals, or data paths have varying transit times, leading to inconsistent data sets

Common Problems

- **Format mismatch:** Different serialization or file formats (XML, JSON, CSV, binary).
- **Schema mismatch:** Different field names, structures, or cardinality.
- **Type mismatch:** Precision loss, overflow, truncation (e.g., floating point for currency).
- **Encoding errors:** Garbled text, especially with non-ASCII characters.

- **Semantic mismatch:** Data parses fine but means something different to each side. This is the hardest to detect.
- **Schema evolution:** A component changes its data structure and breaks existing consumers.
- **Null and missing value handling:** One side uses null, another uses empty string or a default value.
- **Data quality and integrity:** Different validation rules, constraints, or reference data (e.g., country codes).

Activities Involved in Data Compatibility

- 1. Data type matching: Ensure that components use compatible data types when exchanging information.
- Example: One component sends a student's age as an integer, while another expects a numeric value of the same compatible type.
- 2. Data format standardization: Ensure that components use consistent formats when exchanging data.
- Example: One component uses JSON to exchange student records, and another component is configured to receive JSON.
- 3. Data structure matching: Ensure that components agree on the organization and arrangement of data fields.
- Example: A student component provides student_id, name, and course, while the receiving component expects these fields under the same agreed structure.

Activities Involved in Data Compatibility

- 4. Data conversion: Convert data from one representation to another when components use different formats or types.
- Example: Converting a date from 10/10/2026 to 2026-10-10 before sending it to another component.
- 5. Data validation: Check that exchanged data meets the receiving component's requirements.

- Example: Ensuring that a student's admission number is provided and follows the required format before saving the record.
- 6. Data consistency: Ensure that components use consistent values and meanings for shared information.
- Example: Two components should use the same agreed currency, such as Kenyan shillings (KES), when processing a student's fee payment.

Integration mechanisms

- **Integration mechanisms** are the technical means, techniques, tools, and communication patterns used to connect independently developed software components so that they can communicate, exchange data, and coordinate their behaviour to form a unified software system.
- They define *how* components interact during system assembly after their interfaces have been matched and dependencies resolved. Components may communicate within the same application or across different computers and networks.
- Since a key principle of CBA is to promote loose coupling and independent component development and deployment, selecting an appropriate integration mechanism influences the system's performance, scalability, reliability, resilience to failures, and ease of maintenance and component replacement.

Main Mechanisms

1. Direct API / Procedure Call

- An API provides a defined set of operations through which one software component can request services or exchange information with another component. One component invokes another’s interface within the same process or runtime.
- Defines how components interact, Specifies the operations and data that can be exchanged and Reduces the need for components to know each other's internal implementation.
- **Examples:** Library calls, method invocation, .NET assemblies.

2. Remote Invocation (RPC / RMI)

- RPC allows one component to invoke a procedure or operation provided by another component, potentially running on a different computer, as though it were a local call.
- Supports communication between distributed components, Hides some of the details of network communication, Requires suitable communication infrastructure and Calls cross process or network boundaries but look like local calls.
- **Examples:** gRPC, Java RMI, CORBA, DCOM.

Cont.: Main Mechanisms

3. Service-Based Integration (Web Services)

- Components are exposed as services over standard protocols with published contracts. Web services enable software components to communicate over a network using standardized web technologies and protocols.
- Allow components built using different programming languages to communicate, Commonly use HTTP for communication and May exchange data using JSON or XML.
- **Examples:** REST, SOAP, GraphQL; microservices.

4. Message-Based / Event-Driven Integration

- Components communicate asynchronously through a message broker or event bus, using queues or publish/subscribe. Message queues allow components to exchange messages asynchronously. The sending component places a message in a queue, and the receiving component processes it when available.
- Reduces the need for components to be active simultaneously, Supports communication between loosely coupled components and Helps manage temporary delays in processing.
- **Examples:** Kafka, RabbitMQ, ActiveMQ, cloud queues.

COTS (Commercial Off-The-Shelf)

- **Commercial Off-The-Shelf (COTS)** components are pre-existing, ready-made software or hardware products developed by vendors and made available to the general market for purchase, lease, or licensing. In

Component-Based Architecture (CBA) and Component-Based Engineering (CBE), these components are integrated into larger systems to provide existing functionality instead of developing everything from scratch.

- COTS products are generally designed to meet the needs of a broad range of users rather than the specific requirements of a single organization. They are typically used as supplied or configured to suit the system's needs, without modifying their internal source code, which is often unavailable to users and controlled by the supplier.

Cont.: COTS (Commercial Off-The-Shelf)

- Examples of COTS components include database management systems such as
Oracle Database and Microsoft SQL Server, payment gateways, Customer Relationship Management (CRM) systems, Enterprise Resource Planning (ERP) packages, reporting tools, and user interface component libraries.
- Integrating COTS components can reduce development effort and accelerate software delivery. However, developers must consider potential challenges, including interface incompatibility, dependency conflicts, data compatibility, security requirements, licensing restrictions, and dependence on the vendor for updates and maintenance.

COTS in Component Integration

- During integration, developers must ensure that a COTS component is compatible with the system's interfaces, dependencies, data formats, and security requirements.
- Adapters or wrappers may be necessary to resolve incompatibilities.

Integration Challenges

COTS integration is fundamentally different from custom development. As IEEE Software observed, "dealing with COTS is a high-risk activity for a variety of reasons" .

- **Architecture mismatch** is the initial manifestation that integration is anything but simple. COTS components make assumptions about architectural issues; when these assumptions conflict with your system or with other components, the simplicity of "buy not build" disappears in a cloud of integration scaffolding .

- **Glue code** becomes necessary to repair mismatched assumptions, but this code is "often ad hoc and brittle." As a consequence, COTS-based systems can be difficult to comprehend, less evolvable than intended, and less reliable than the constituent products .
- **Evolution costs** are substantial. Research with industry practitioners found that "a large amount of work is required to integrate new releases of COTS software into their systems." For systems with multiple COTS components on independent release schedules, "the cost of integrating each new release of each component becomes prohibitive" .
- **Late discovery** is common: "Many of the problems encountered with integrating COTS components cannot be determined before integration begins. The problems are of such a nature that they only appear well into the integration process

Integration Strategies

- **Treat components as black boxes** and rely on external techniques for resolving mismatches .

Since you cannot modify the component, you must adapt around it.

- **Use wrappers and glue** to allow for interface complications in future releases, "even if not strictly needed now" . Think of them as gaskets and washers between mechanical components.
- **Select for integration mechanisms** such as public APIs and scripting languages of appropriate quality when evaluating products .
- **Manage implicit dependencies**: If one COTS component depends on the presence of another, the dependency can break and the system can fail. Select components to minimize implicit dependencies and be aware they exist .
- **Plan releases strategically**: Preplan releases at approximately six-to-nine month intervals, with multiple releases simultaneously in progress, to manage the influx of vendor updates

Characteristics

- **Pre-built and pre-tested:** Already developed and used by other customers.

- **Black-box nature:** You get the interface and documentation, usually not the source code.
- **Vendor-controlled:** The vendor decides features, release schedule, and end-of-life.
- **Generic:** Designed for many customers, so it may not fit your needs perfectly.
- **Licensed:** Use is governed by commercial terms, not ownership.

Advantages of COTS

- Reduced development time: Ready-made functionality reduces the need to build software from scratch.
- Lower initial development cost: Reusing an existing product may cost less than developing an equivalent component.
- Proven functionality: Established products may have undergone extensive testing and use.
- Vendor support: Vendors may provide updates, documentation, and technical assistance.
- Faster deployment: Products can often be installed and configured quickly.

Disadvantages of COTS

- Limited customization: The product may not fully meet specific user requirements.
- Integration difficulties: Its interfaces or data formats may be incompatible with existing components.
- Vendor dependence: Continued use may depend on the vendor's pricing, support, and product availability.
- Licensing costs: Purchase, subscription, or renewal fees may be required.
- Security and compatibility risks: Updates or changes may introduce vulnerabilities or compatibility problems.